

Message Oriented Middleware

Systemy kolejkiowania komunikatów

Plan

1. Koncepcja
 - ↳ paradygmat kolejowania (punkt-punkt)
 - ↳ paradygmat publish/subscribe
2. Model systemu
3. Przykłady rozwiązań
4. JMS

Cechy MOM

- Uniezależnienie funkcjonowania składników aplikacji od dostępności informacji o interfejsach innych składników
- Uniezależnienie funkcjonowania warstwy komunikacyjnej (kanału komunikacyjnego) od działania (obecności) komunikujących się procesów $\Rightarrow$ komunikacja nieustanna
- Mechanizm komunikacji pośredniej
 - ↳ oparty na identyfikacji miejsca pośredniczącego (tzw. skrzynki, a nie adresu nadawcy/odbiorcy)
 - ↳ komunikujące się strony nie muszą znać się wzajemnie
- Łatwość wdrożenia komunikacji asynchronicznej

Koncepcja komunikacji oparta na kolejkowaniu

- Komunikacja *punkt-punkt*
- Organizacja warstwy komunikacyjnej w postaci systemu kolejek (ang. queue) {skrzynka $\equiv$ kolejka} realizowanych w oparciu o zasoby pamięci, w tym pamięci dyskowej (gwarancja trwałości na wypadek awarii $\Rightarrow$ niezawodność)
- Udostępnienie mechanizmów komunikacji polegających na:
 - ↳ umieszczeniu komunikatów w kolejkach,
 - ↳ pobieraniu komunikatów z kolejek

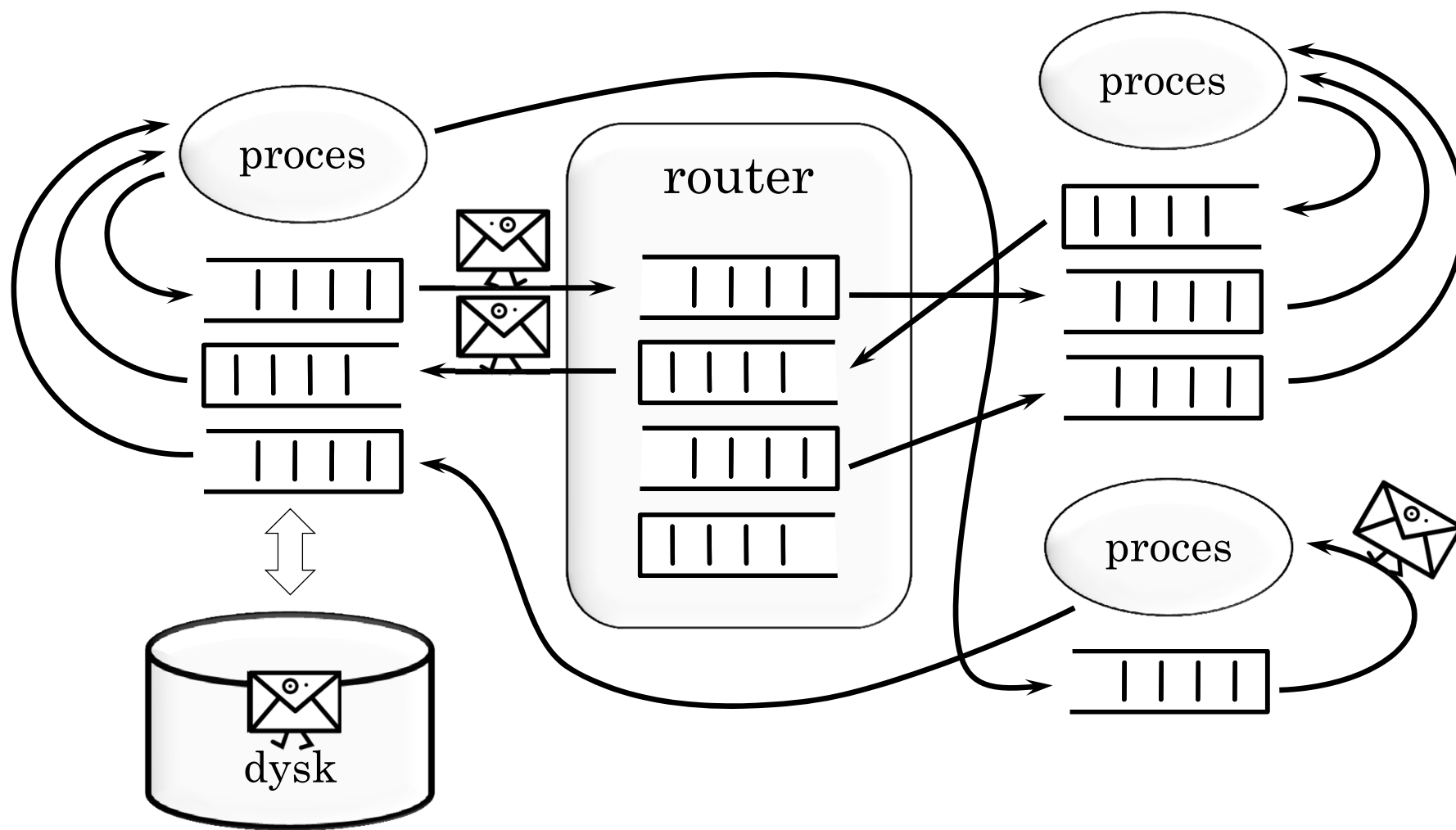
Paradygmat publish/subscribe

- Komunikacja *jeden do wielu* (potencjalnie *wielu do wielu*)
- Strona publikująca (nadawca) udostępnia treść związaną z określonym tematem (ang. topic)
{skrzynka $\equiv$ temat}
- Środowisko komunikacyjne (usługa) przekazuje treść udostępnionych wiadomości odbiorcom (subskrybentom), którzy zarejestrowali (zapisali) się wcześniej na dany temat

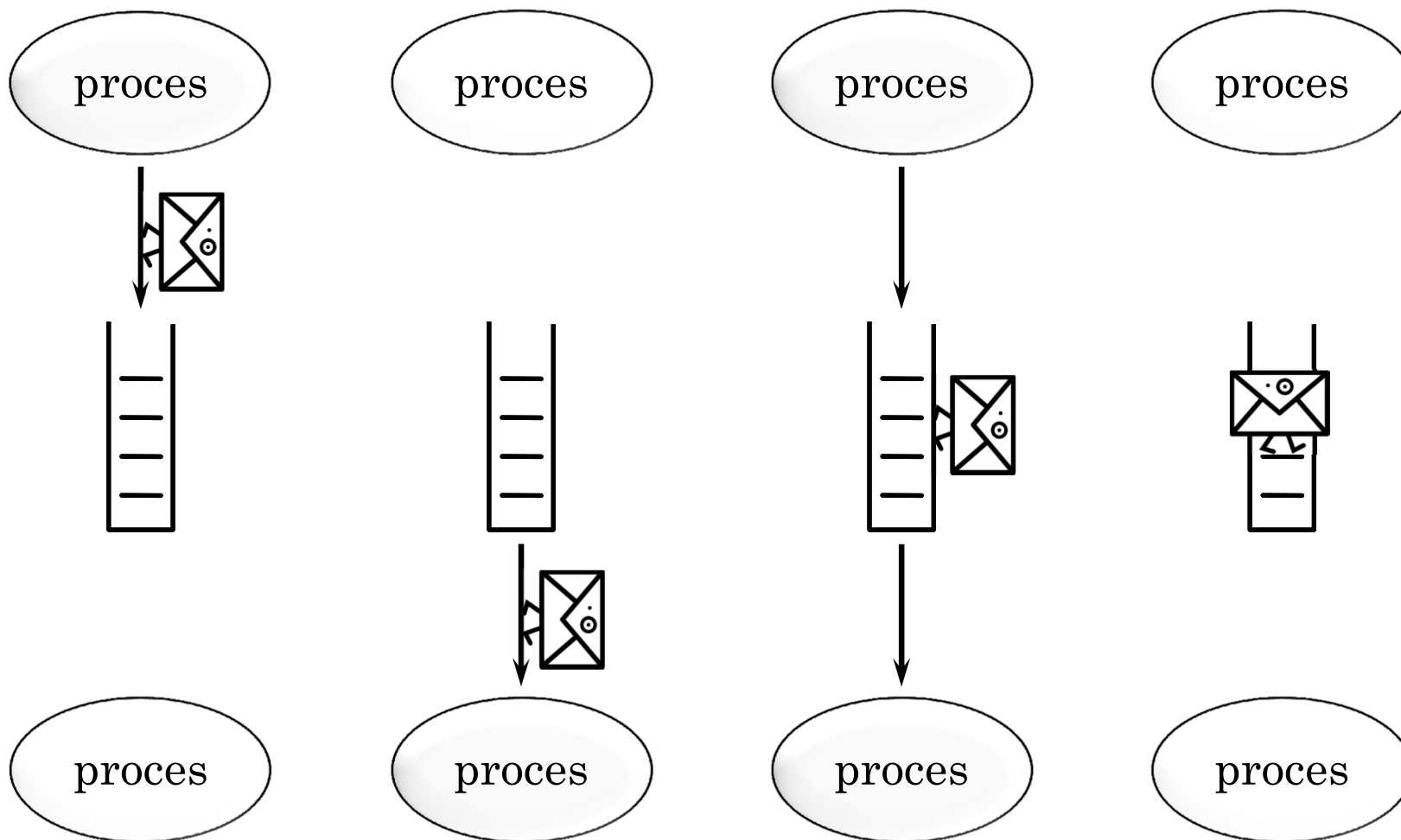
Model systemu typu MOM — podstawowe pojęcia

- Wiadomość (ang. message) — porcja danych (często z dodatkowymi własnościami) składowana w kolejce
- Kolejka (ang. queue) — miejsce przechowywania wiadomości (komunikatów) {temat $\cong$ kolejka dla wielu odbiorców, ang. multiconsumer queue}
- Proces (ang. process) — element aplikacji, zlecający operacje dotyczące wiadomości w kolejce
- Zarządca zbioru kolejek — moduł (proces na określonym węźle, dostawca, *broker*), odpowiedzialny za wykonywanie operacji na kolejkach, np. tworzenie, usuwanie, lokalizowanie kolejek, definiowanie atrybutów kolejek itp.)

Funkcjonowanie systemu kolejkowania komunikatów



Warianty komunikacji



Przykłady rozwiązań typu MOM

- ☛ WebSphere MQ (IBM MQSeries , XMS — Message Service Client)
- ☛ Microsoft Message Queuing (MSMQ)
- ☛ Java Message Service (JMS) — standard, specyfikacja interfejsu. Implementacje oparte na JMS:
 - ☛ Sun Java System Message Queue (OpenMQ — wersja open source)
 - ☛ Apache ActiveMQ
- ☛ Oracle Advanced Queueing
- ☛ RabbitMQ
- ☛ ZeroMQ
- ☛ Apache Kafka

ZeroMQ

- Mechanizm komunikacji ze zintegrowanym kolejkowaniem komunikatów (brak brokera)
- Interfejs wzorowany na gniazdach BSD
- Interfejs dostępny dla między innymi dla języka C, C++, C#, Java, Python, Ruby, Ada
- Zdefiniowane schematy komunikacji (REQ-REP, PUB-SUB, PUSH-PULL)
- Brak struktury komunikatu (komunikat jest sekwencją bajtów, których typ/struktura zdefiniowana jest na poziomie aplikacji)

Ogólny schemat komunikacji

Proces 1

1. utworzenie kontekstu
2. utworzenie gniazda
3. połączenie gniazda
4. wysłanie komunikatu
5. inne operacje komunikacyjne
6. zamknięcie gniazda
7. usunięcie kontekstu

Proces 2

1. utworzenie kontekstu
2. utworzenie gniazda
3. związanie gniazda
4. odbiór komunikatu
5. inne operacje komunikacyjne
6. zamknięcie gniazda
7. usunięcie kontekstu

Kontekst

- ☛ Kontekst jest zbiorem gniazd.
- ☛ Tworzenie kontekstu
`void *context = zmq_ctx_new ();`
- ☛ Zalecenie: 1 kontekst na proces.
- ☛ Usuwanie kontekstu
`zmq_ctx_destroy (context);`
lub
`zmq_ctx_term (context);`
 - zakończenie wszystkich operacji blokujących
 - oczekiwanie na zakończenie wysyłania komunikatów
 - oczekiwanie na zamknięcie wszystkich gniazd

Tworzenie gniazda

- ☛ Wywołanie:

```
void *zmq_socket (void *context, int type);
```

- ☛ Podstawowe typy gniazd:

- ↳ REQ i REP (naprzemienne wysyłanie i odbiór)
- ↳ PUB i SUB (publikowanie do wielu subskrybentów)
- ↳ PUSH i PULL (przetwarzanie potokowe)

Wiązanie i łączenie gniazd

- Wiązanie gniazda:

```
void * gniazdo = zmq_socket (... , ...);  
zmq_bind (gniazdo, "tcp://*:5556");
```

- Łączenie gniazd:

```
void * gniazdo = zmq_socket (... , ...);  
zmq_connect (gniazdo, "tcp://*:5556");
```

- Łączyć można tylko „kompatybilne” gniazda.

Komunikacja

- ✦ Wysyłanie wiadomości:

```
int zmq_send (void *socket, void *buf,  
size_t len, int flags);  
int zmq_sendmsg (void *socket,  
zmq_msg_t *msg, int flags);
```

- ✦ Odbiór wiadomości:

```
int zmq_recv (void *socket, void *buf,  
size_t len, int flags);  
int zmq_recvmsg (void *socket,  
zmq_msg_t *msg, int flags);
```

Wywoływanie procedur zdalnych — RPC

Zagadnienia projektowe

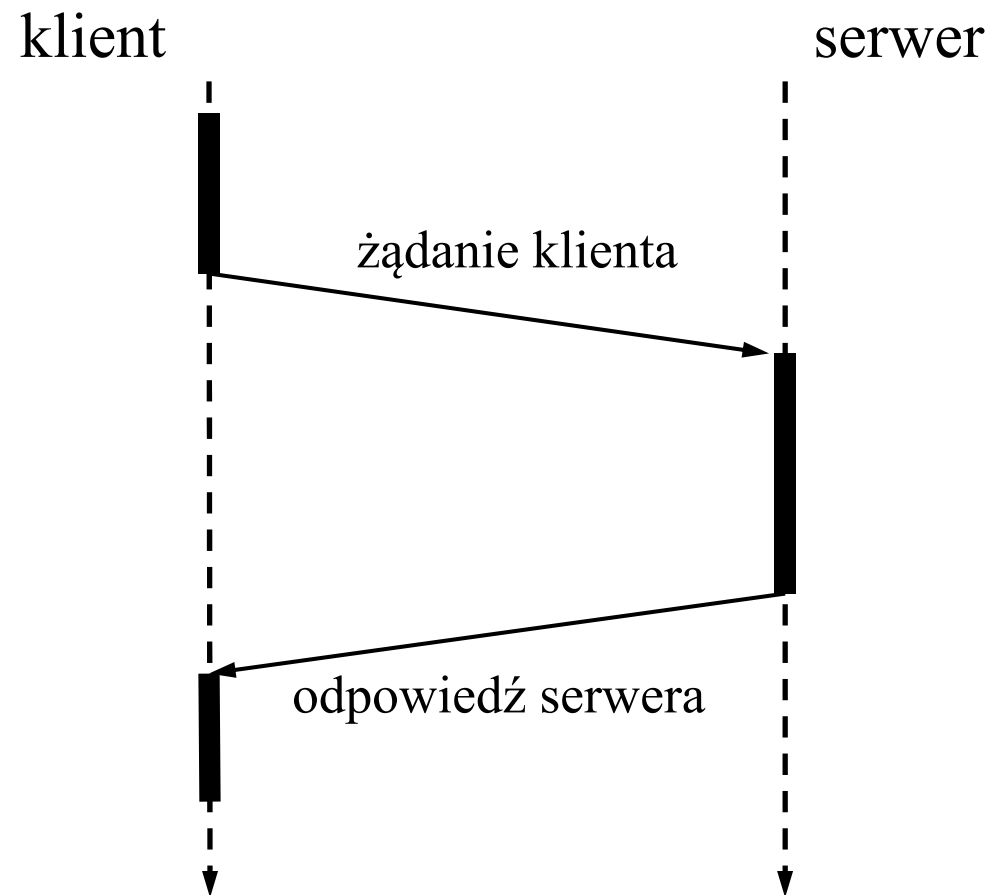
Zagadnienia realizacyjne

Mechanizm wywołania procedury

```
main(int argc, char* argv[]) {  
    int id, status;  
    id = atoi(argv[1]);  
    status = zabij_proces(id);  
    exit(status)  
}  
:  
:  
:  
int zabij_proces (int pid) {  
    int stat;  
    stat = kill(pid, 9);  
    return stat;  
}
```

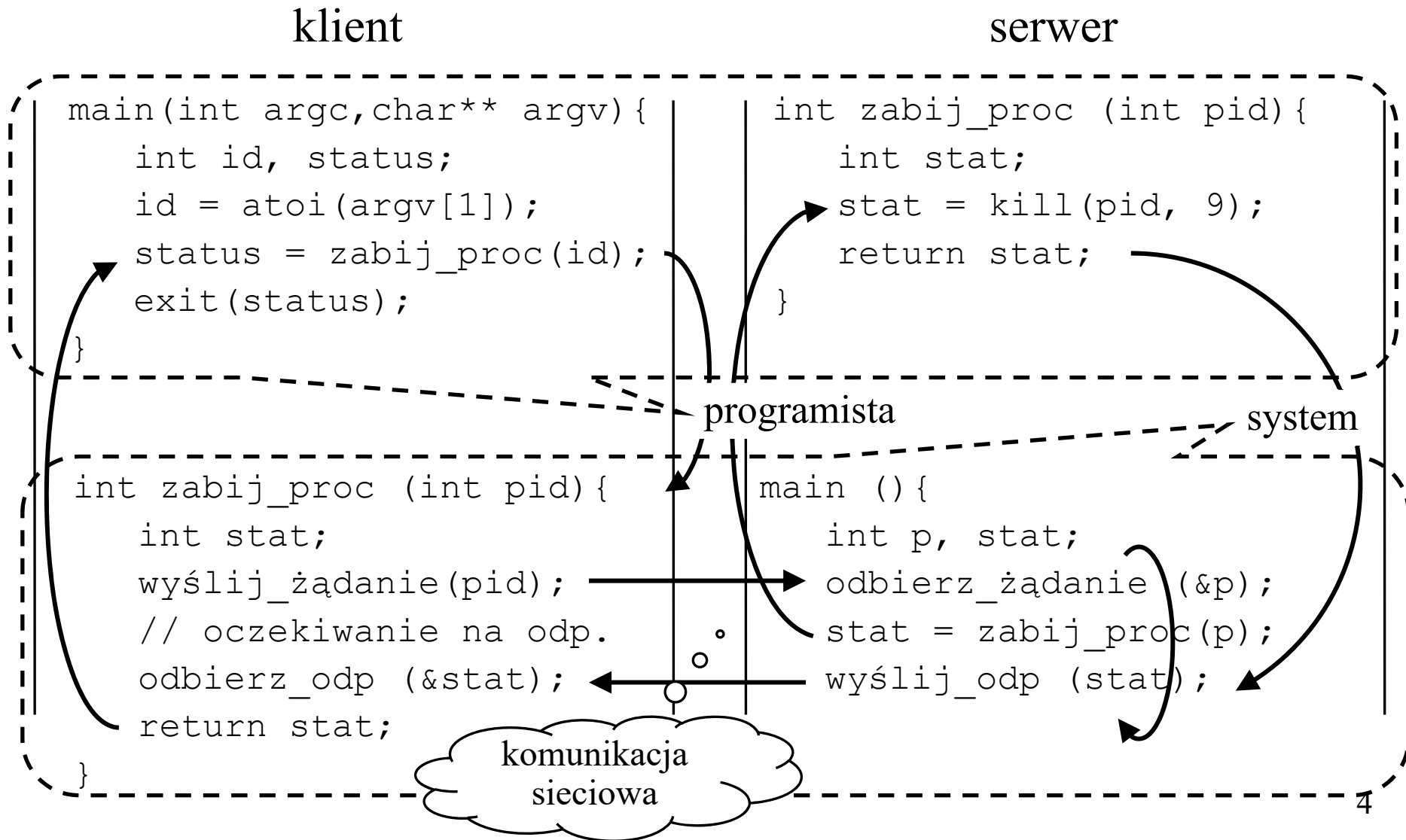
The diagram illustrates a function call mechanism. A curved arrow originates from the `zabij_proces(id)` call in the `main` function and points to the start of the `zabij_proces` function definition. Another curved arrow originates from the `return stat;` line in the `zabij_proces` function and points back to the line following the function call in the `main` function, specifically the closing brace of the `main` function.

Schemat interakcji klient-serwer (transakcja komunikatu)



RPC

Semantyka wywołania procedury w komunikacji sieciowej



Zagadnienia projektowe

- ☛ przeźroczystość dostępu — ukrycie komunikacji sieciowej przed aplikacją przez odpowiednie opakowanie funkcji komunikacyjnych namiastką klienta oraz serwera.
- ☛ gwarancja wykonania — ukrywanie błędów komunikacyjnych
- ☛ specyfikacja interfejsu — sposób opisu sygnatur procedur zdalnych (nazwy, typy parametrów)
- ☛ obsługa sytuacji wyjątkowych

Przezroczystość dostępu

- ☛ namiastka klienta (ang. client stub) — udostępnienie aplikacji klienckiej procedury lokalnej odpowiedzialnej za przesłanie danych do serwera oraz odebranie wyników
- ☛ namiastka serwera (ang. server stub) — udostępnienie aplikacji po stronie serwera procedury lokalnej odpowiedzialnej za odebranie identyfikatora procedury zdalnej do wywołania, parametrów procedury, a odesłanie wyników lub zgłoszenie wyjątków

Warstwy protokołu RPC

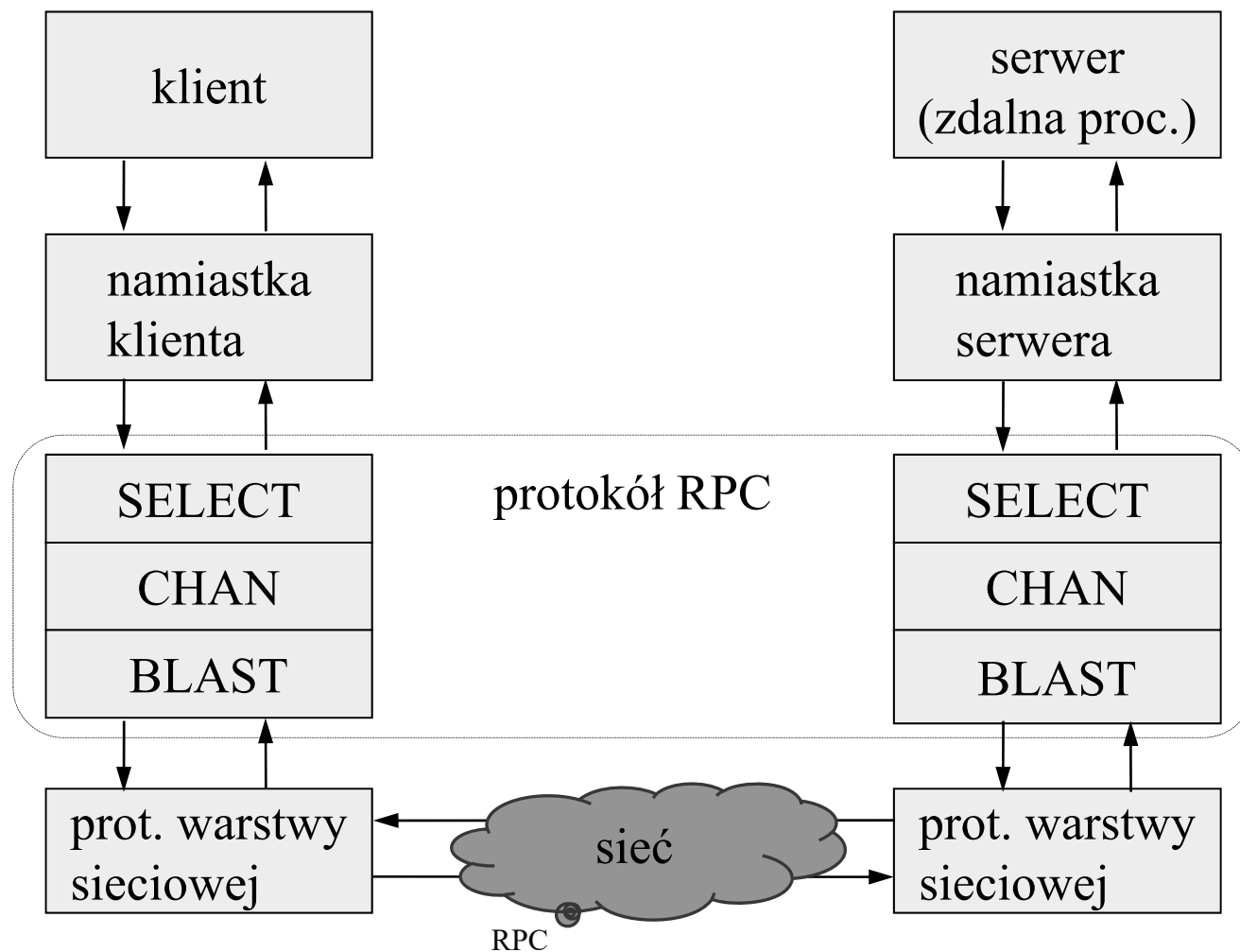
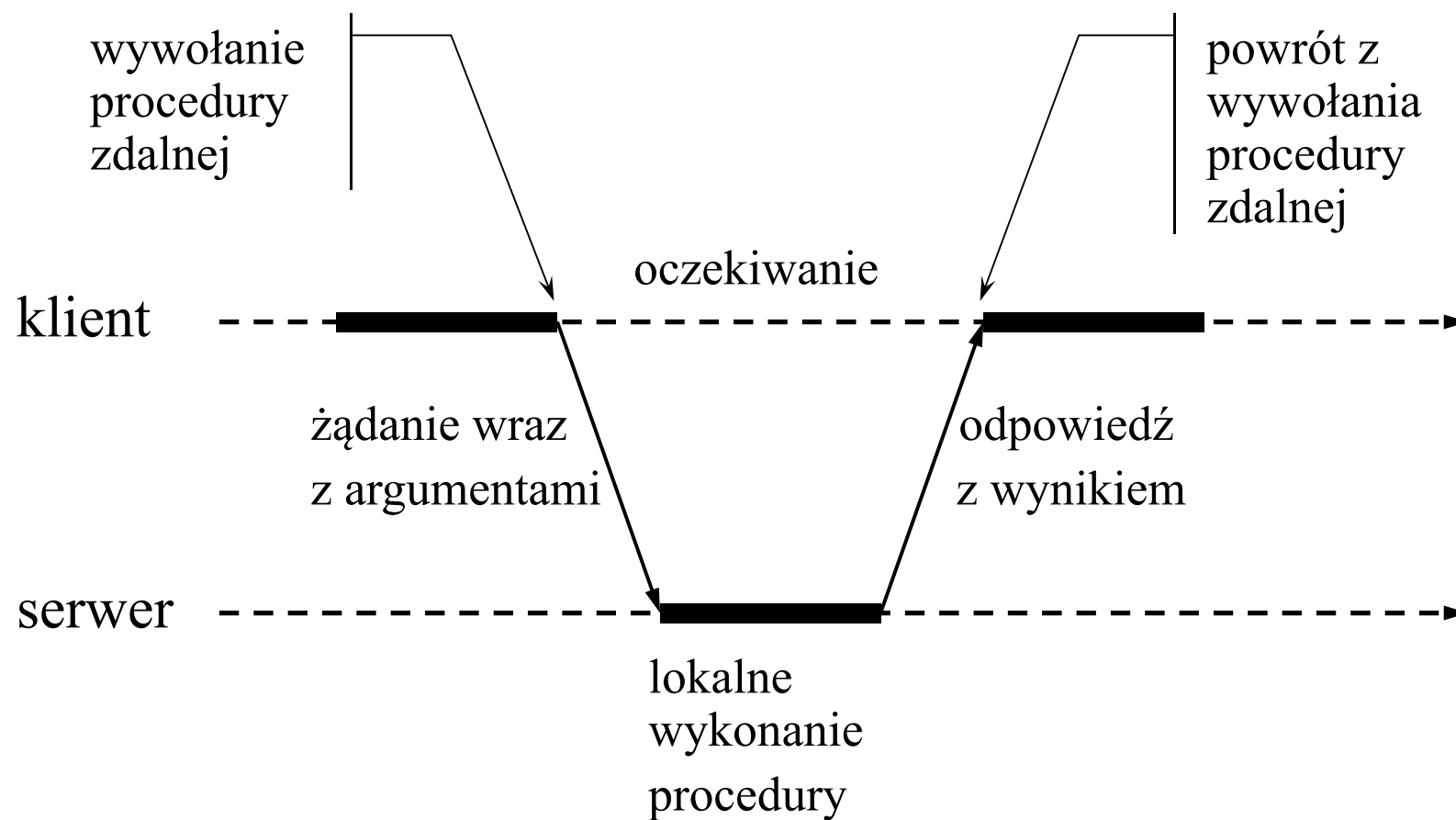


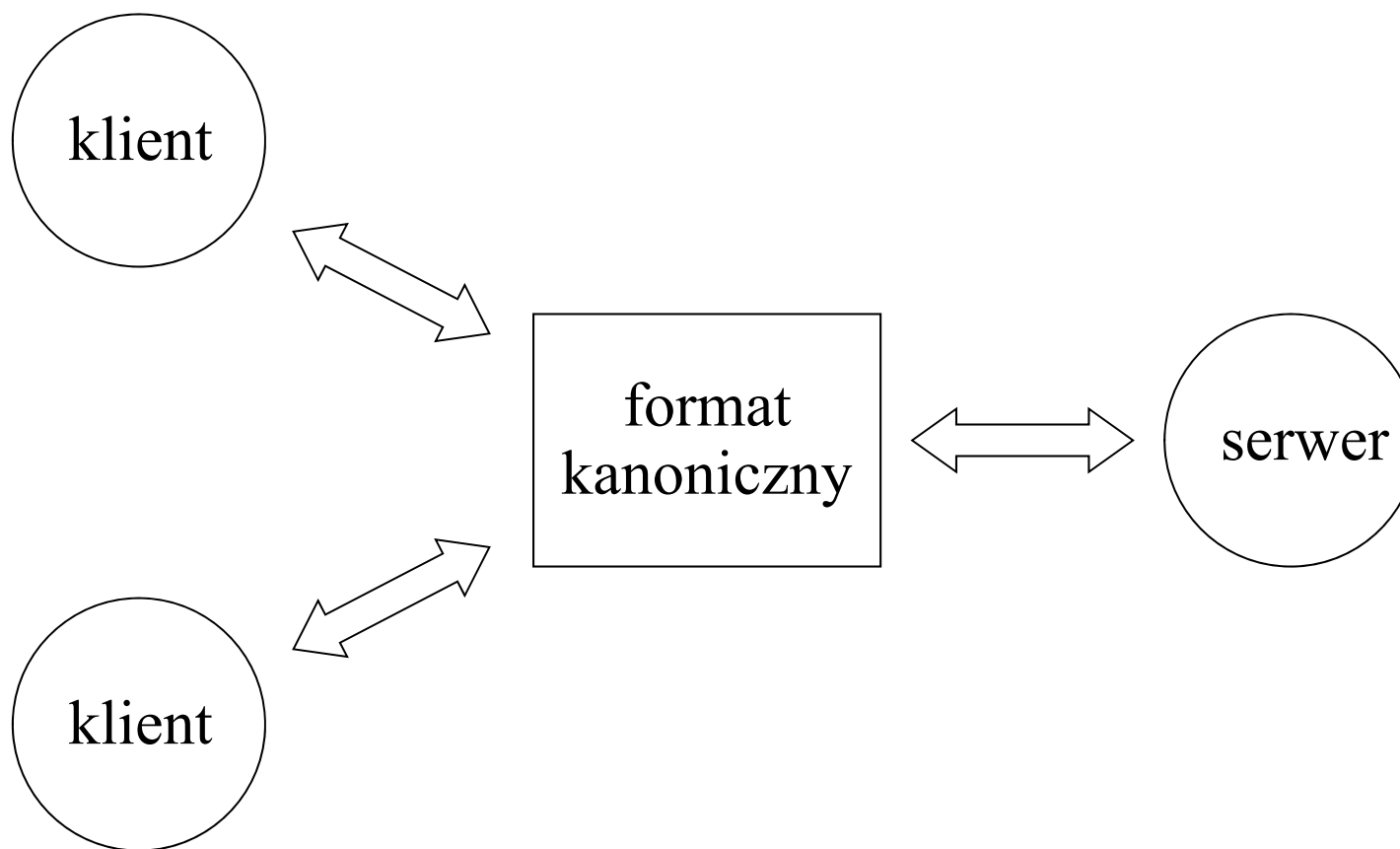
Diagram przestrzenno-czasowy interakcji klient-serwer



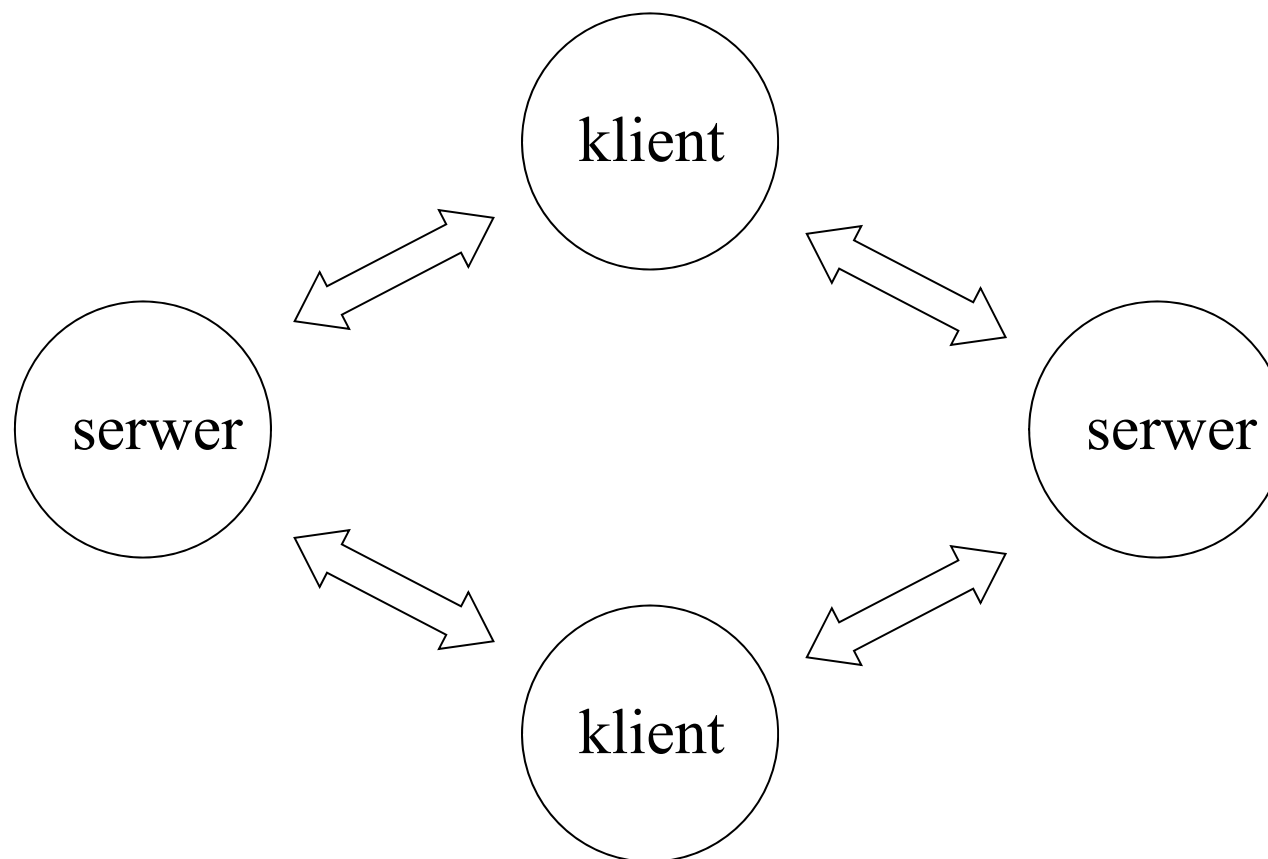
Przekazywanie parametrów

- ☛ przekazywanie przez wartość (ang. call-by-value)
 - ↳ problem różnic w reprezentacji danych, np. w kodowaniu znaków, uporządkowaniu bajtów, w formatach liczb zmiennopozycyjnych itp.
- ☛ przekazywanie przez referencje (ang. call-by-reference)
 - ↳ problem zinterpretowania wartości wskaźnika w innej przestrzeni adresowej
- ☛ przekazywanie przez kopiowanie i odtwarzanie —
 - ↳ problem przetaczania (kwestia opisu struktur danych w celu prawidłowego zidentyfikowania wszystkich składowych)

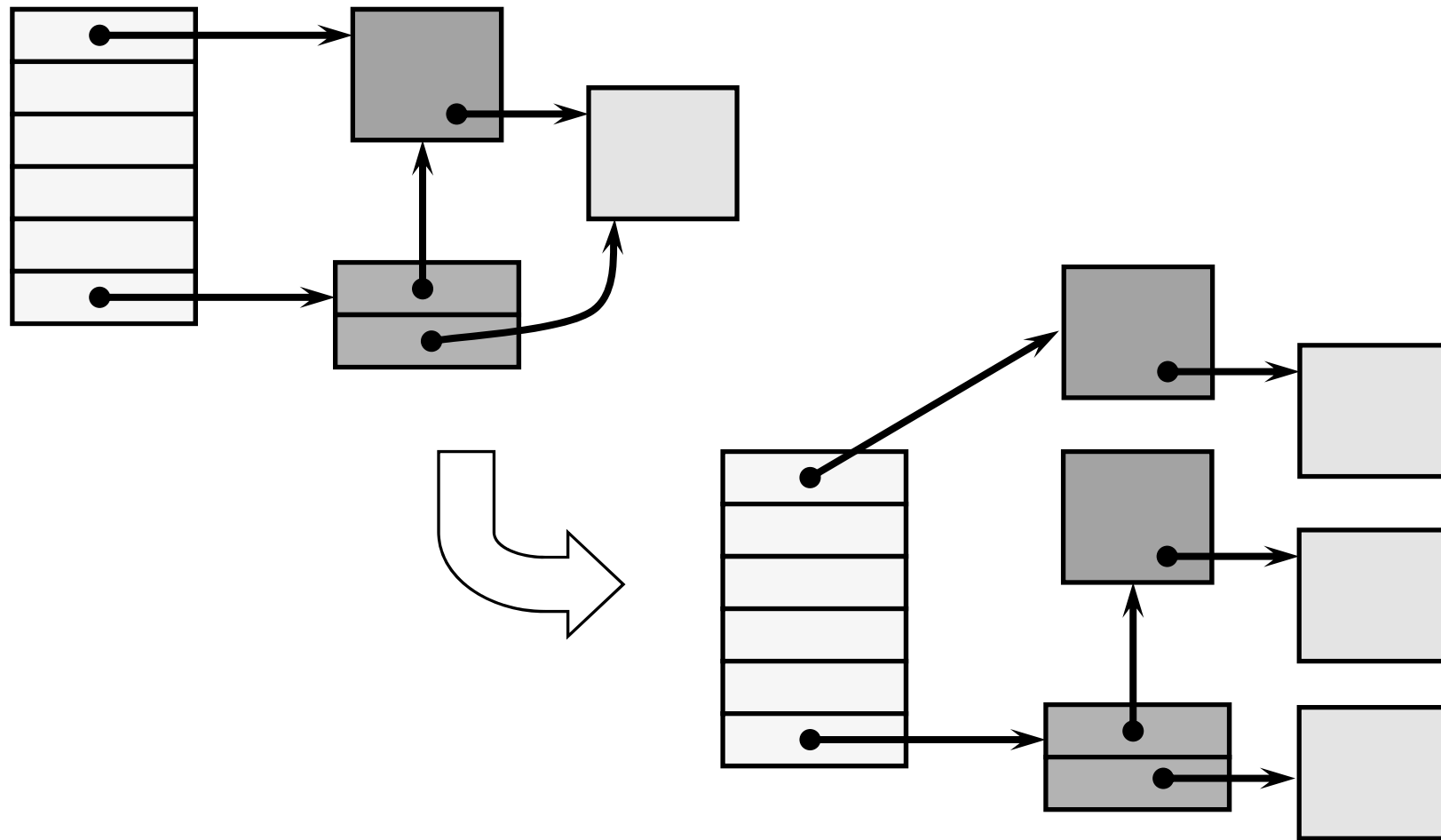
Konwersja danych — reprezentacja kanoniczna



Konwersja bezpośrednia



Problem przetaczania — przykład złożonej struktury danych



RPC

Gwarancja wykonania (semantyka błędu)

- ☛ Semantyka *co najmniej raz* — po uzyskaniu odpowiedzi (wyniku) od serwera klient ma pewność, że wywoływana procedura wykonała się co najmniej raz.
- ☛ Semantyka *co najwyżej raz* — po uzyskaniu odpowiedzi (wyniku) od serwera klient wie, że wywoływana procedura wykonała się dokładnie raz.

Jak należy zinterpretować przypadek wystąpienia błędu (wyjątku) w wywołaniu procedury zdalnej?

- ☛ Semantyka *dokładnie raz* — niemożliwa do uzyskania, jeśli system narażony jest na awarie (np. serwera lub łączy).
- ☛ Semantyka *ewentualnie* — brak gwarancji, procedura mogła się wykonać lub mogła się nie wykonać.

Zagadnienia realizacyjne

- Przetwarzanie interfejsu
- Wiązanie klienta z serwerem
- Obsługa komunikacji klient-serwer
- Realizacja semantyki błędu
- Problem osieroconych obliczeń

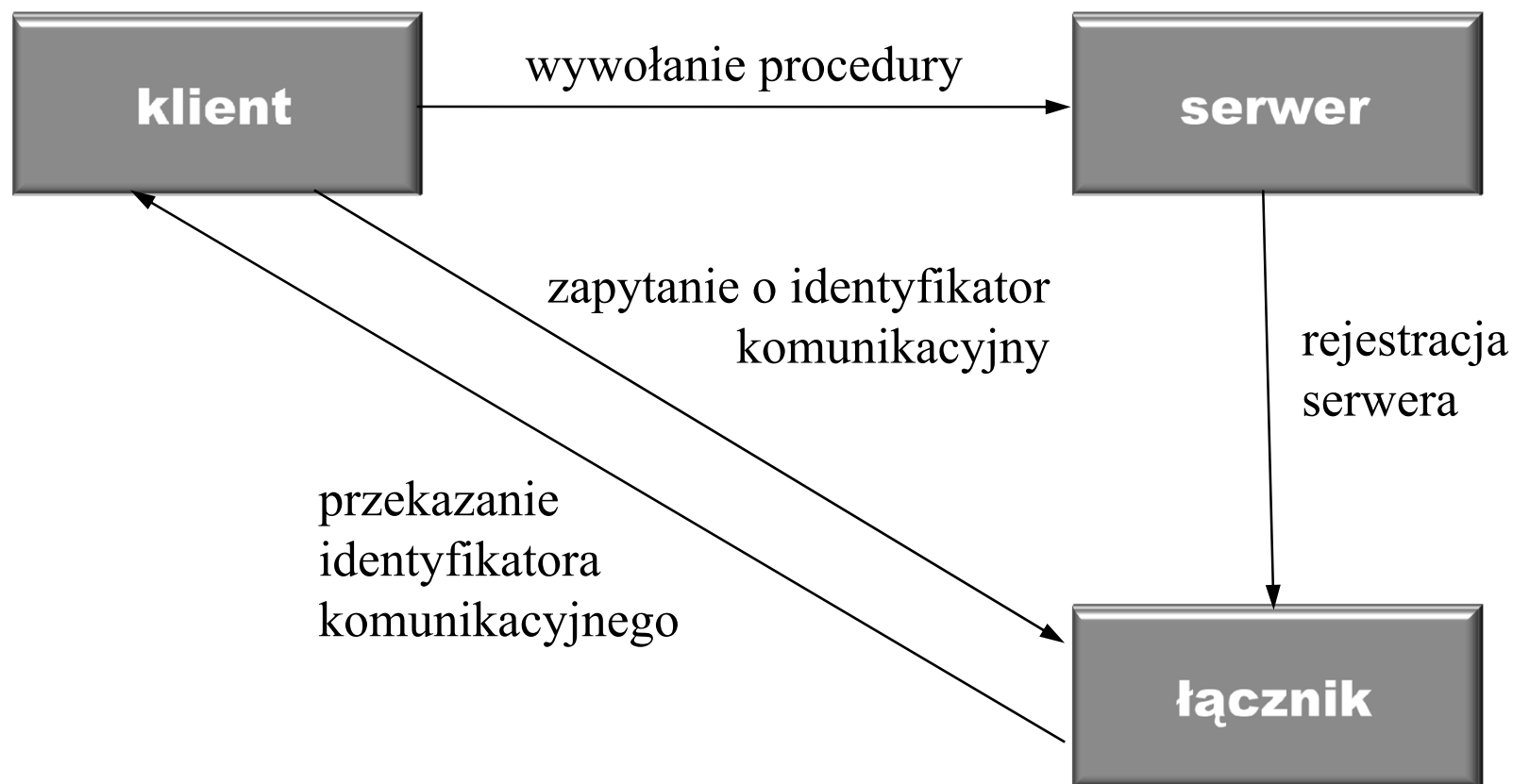
Przetwarzanie interfejsu procedur zdalnych

- Generowanie namiastki klienta
- Generowanie namiastki serwera
- Generowanie przykładowego programu klienta (client sample)
- Generowanie wzorca do implementacji procedur zdalnych (template)
- Generowanie plików do zarządzania kompilacją

Wiązanie klienta z serwerem

- ☛ Wiązanie statyczne — klient ma wprowadzony na stałe identyfikator komunikacyjny serwera (np. para: adres IP, nr portu).
- ☛ Wiązanie dynamiczne — klient uzyskuje adres serwera za pośrednictwem łącznika (np. portmap lub rpcbind w Sun RPC).

Wiązanie dynamiczne klienta z serwerem



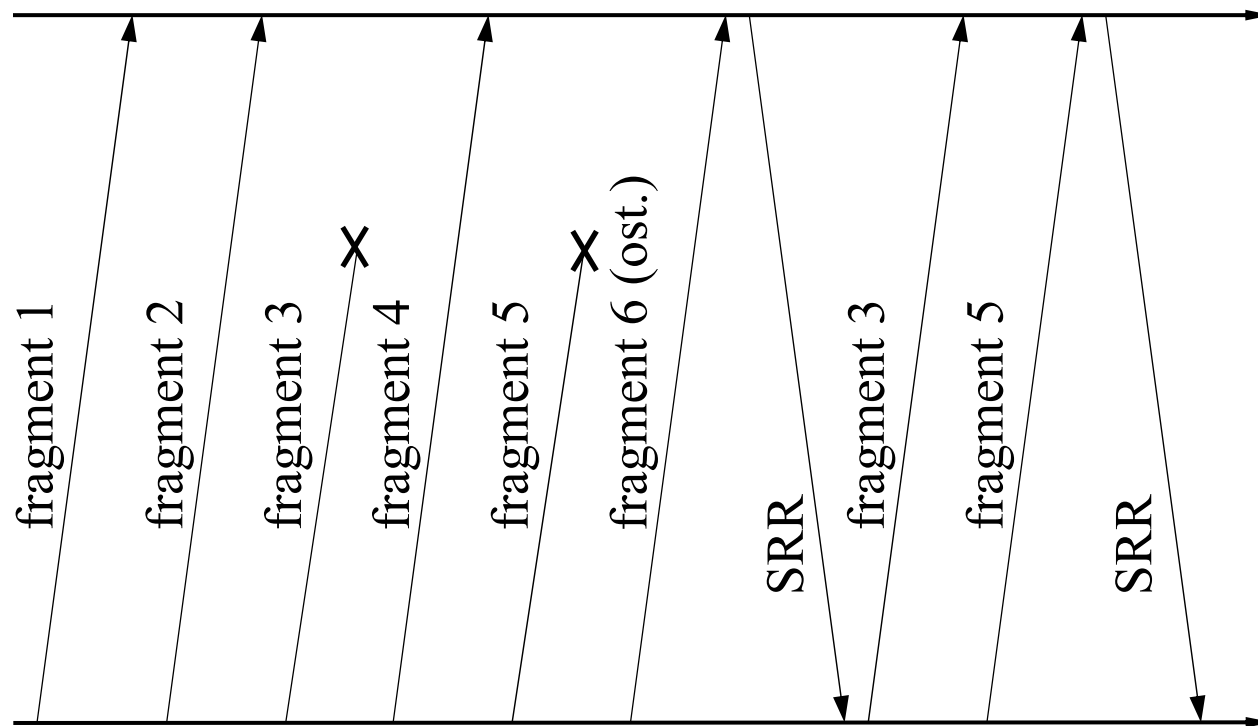
Obsługa komunikacji klient-serwer

- BLAST — realizuje przesyłanie dużych komunikatów poprzez podział na mniejsze części, transmisję poszczególnych części i ponowne złożenie w jeden komunikat po stronie odbiorczej,
- CHAN — synchronizuje wymianę komunikatów z zadaniami wywołania procedur oraz odpowiedziami,
- SELECT — rozdziela i przekazuje komunikaty z zadaniami do odpowiednich procesów.

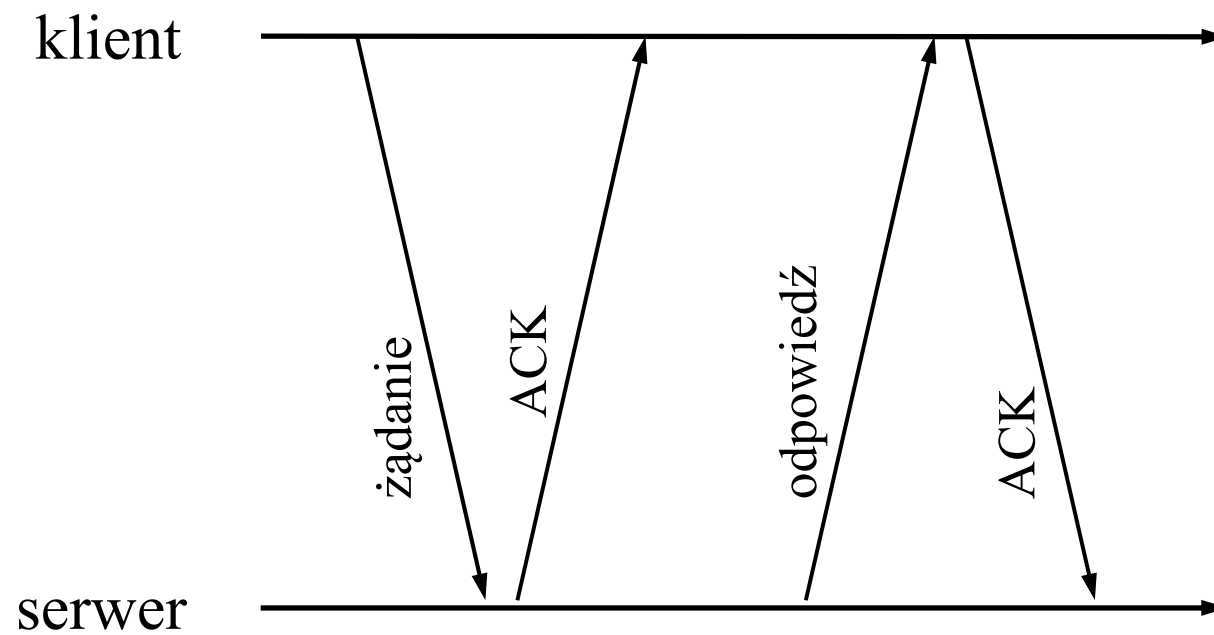
BLAST

odbiorca

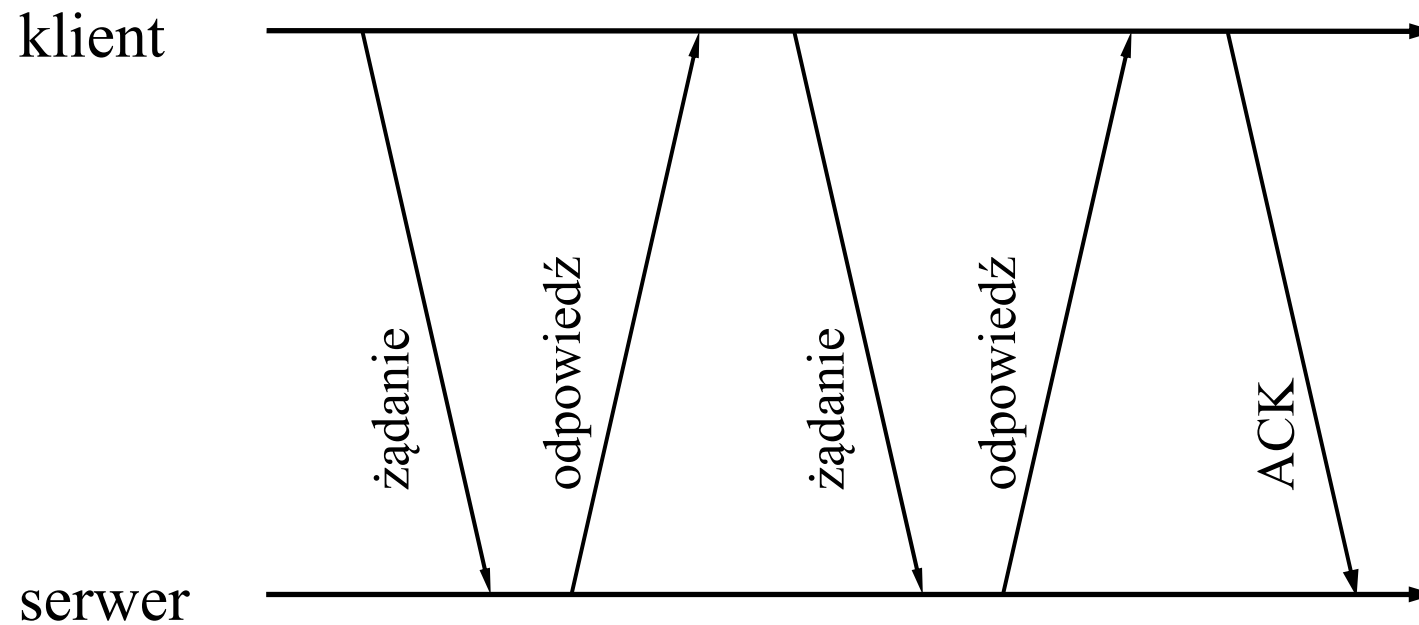
nadawca



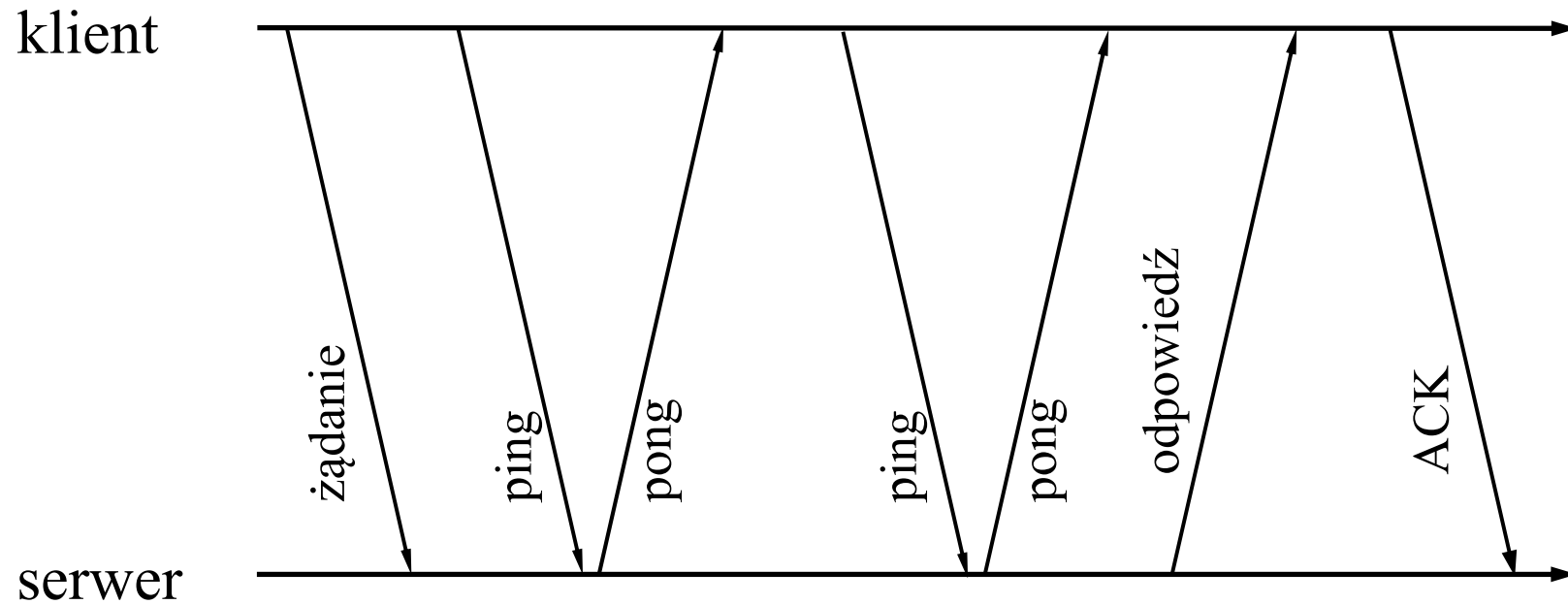
CHAN



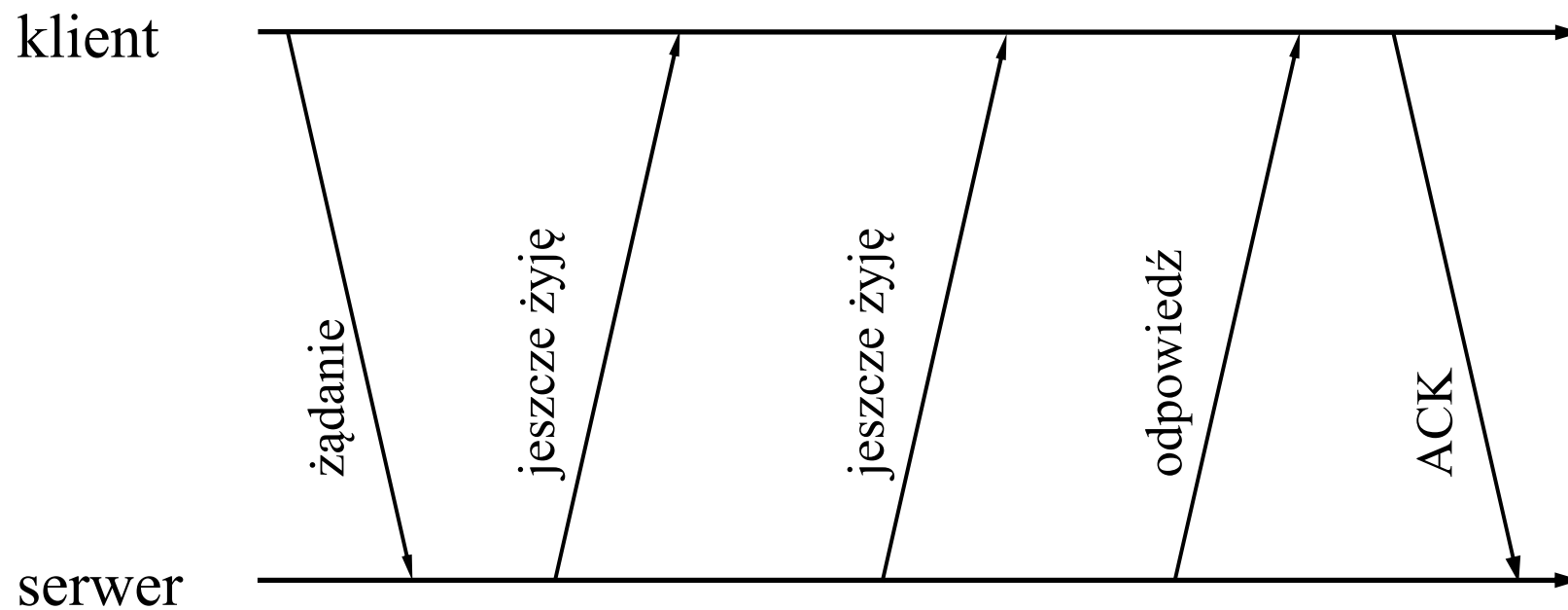
CHAN — domniemane potwierdzenia



CHAN — próbkowanie serwera



CHAN — „bicie serca”



Realizacja semantyki błędu

- nie można zlokalizować serwera — zgłoszenie wyjątku
- zaginione żądanie — retransmisja żądania po upływie ustalonego czasu oczekiwania
- zaginiona odpowiedź — retransmisja żądania
 - ↳ stosowanie procedur idempotentnych
 - ↳ numerowanie żądań i retransmisja odpowiedzi
- awaria serwera
 - ↳ przed podjęciem realizacji → retransmisja żądania
 - ↳ po wykonaniu → zgłoszenie wyjątku
- awaria klienta — osierocenie obliczeń

Usuwanie osieroconych obliczeń (1)

- Eksterminacja — rejestrowanie działań podejmowanych przez klienta na nośniku niewrażliwym na awarie i usuwanie na tej podstawie osieroconych obliczeń po restarcie klienta.
- Reinkarnacja — każdy restart klienta rozpoczyna nową epokę (identyfikowaną przez numer kolejny), po której usuwane są wszystkie obliczenia związane z poprzednią epoką.

Usuwanie osieroconych obliczeń (2)

- Łagodna reinkarnacja — reinkarnacja, w której usuwa się tylko te obliczenia rozpoczęte w starej epoce, dla których nie ma właściciela.
- Wygaśnięcie — przydział określonego czasu T serwerowi na wykonanie procedury. Jeśli wykonanie nie zakończy się w czasie T , serwer musi uzyskać kolejny przydział, pod warunkiem, że obliczenia nie zostały osierocone. Jeśli klient odczeka czas T przy restarcie, osierocone obliczenia same się zakończą.

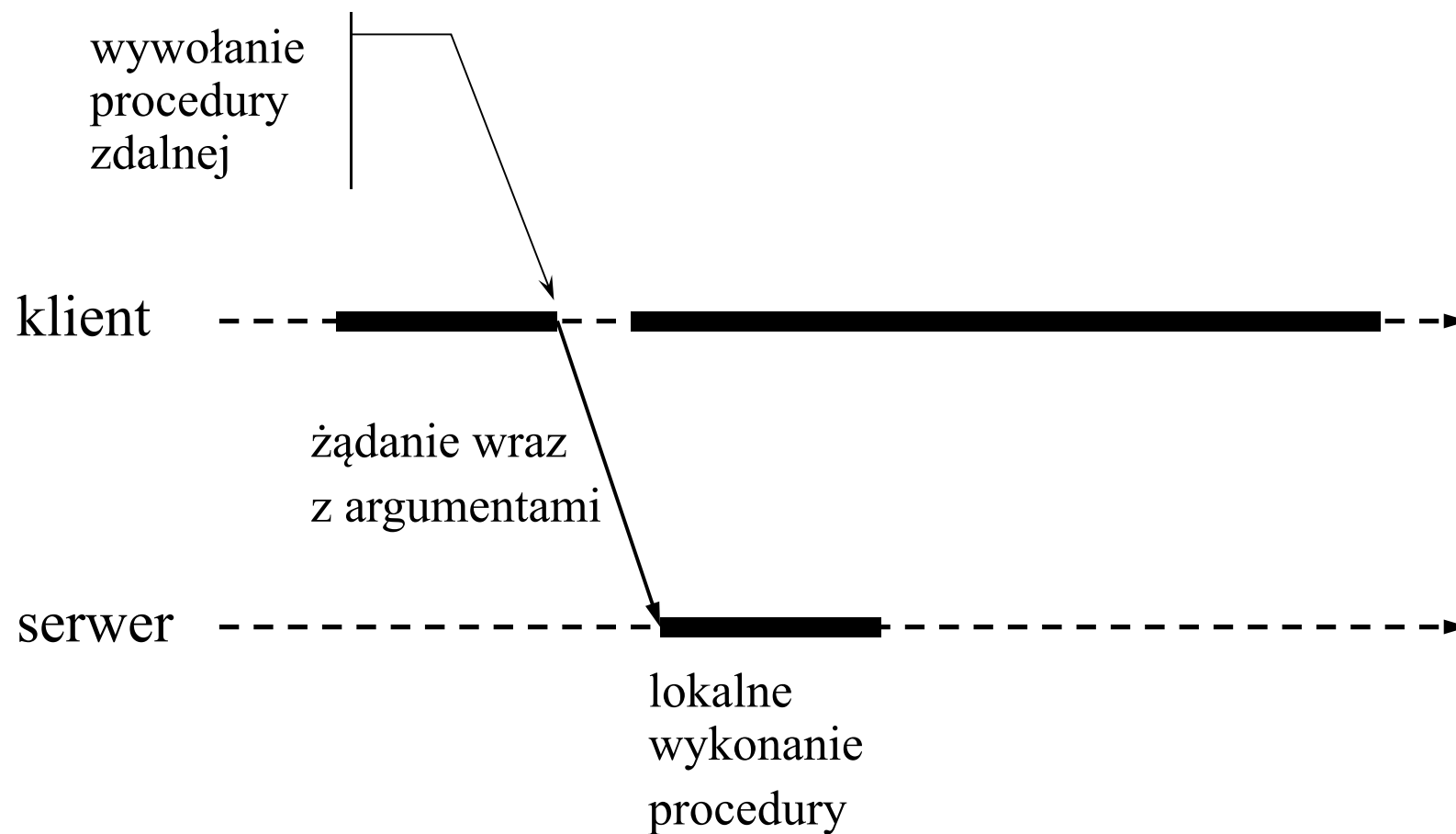
SELECT

- ☛ po stronie klienta: odwzorowanie wywoływanej procedury na jej identyfikator, przekazywany do serwera.
- ☛ po stronie serwera: zlokalizowanie wywoływanej procedury na podstawie identyfikatora.

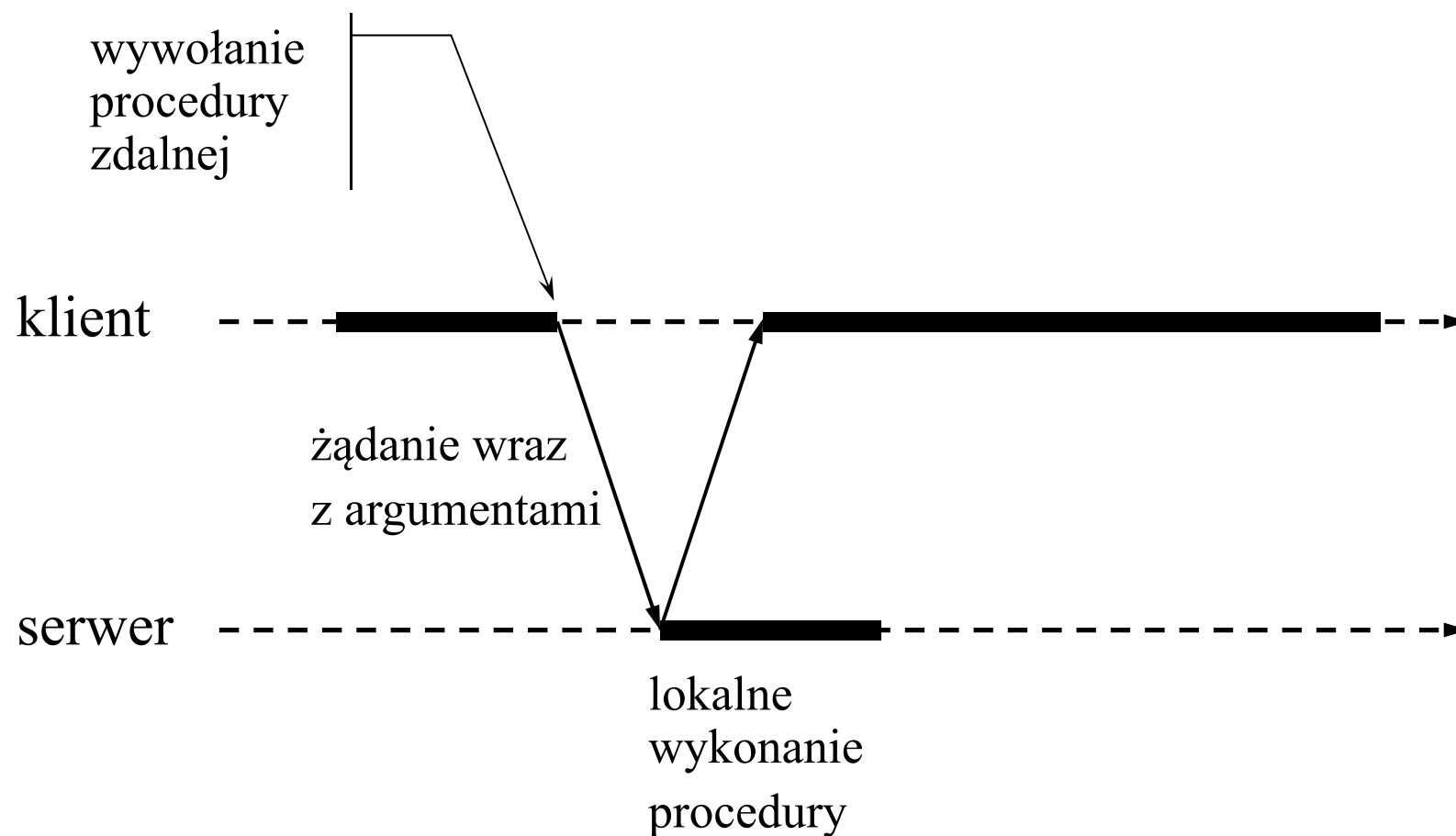
Warianty użycia mechanizmu RPC

- Wywołanie asynchroniczne — klient nie czeka na wynik wykonania procedury zdalnej (wykonywanie procedury zdalnej odbywa się równolegle z przetwarzaniem po stronie klienta).
- Wywołanie zwrotne — klient udostępnia procedurę zdalną po swojej stronie i przekazuje serwerowi informacje umożliwiające jej wywołanie (zamiana ról pomiędzy klientem a serwerem).

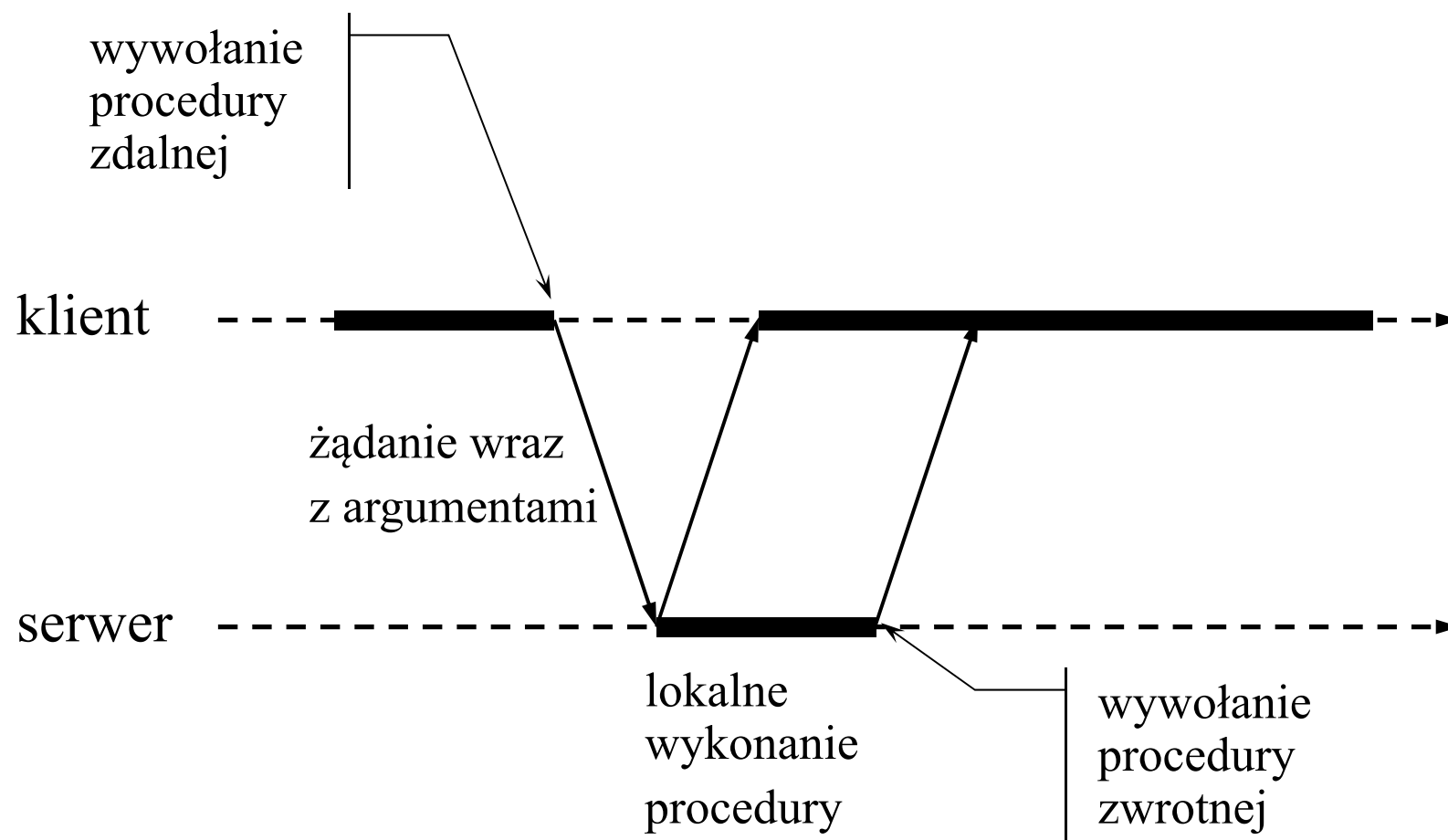
Wywołanie asynchroniczne



Wywołanie asynchroniczne z potwierdzeniem odbioru



Wywołanie zwrotne



Sun RPC/XDR



Remote Procedure Call
eXternal Data Representation

Sun RPC

- ☛ trójwymiarowa identyfikacja procedur (nr programu, nr wersji, nr procedury)
- ☛ protokół RPC oparty na protokołach warstwy transportowej stosu TCP/IP (TCP lub UDP)
- ☛ wiązanie dynamiczne (portmap lub rpcbind)
- ☛ kanoniczny format reprezentacji danych (XDR)
- ☛ opis interfejsu w języku C-podobnym i przetwarzanie przez narzędzie **rpcgen**

Tworzenie aplikacji (1)

- przygotowanie pliku z opisem interfejsu w języku RPC (języku narzędzia **rpcgen**), zawierającego:
 - ↳ definicją struktur danych dla przekazywanych parametrów ,
 - ↳ specyfikację procedur zdalnych, obejmującą:
 - ✖ numer programu, numer wersji i numer procedury,
 - ✖ typy argumentów
 - ✖ typ zwracanej wartości.

Tworzenie aplikacji (2)

- ☛ wygenerowanie przez **rpcgen** plików do kompilacji w języku C:
 - ↳ pliku nagłówkowego z definicjami stałych,
 - ↳ plików z namiastką klienta i namiastką serwera,
 - ↳ plików zawierających szkielet procedur zdalnych (fragment programu serwera) i szkielet programu klienta dla potrzeb przetestowania procedur zdalnych,
 - ↳ plik do zarządzania kompilacją (`Makefile`),
 - ↳ plik z funkcjami XDR do konwersji zdefiniowanych typów danych.

Tworzenie aplikacji (3)

- uzupełnienie lub dokonanie niezbędnych modyfikacji w zawartości wygenerowanych plików
 - ↳ uzupełnienie plików zawierających szkielety procedur zdalnych,
 - ↳ uzupełnienie/modyfikacja programu klienta,
 - ↳ uzupełnienie/modyfikacja pliku Makefile.

Tworzenie aplikacji (4)

- ☛ kompilacja programów,
- ☛ testowanie działania procedur zdalnych,
 - ➔ ewentualne przygotowanie aplikacji w wersji zwartej (nie rozproszonej) w celu przetestowania działania samych procedur,
- ☛ przygotowanie programu klienta korzystającego z procedur zdalnych w sposób wynikający z potrzeb aplikacji.

Język RPC — specyfikacja procedur zdalnych

```
program NAZWA_PROGRAMU {  
    version NAZWA_WERSJI1 {  
        typ nazwa_proc1(typ_arg)=nr_proc;  
        typ nazwa_proc2(typ_arg)=nr_proc;  
        ⋮  
    }=numer_wersji;  
    version NAZWA_WERSJI2 {  
        typ nazwa_proc1(typ_arg)=nr_proc;  
        typ nazwa_proc2(typ_arg)=nr_proc;  
        ⋮  
    }=numer_wersji;  
}=numer_programu;
```

Dostępne zakresy numerów programów

- ☛ 0 – 0x1FFFFFFFFF — numery standardowe, zdefiniowane (przydzielane) przez firmę Sun,
- ☛ 0x20000000 – 0x3FFFFFFFFF — przydzielone przez użytkownika,
- ☛ 0x40000000 – 0x5FFFFFFFFF — zarezerwowane dla aplikacji, które generują numery programów dynamicznie,
- ☛ 0x60000000 – 0xFFFFFFFF — zarezerwowane.

Przykład opisu prostego interfejsu

```
program REMOTE_KILL {  
    version DEFAULT_SIGNALUM {  
        int rkill(int) = 1;  
    } = 1;  
} = 0x200000001;
```

Przykład opisu interfejsu z definicją struktury danych

```
struct rkill_params{  
    int pid;  
    int signum;  
};
```

```
program REMOTE_KILL {  
    version SPECIFIED_SIGNAL {  
        int rkill(rkill_params) = 1;  
    } = 2;  
} = 0x20000001;
```


Przykład opisu interfejsu dla **rpcgen -N**

```
program REMOTE_KILL {  
    version SPECIFIED_SIGNAL {  
        int rkill(int, int) = 1;  
    } = 2;  
} = 0x200000001;
```

Standard XDR

- ☛ Opis standardu — RFC 1014
- ☛ Kanoniczna reprezentacja danych oparta na formacie IEEE
- ☛ Deklaratywny język opisu struktur danych (zblizony do języka C)
- ☛ Koncepcja konwersji oparta na
 - ↳ potokach — miejscach przechowywania danych w formacie XDR
 - ↳ filtrach — procedurach konwersji pomiędzy własnym formatem maszyny a formatem kanonicznym (w obu kierunkach)

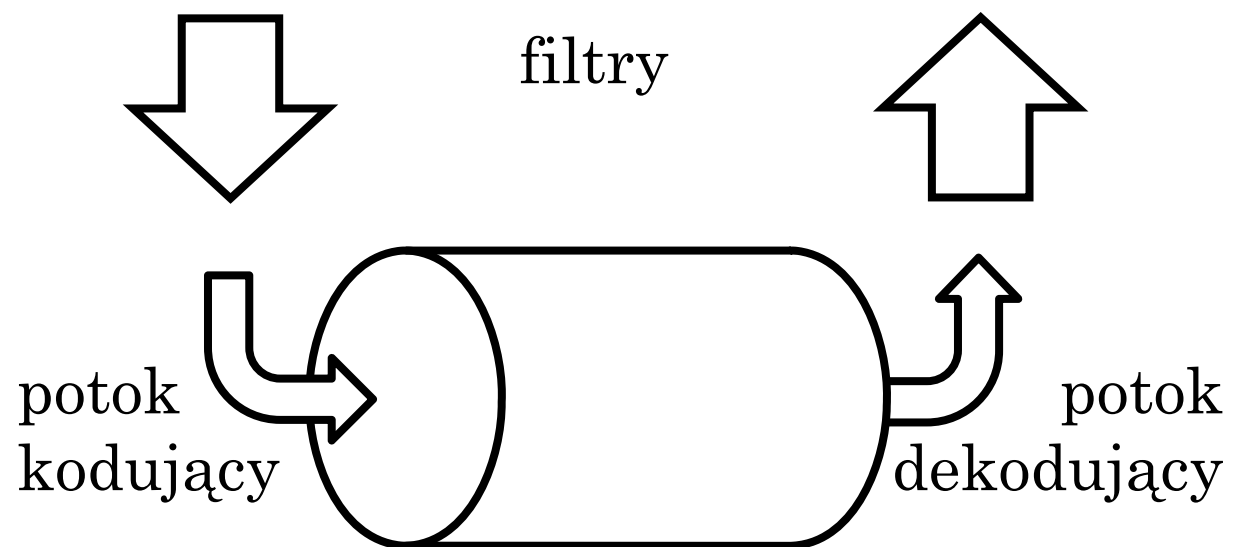
Potok XDR

- ☛ Potok jest miejscem składowania danych XDR, a dokładniej środkiem dostępu do nich w celu:
 - ↳ zapisu — potok kodujący
 - ↳ odczytu — potok dekodujący
- ☛ Rodzaje potoków:
 - ↳ potok na standardowym wejściu-wyjściu — na otwartym pliku
 - ↳ potok w pamięci — w obszarze pamięci opisanym przez adres i rozmiar w bajtach
 - ↳ potok komunikatów (rekordów) — na strumieniu danych, zdefiniowanym wraz z procedurami obsługi tego strumienia (zapis, odczyt)

Filtr XDR

- ☛ Filtr jest procedurą konwersji, służącą do realizacji dostępu do danych w potoku w celu zapisu (kodująca) lub odczytu (dekodująca) — zależnie od kierunku działania potoku.
- ☛ Rodzaje filtrów
 - ↳ filtr prosty — służy do konwersji typów prostych
 - ↳ filtr złożony — służy do konwersji typów złożonych (np. tablicowych, wskaźnikowych)
 - ↳ filtr pochodny — połączenie innych filtrów w ramach jednej procedury filtrującej

Potoki i filtry XDR



Filtry proste

- Filtr dla:
 - ↳ typów prostych: `bool_t`, `char`, `short`, `int`, `long`,
(również bez znaku) `float`, `double`
 - ↳ typu wyliczeniowego (`enum`)
 - ↳ typu `void`
- Ogólna postać procedury filtrującej (na przykładzie
typu `unsigned int`):

```
bool_t  
xdr_u_int(XDR* xdrs,  
          unsigned int *ptr);
```

Filtry złożone (1)

- ☛ Filtr do konwersji
 - ☞ tablic bajtów o ustalonej lub zmiennej wielkości — `xdr_opaque`, `xdr_bytes`
 - ☞ tablic elementów określonego typu o ustalonej lub zmiennej wielkości — `xdr_vector`, `xdr_array`
 - ☞ łańcuchów znaków — `xdr_string`, `xdr_wrapstring`
 - ☞ typów wskaźnikowych — `xdr_reference`, `xdr_pointer`
 - ☞ unii — `xdr_union` (w praktyce dla statycznie zdefiniowanych typów unijnych stosowany jest filtr pochodny)

Filtry złożone (2)

- ☛ Ogólna postać filtru złożonego:

```
xdr_typ (XDR* xdrs, ptr, ...,  
        [xdrproc_t elproc]);
```

- ☛ W przypadku typu statycznego (ustalona zajętość pamięci) wskaźnik *ptr*, ma postać `char *ptr`, w przypadku typu o zmiennej wielkości (konieczność dynamicznej alokacji pamięci) `char **ptr`.
- ☛ Jeśli typ podstawowy (w przypadku tablic, wskaźników) nie jest z góry określony, konieczne jest wskazanie filtru XDR do konwersji elementu typu podstawowego.

Filtry złożone do konwersji tablic

```
xdr_array(xdrs, arrp, sizep, maxsize, elsize, elproc)
    XDR *xdrs;
    char **arrp;
    u_int *sizep, maxsize, elsize;
    xdrproc_t elproc;

xdr_vector(xdrs, arrp, size, elsize, elproc)
    char *arrp;
    u_int size, elsize; // pozostałe parametry – j.w.

xdr_bytes(xdrs, sp, sizep, maxsize)
    char **sp; // pozostałe parametry – j.w.

xdr_opaque(xdrs, cp, cnt)
    char *cp;
    u_int cnt;
```

Filtry złożone do konwersji łańcuchów znaków

```
xdr_string(xdrs, sp, maxsize)
```

```
    XDR *xdrs;
```

```
    char **sp;
```

```
    u_int maxsize;
```

```
xdr_wrapstring(xdrs, sp)
```

```
    ≡ xdr_string(xdrs, sp, MAXUN.UNSIGNED );
```

`xdr_wrapstring` nie wymaga podania rozmiaru (liczby znaków łańcucha)

Filtry złożone do konwersji struktur wskaźnikowych

```
xdr_reference(xdrs, pp, size, proc)
```

```
    XDR *xdrs;  
    char **pp;  
    u_int size;  
    xdrproc_t proc;
```

```
xdr_pointer(xdrs, objpp, objsize, xdrobj)
```

```
    XDR *xdrs;  
    char **objpp;  
    u_int objsize;  
    xdrproc_t xdrobj;
```

`xdr_pointer` w przeciwieństwie do `xdr_reference` interpretuje wskaźnik pusty, co umożliwia obsługę rekurencyjnych struktur danych

Przykład filtru pochodnego

Definicja struktury

```
struct struktura {  
    int x;  
    long y;  
    char c;  
    short s;  
};
```

Filtr XDR

```
bool_t  
xdr_struktura (XDR *xdrs,  
               struktura *objp) {  
    if (!xdr_int (xdrs,  
                  &objp->x)) return FALSE;  
    if (!xdr_long (xdrs,  
                   &objp->y)) return FALSE;  
    if (!xdr_char (xdrs,  
                   &objp->c)) return FALSE;  
    if (!xdr_short (xdrs,  
                    &objp->s)) return FALSE;  
    return TRUE;  
}
```

Zarządzanie pamięcią

- ☛ Przekazanie adresu pustego wskaźnika (typu `char**`) przy konwersji dekodującej spowoduje dynamiczną alokację pamięci przez filtr XDR.
- ☛ Zwolnienie obszaru dynamicznie zaalokowanej pamięci przez filtr XDR musi nastąpić w programie aplikacyjnym.
- ☛ Ogólna postać funkcji zwalniania pamięci:

```
xdr_free(xdrproc_t proc,  
         char* objp);
```

Tworzenie potoków XDR

- ☛ Potok na standardowym wejściu-wyjściu:

```
void xdrstdio_create(XDR *xdrs,  
                    FILE *file, enum xdr_op op);
```

- ☛ Potok w pamięci:

```
void xdrmem_create(XDR *xdrs, char *addr,  
                  u_int size, enum xdr_op op);
```

- ☛ Potok komunikatów:

```
void xdrrec_create(XDR *xdrs,  
                  u_int sendsize, u_int recvsize,  
                  char *handle,  
                  int (*readit)(char*,char*,int),  
                  int (*writeit)(char*,char*,int));
```

Potoki komunikatów (1)

- ☛ Ustalenie kierunku potoku musi nastąpić po jego utworzeniu (np. `xdrs -> x_op = XDR_ENCODE`)
- ☛ Rozmiary buforów (parametry *sendsize* i *recvsize*) mogą mieć wartość 0, co oznacza przyjęcie wartości domyślnych.
- ☛ Gdy konieczne jest opróżnienie bufora wyjściowego (wysyłanie danych) lub zapełnienie bufora wejściowego, wywoływana jest odpowiednia funkcja (*writeit*, *readit*) z trzema parametrami:
 - ↳ uchwytem *handle*
 - ↳ adresem bufora
 - ↳ liczbą bajtów do zapisu/odczytu

Potoki komunikatów (2)

- Oznaczenie końca rekordu (przy zapisie)
`xdrrec_endofrecord(XDR *xdrs,
int sendnow);`
- Pominięcie reszty rekordu (przy odczycie)
`xdrrec_skiprecord(XDR *xdrs);`
- Sprawdzenie zakończenia strumienia (przy odczycie)
`xdrrec_eof(XDR *xdrs);`

Definicja struktur danych (1)

XDR (rpcgen)

- stałe

```
const MAX = 512;
```

- typy wyliczeniowe

```
enum COLOR {  
    red    = 1,  
    green  = 2,  
    blue   = 4  
};
```

język C

```
#define MAX 512
```

```
enum COLOR {  
    red    = 1,  
    green  = 2,  
    blue   = 4  
};
```

```
typedef enum COLOR  
COLOR;
```

Definicja struktur danych (2)

XDR (rpcgen)

- stuktury

```
struct ST {  
    int a;  
    int b;  
};
```

język C

```
struct ST {  
    int a;  
    int b;  
};  
typedef struct ST ST;
```

Definicja struktur danych (3)

XDR (rpcgen)

- unie

```
union UN switch (int d)
{
    case 1: int a;
    case 2: char b;
    default: short c;
};
```

język C

```
struct UN {
    int d;
    union {
        int a;
        char b;
        short c;
    } UN_u;
};
typedef struct UN UN;
```

Definicja struktur danych (4)

XDR (rpcgen)

- tablice

```
typedef int tabf[10];  
typedef int tabv<10>;
```

- podobnie tablice bajtów

```
typedef opaque btv[10];  
typedef opaque btv<10>;
```

- łańcuchy znaków

```
typedef string s10<10>;  
typedef string sbo<>;
```

język C

```
typedef int tabf[10];  
typedef struct {  
    u_int tabv_len;  
    int *tabv_val;  
} tabv;
```

- typem bazowym w jest wówczas char

Sun RPC/XDR



Remote Procedure Call
eXternal Data Representation

Sun RPC

- ☛ trójwymiarowa identyfikacja procedur (nr programu, nr wersji, nr procedury)
- ☛ protokół RPC oparty na protokołach warstwy transportowej stosu TCP/IP (TCP lub UDP)
- ☛ wiązanie dynamiczne (portmap lub rpcbind)
- ☛ kanoniczny format reprezentacji danych (XDR)
- ☛ opis interfejsu w języku C-podobnym i przetwarzanie przez narzędzie **rpcgen**

Tworzenie aplikacji (1)

- przygotowanie pliku z opisem interfejsu w języku RPC (języku narzędzia **rpcgen**), zawierającego:
 - ↳ definicją struktur danych dla przekazywanych parametrów ,
 - ↳ specyfikację procedur zdalnych, obejmującą:
 - ✖ numer programu, numer wersji i numer procedury,
 - ✖ typy argumentów
 - ✖ typ zwracanej wartości.

Tworzenie aplikacji (2)

- ☛ wygenerowanie przez **rpcgen** plików do kompilacji w języku C:
 - pliku nagłówkowego z definicjami stałych,
 - plików z namiastką klienta i namiastką serwera,
 - plików zawierających szkielet procedur zdalnych (fragment programu serwera) i szkielet programu klienta dla potrzeb przetestowania procedur zdalnych,
 - plik do zarządzania kompilacją (`Makefile`),
 - plik z funkcjami XDR do konwersji zdefiniowanych typów danych.

Tworzenie aplikacji (3)

- uzupełnienie lub dokonanie niezbędnych modyfikacji w zawartości wygenerowanych plików
 - ↳ uzupełnienie plików zawierających szkielety procedur zdalnych,
 - ↳ uzupełnienie/modyfikacja programu klienta,
 - ↳ uzupełnienie/modyfikacja pliku Makefile.

Tworzenie aplikacji (4)

- kompilacja programów,
- testowanie działania procedur zdalnych,
 - ↳ ewentualne przygotowanie aplikacji w wersji zwartej (nie rozproszonej) w celu przetestowania działania samych procedur,
- przygotowanie programu klienta korzystającego z procedur zdalnych w sposób wynikający z potrzeb aplikacji.

Język RPC — specyfikacja procedur zdalnych

```
program NAZWA_PROGRAMU {  
    version NAZWA_WERSJI1 {  
        typ nazwa_proc1(typ_arg)=nr_proc;  
        typ nazwa_proc2(typ_arg)=nr_proc;  
        ⋮  
    }=numer_wersji;  
    version NAZWA_WERSJI2 {  
        typ nazwa_proc1(typ_arg)=nr_proc;  
        typ nazwa_proc2(typ_arg)=nr_proc;  
        ⋮  
    }=numer_wersji;  
}=numer_programu;
```

Dostępne zakresy numerów programów

- ☛ 0 – 0x1FFFFFFFFF — numery standardowe, zdefiniowane (przydzielane) przez firmę Sun,
- ☛ 0x20000000 – 0x3FFFFFFFFF — przydzielone przez użytkownika,
- ☛ 0x40000000 – 0x5FFFFFFFFF — zarezerwowane dla aplikacji, które generują numery programów dynamicznie,
- ☛ 0x60000000 – 0xFFFFFFFF — zarezerwowane.

Przykład opisu prostego interfejsu

```
program REMOTE_KILL {  
    version DEFAULT_SIGNALUM {  
        int rkill(int) = 1;  
    } = 1;  
} = 0x200000001;
```

Przykład opisu interfejsu z definicją struktury danych

```
struct rkill_params{  
    int pid;  
    int signum;  
};
```

```
program REMOTE_KILL {  
    version SPECIFIED_SIGNAL {  
        int rkill(rkill_params) = 1;  
    } = 2;  
} = 0x20000001;
```

Przykład opisu interfejsu dla **rpcgen -N**

```
program REMOTE_KILL {  
    version SPECIFIED_SIGNAL {  
        int rkill(int, int) = 1;  
    } = 2;  
} = 0x20000001;
```

Standard XDR

- ☛ Opis standardu — RFC 1014
- ☛ Kanoniczna reprezentacja danych oparta na formacie IEEE
- ☛ Deklaratywny język opisu struktur danych (zblizony do języka C)
- ☛ Koncepcja konwersji oparta na
 - ↳ potokach — miejscach przechowywania danych w formacie XDR
 - ↳ filtrach — procedurach konwersji pomiędzy własnym formatem maszyny a formatem kanonicznym (w obu kierunkach)

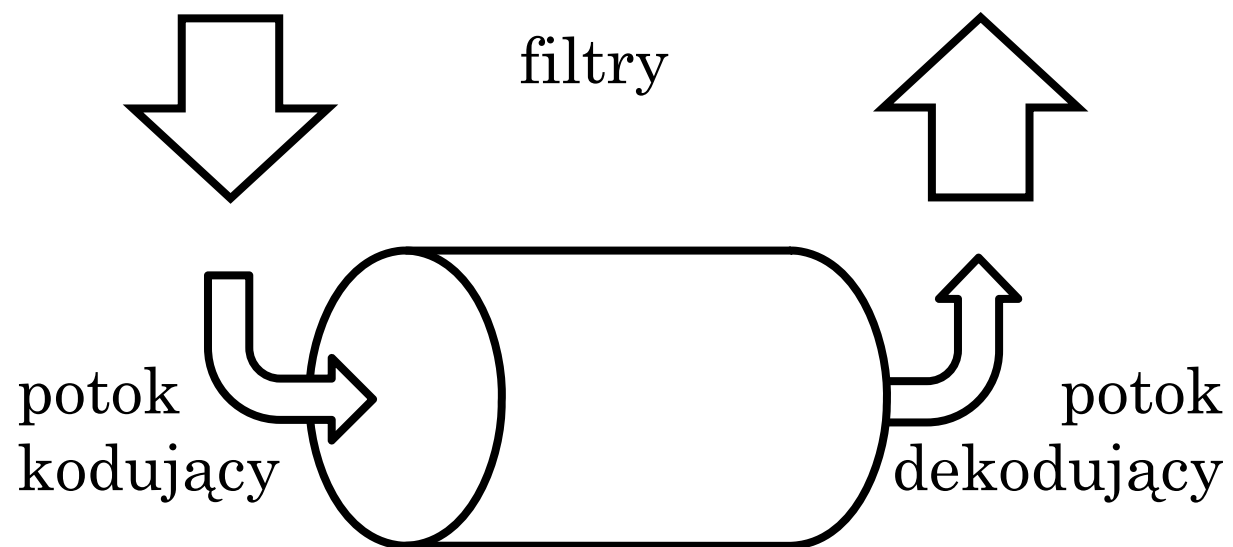
Potok XDR

- ☛ Potok jest miejscem składowania danych XDR, a dokładniej środkiem dostępu do nich w celu:
 - ↳ zapisu — potok kodujący
 - ↳ odczytu — potok dekodujący
- ☛ Rodzaje potoków:
 - ↳ potok na standardowym wejściu-wyjściu — na otwartym pliku
 - ↳ potok w pamięci — w obszarze pamięci opisanym przez adres i rozmiar w bajtach
 - ↳ potok komunikatów (rekordów) — na strumieniu danych, zdefiniowanym wraz z procedurami obsługi tego strumienia (zapis, odczyt)

Filtr XDR

- ☛ Filtr jest procedurą konwersji, służącą do realizacji dostępu do danych w potoku w celu zapisu (kodująca) lub odczytu (dekodująca) — zależnie od kierunku działania potoku.
- ☛ Rodzaje filtrów
 - ↳ filtr prosty — służy do konwersji typów prostych
 - ↳ filtr złożony — służy do konwersji typów złożonych (np. tablicowych, wskaźnikowych)
 - ↳ filtr pochodny — połączenie innych filtrów w ramach jednej procedury filtrującej

Potoki i filtry XDR



Filtry proste

- Filtr dla:
 - ↳ typów prostych: `bool_t`, `char`, `short`, `int`, `long`,
(również bez znaku) `float`, `double`
 - ↳ typu wyliczeniowego (`enum`)
 - ↳ typu `void`
- Ogólna postać procedury filtrującej (na przykładzie
typu `unsigned int`):

```
bool_t  
xdr_u_int(XDR* xdrs,  
          unsigned int *ptr);
```

Filtry złożone (1)

☛ Filtr do konwersji

- ☞ tablic bajtów o ustalonej lub zmiennej wielkości — `xdr_opaque`, `xdr_bytes`
- ☞ tablic elementów określonego typu o ustalonej lub zmiennej wielkości — `xdr_vector`, `xdr_array`
- ☞ łańcuchów znaków — `xdr_string`, `xdr_wrapstring`
- ☞ typów wskaźnikowych — `xdr_reference`, `xdr_pointer`
- ☞ unii — `xdr_union` (w praktyce dla statycznie zdefiniowanych typów unijnych stosowany jest filtr pochodny)

Filtry złożone (2)

- ☛ Ogólna postać filtru złożonego:

```
xdr_typ (XDR* xdrs, ptr, ...,  
        [xdrproc_t elproc]);
```

- ☛ W przypadku typu statycznego (ustalona zajętość pamięci) wskaźnik *ptr*, ma postać `char *ptr`, w przypadku typu o zmiennej wielkości (konieczność dynamicznej alokacji pamięci) `char **ptr`.
- ☛ Jeśli typ podstawowy (w przypadku tablic, wskaźników) nie jest z góry określony, konieczne jest wskazanie filtru XDR do konwersji elementu typu podstawowego.

Filtry złożone do konwersji tablic

```
xdr_array(xdrs, arrp, sizep, maxsize, elsize, elproc)
    XDR *xdrs;
    char **arrp;
    u_int *sizep, maxsize, elsize;
    xdrproc_t elproc;

xdr_vector(xdrs, arrp, size, elsize, elproc)
    char *arrp;
    u_int size, elsize; // pozostałe parametry – j.w.

xdr_bytes(xdrs, sp, sizep, maxsize)
    char **sp; // pozostałe parametry – j.w.

xdr_opaque(xdrs, cp, cnt)
    char *cp;
    u_int cnt;
```

Filtry złożone do konwersji łańcuchów znaków

```
xdr_string(xdrs, sp, maxsize)
```

```
    XDR *xdrs;
```

```
    char **sp;
```

```
    u_int maxsize;
```

```
xdr_wrapstring(xdrs, sp)
```

```
    ≡ xdr_string(xdrs, sp, MAXUN.UNSIGNED );
```

`xdr_wrapstring` nie wymaga podania rozmiaru (liczby znaków łańcucha)

Filtry złożone do konwersji struktur wskaźnikowych

```
xdr_reference(xdrs, pp, size, proc)
```

```
    XDR *xdrs;  
    char **pp;  
    u_int size;  
    xdrproc_t proc;
```

```
xdr_pointer(xdrs, objpp, objsize, xdrobj)
```

```
    XDR *xdrs;  
    char **objpp;  
    u_int objsize;  
    xdrproc_t xdrobj;
```

`xdr_pointer` w przeciwieństwie do `xdr_reference` interpretuje wskaźnik pusty, co umożliwia obsługę rekurencyjnych struktur danych

Przykład filtru pochodnego

Definicja struktury

```
struct struktura {  
    int x;  
    long y;  
    char c;  
    short s;  
};
```

Filtr XDR

```
bool_t  
xdr_struktura (XDR *xdrs,  
               struktura *objp) {  
    if (!xdr_int (xdrs,  
                  &objp->x)) return FALSE;  
    if (!xdr_long (xdrs,  
                   &objp->y)) return FALSE;  
    if (!xdr_char (xdrs,  
                   &objp->c)) return FALSE;  
    if (!xdr_short (xdrs,  
                    &objp->s)) return FALSE;  
    return TRUE;  
}
```

Zarządzanie pamięcią

- ☛ Przekazanie adresu pustego wskaźnika (typu `char**`) przy konwersji dekodującej spowoduje dynamiczną alokację pamięci przez filtr XDR.
- ☛ Zwolnienie obszaru dynamicznie zaalokowanej pamięci przez filtr XDR musi nastąpić w programie aplikacyjnym.
- ☛ Ogólna postać funkcji zwalniania pamięci:

```
xdr_free(xdrproc_t proc,  
         char* objp);
```

Tworzenie potoków XDR

- ☛ Potok na standardowym wejściu-wyjściu:

```
void xdrstdio_create(XDR *xdrs,  
                    FILE *file, enum xdr_op op);
```

- ☛ Potok w pamięci:

```
void xdrmem_create(XDR *xdrs, char *addr,  
                  u_int size, enum xdr_op op);
```

- ☛ Potok komunikatów:

```
void xdrrec_create(XDR *xdrs,  
                  u_int sendsize, u_int recvsize,  
                  char *handle,  
                  int (*readit)(char*,char*,int),  
                  int (*writeit)(char*,char*,int));
```

Potoki komunikatów (1)

- ☛ Ustalenie kierunku potoku musi nastąpić po jego utworzeniu (np. `xdrs -> x_op = XDR_ENCODE`)
- ☛ Rozmiary buforów (parametry *sendsize* i *recvsize*) mogą mieć wartość 0, co oznacza przyjęcie wartości domyślnych.
- ☛ Gdy konieczne jest opróżnienie bufora wyjściowego (wysyłanie danych) lub zapełnienie bufora wejściowego, wywoływana jest odpowiednia funkcja (*writeit*, *readit*) z trzema parametrami:
 - ↳ uchwytem *handle*
 - ↳ adresem bufora
 - ↳ liczbą bajtów do zapisu/odczytu

Potoki komunikatów (2)

- Oznaczenie końca rekordu (przy zapisie)
`xdrrec_endofrecord(XDR *xdrs,
int sendnow);`
- Pominięcie reszty rekordu (przy odczycie)
`xdrrec_skiprecord(XDR *xdrs);`
- Sprawdzenie zakończenia strumienia (przy odczycie)
`xdrrec_eof(XDR *xdrs);`

Definicja struktur danych (1)

XDR (rpcgen)

- stałe

```
const MAX = 512;
```

- typy wyliczeniowe

```
enum COLOR {  
    red    = 1,  
    green  = 2,  
    blue   = 4  
};
```

język C

```
#define MAX 512
```

```
enum COLOR {  
    red    = 1,  
    green  = 2,  
    blue   = 4  
};
```

```
typedef enum COLOR  
COLOR;
```

Definicja struktur danych (2)

XDR (rpcgen)

- stuktury

```
struct ST {  
    int a;  
    int b;  
};
```

język C

```
struct ST {  
    int a;  
    int b;  
};  
typedef struct ST ST;
```


Definicja struktur danych (3)

XDR (rpcgen)

- unie

```
union UN switch (int d)
{
    case 1: int a;
    case 2: char b;
    default: short c;
};
```

język C

```
struct UN {
    int d;
    union {
        int a;
        char b;
        short c;
    } UN_u;
};
typedef struct UN UN;
```

Definicja struktur danych (4)

XDR (rpcgen)

- tablice

```
typedef int tabf[10];  
typedef int tabv<10>;
```

- podobnie tablice bajtów

```
typedef opaque btv[10];  
typedef opaque btv<10>;
```

- łańcuchy znaków

```
typedef string s10<10>;  
typedef string sbo<>;
```

język C

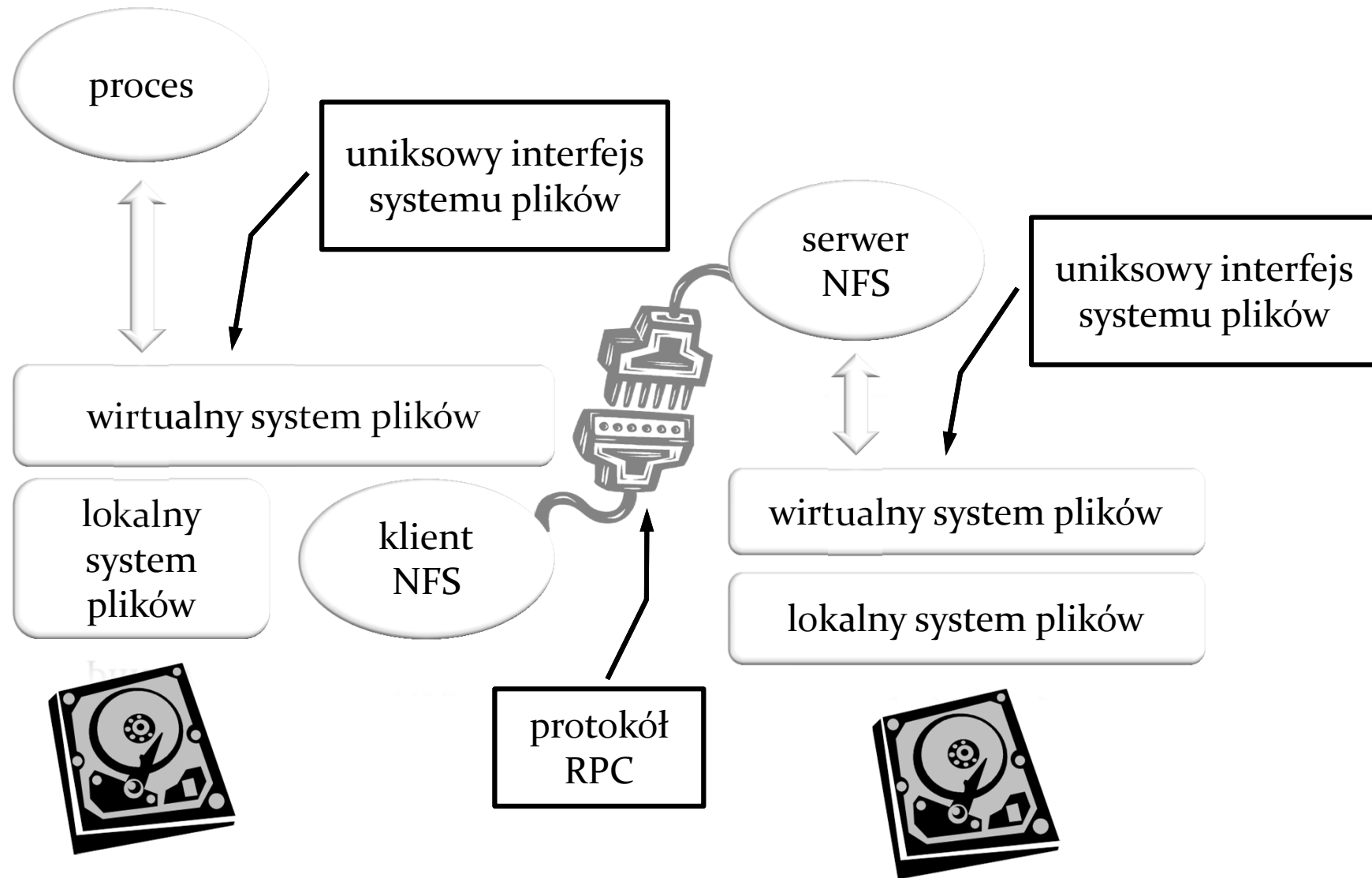
```
typedef int tabf[10];  
typedef struct {  
    u_int tabv_len;  
    int *tabv_val;  
} tabv;
```

- typem bazowym w jest wówczas char

Idempotentność i bezstanowość

Przykład — Network File System

Architektura NFS



Uniksowy interfejs systemu plików

- `open (pathname, flags) → fd`
- `creat (pathname, mode) → fd`
- `read (fd, buf, count)`
- `write (fd, buf, count)`
- `lseek (fd, offset, whence)`
- `close (fd)`
- `unlink (pathname)`

Interfejs zdalnego dostępu do pliku — protokół oparty na RPC

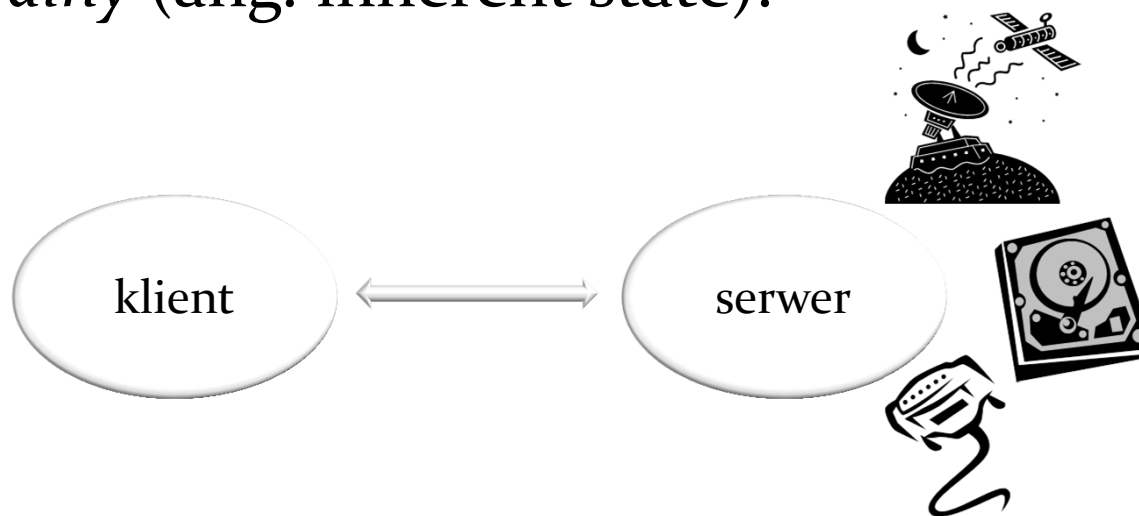
- lookup (dirfh, name) → fh, attr
- create (dirfh, name, attr) → newfh, attr
- read (fh, offset, count) → data, attr
- write (fh, data, offset, count) → attr
- remove (dirfh, name) → status

Odwzorowanie interfejsu uniksowego na interfejs zdalnego dostępu

- Jak wygląda realizacja uniksowej funkcji *open*?
- Jak wygląda realizacja uniksowej funkcji *creat*?
- Gdzie przechowywane są dane identyfikujące **otwarty** plik?
- Jak wygląda realizacja uniksowej funkcji *read/write*?
- Jak wygląda realizacja uniksowej funkcji *lseek*?

Problem opisu stanu systemu

- Stan systemu/obiektu z perspektywy klienta postrzegany jest poprzez odpowiedzi serwera — jest to tzw. *stan postrzegany* (ang. observable state).
- Stan systemu po stronie serwera może być współtworzony przez niezależne zasoby — jest to tzw. *stan integralny* (ang. inherent state).



Idempotentność procedur

- Procedura/metoda/operacja zdalna jest idempotentna
 - w sensie postrzegania (ang. observably idempotent), jeśli jej wywołanie z określonymi wartościami parametrów zawsze daje taki sam efekt (wynik, następstwa), tzn. efekt ten w sposób jednoznaczny jest zdeterminowany wartościami parametrów.
 - w sensie integralnym (ang. inherently idempotent), jeśli jej wywołanie z określonymi wartościami parametrów i przy określonym stanie zasobów zawsze daje taki sam wynik (wynik w sposób jednoznaczny jest zdeterminowany wartościami parametrów oraz stanem niezależnych zasobów)

Bezstanowość

- Serwer/obiekt jest bezstanowy (ang. stateless)
 - w sensie postrzegania, jeśli wszystkie jego procedury/metody/operacje są idempotentne w sensie postrzegania.
 - w sensie integralnym, jeśli wszystkie jego procedury/metody/operacje są idempotentne w sensie integralnym.

Współbieżność w środowisku Java

Obsługa i elementarna synchronizacja wątków

Zagadnienia

- Tworzenie wątków
- Stany wątków i ich zmiana
- Demony
- Grupy wątków
- Synchronizacja wątków
 - ↳ wzajemne wykluczanie
 - ↳ oczekiwanie na zmiennych warunkowych
 - ↳ **pakiet `java.util.concurrent`**

Klasa `java.lang.Thread`

Interfejs `java.lang.Runnable`

- Wątek reprezentowany jest w procesie na JVM przez obiekt klasy **Thread** (w szczególności jej pochodnej).
- Programem głównym wątku jest metoda **run()** klasy wywiedzionej z **Thread** lub dowolnej klasy implementującej interfejs **Runnable**.

Tworzenie wątków

- ☛ Dziedziczenie z klasy **Thread**
 - ↳ definicja klasy pochodnej od **Thread**,
 - ↳ utworzenie obiektu zdefiniowanej klasy.
- ☛ Implementacja interfejsu **Runnable**
 - ↳ definicja klasy implementującej interfejs **Runnable**,
 - ↳ utworzenie obiektu zdefiniowanej klasy,
 - ↳ utworzenie obiektu klasy **Thread** z przekazaniem referencji do utworzonego obiektu klasy implementującej **Runnable**.

Tworzenie wątków poprzez dziedziczenie z klasy **Thread**

- Zdefiniowanie klasy **MThread** wywiedzionej z klasy **Thread** (implementacja w tej klasie metody **run()**, która zawiera program wątku)

```
class MThread extends Thread {  
    void run(){  
        ...  
    }  
}
```

- Utworzenie obiektu **th** zdefiniowanej klasy **MThread**
MThread th;
th = new MThread();

Tworzenie wątków poprzez implementację interfejsu **Runnable**

- ☛ Zdefiniowanie klasy **MClass** implementującej **Runnable** oraz implementacja w tej klasie metody **run()**, która zawiera program wątku

```
class MClass implements Runnable {  
    void run(){  
        ...  
    }  
}
```

- ☛ Utworzenie obiektu **obj** zdefiniowanej klasy **MClass**
MClass obj = new MClass();
- ☛ Utworzenie obiektu **th** klasy **Thread**
Thread th = new Thread(obj);

Definicja interfejsu **Runnable**

```
public interface Runnable {  
    public abstract void run();  
}
```

Implementacja metody **run()** w klasie **Thread**

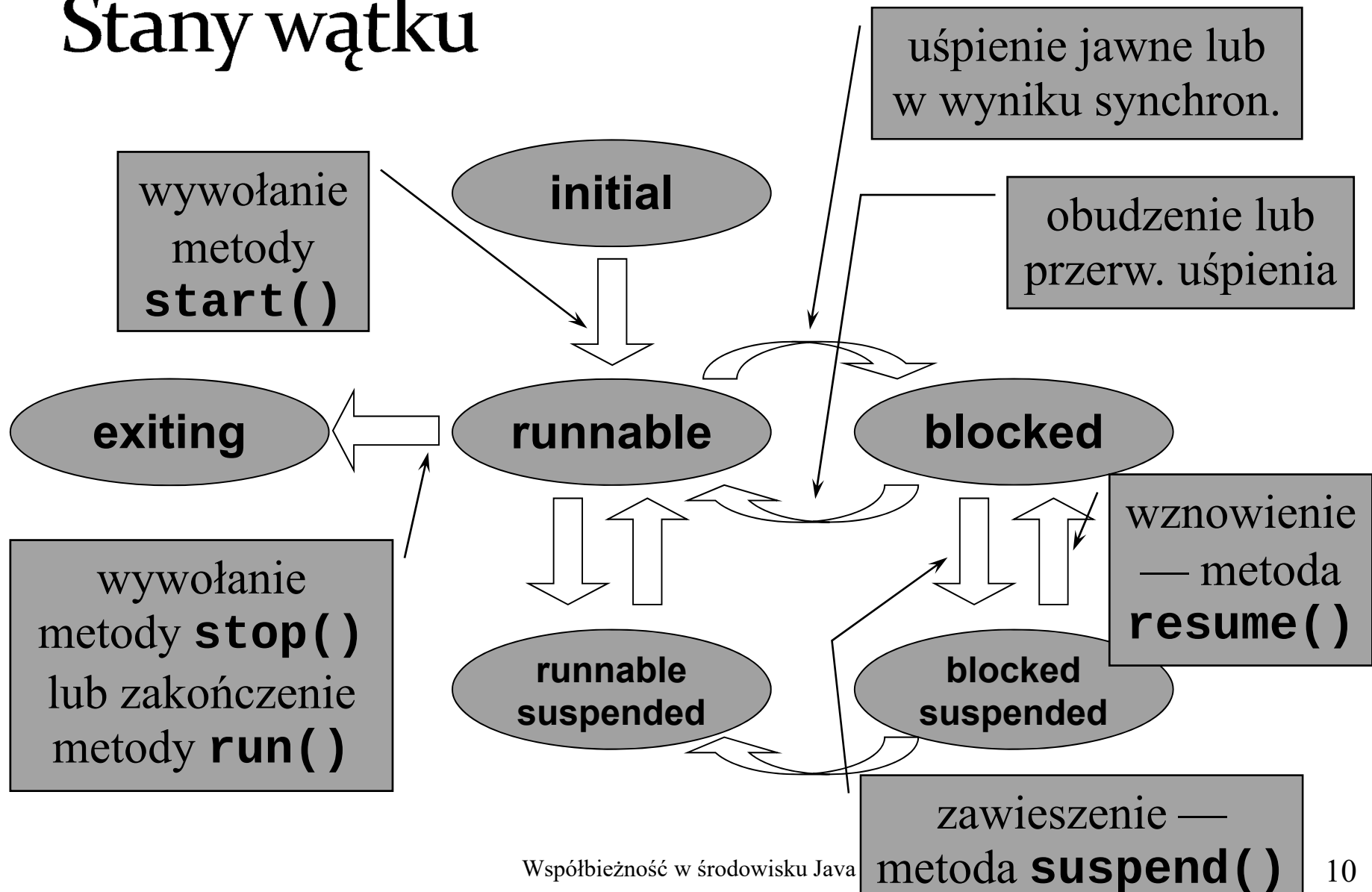
```
private Runnable target;  
public void run() {  
    if (target != null)  
        target.run();  
}
```

Referencja **target** ustawiana jest w konstruktorze, jeśli zostanie przekazany parametr klasy implementującej **Runnable**.

Konstruktory klasy Thread

- `Thread( );`
- `Thread(Runnable target);`
- `Thread(String name);`
- `Thread(Runnable target, String name);`

Stany wątku



Klasa **Thread** — zmiana stanu wątków

- **void start()** — uruchomienie wątku,
- **void stop()** — zakończenie działania wątku,
- **void run()** — metoda wykonywana przez wątek (główny program wątku),
- **void suspend()** — zawieszenie wątku (wątek nie zwalnia blokad),
- **void resume()** — wznowienie wykonywania zawieszonego wątku,
- **void interrupt()** — przerwanie oczekiwania wątku w stanie zablokowania.

Klasa **Thread** — zmiana stanu wątków (metody statyczne)

- **static void sleep(long milsec [, int nanosec])** — uśpienie wątku na podany okres czasu,
- **static void yield()** — „oddanie procesora” innemu wątkowi o tym samym priorytecie.
- Zmiana stanu następuje w wątku wywołującym (wątek wywołuje te metody w celu zmiany własnego stanu)

Przerywanie wątków

- Przerywanie — wywołanie metody **interrupt()** na obiekcie wątku — przerywa oczekiwanie wątku w (np. **sleep**, **join**, **wait**) poprzez zgłoszenie wyjątku **InterruptedException**.
- Jeśli wątek nie jest w stanie *blocked*, fakt przerywania jest odnotowywany poprzez ustawienie odpowiedniej flagi (*interrupted status*).
- Metoda statyczna **Thread.interrupted()** zwraca *true* jeśli flaga jest ustawiona (dla bieżącego wątku) i ją kasuje.
- Metoda **isInterrupted()** (na obiekcie wątku) zwraca tę informację dla danego wątku, ale nie kasuje flagi.

Klasa **Thread** — nadawanie nazw wątkom

- **void setName(String name)** — przypisanie nazwy do wątku,
- **String getName()** — odczytanie przypisanej nazwy.
- Z punktu widzenia systemu nazewnictwo wątków nie ma żadnego znaczenia, jest również raczej mało istotne dla użytkownika.

Klasa **Thread** — priorytety wątków

- **void setPriority(int priority)** — ustawianie priorytetu wątku,
- **int getPriority()** — odczytanie priorytetu wątku.
- Stałe (**final**) w klasie **Thread**:
 - ↳ **Thread.MIN_PRIORITY**
 - ↳ **Thread.MAX_PRIORITY**
 - ↳ **Thread.NORM_PRIORITY**
- Większa wartość oznacza wyższy priorytet.

Klasa **Thread** — inne metody

- **void join([long msec [, int nanosec]])** — oczekiwanie na zakończenie wątku (można podać czas oczekiwania),
- **boolean isAlive()** — sprawdzenie, czy wątek działa (zwraca **true** jeśli wątek został uruchomiony przez **start()**, ale nie zakończył jeszcze działania — wykonywanie metody **run()** nie dobiegło końca).

Klasa **Thread** — inne metody statyczne

- **static Thread currentThread()** — zwraca obiekt reprezentujący aktualnie wykonywany wątek,
- **static int enumerate(Thread threadArray[])** — zwraca obiekty reprezentujące wszystkie wątki procesu,
- **static int activeCount()** — zwraca liczbę aktywnych wątków procesu.

Demony

- Demon jest takim wątkiem, który kończy swoje działanie po zakończeniu ostatniego wątku użytkownika.
- **`void setDaemon(boolean on)`** — w zależności od wartości parametru **`on`** zmienia wątek użytkownika na wątek-demon lub odwrotnie,
- **`boolean isDaemon()`** — sprawdza, czy wątek jest demonem.

Grupowanie wątków

- Łączenie wątków w grupy ma na celu ułatwienie zarządzania zbiorami logicznie powiązanych ze sobą wątków (np. grupa wątków w serwerze do obsługi określonego klienta na połączeniu sieci.)
- Wątek musi zostać przypisany do grupy w momencie tworzenia i pozostaje w niej do końca swego istnienia.
- Grupy tworzą hierarchię wynikającą z zawierania się jednych grup w innych (każda nowo tworzona grupa jest częścią innej grupy).

Konstruktory klasy **Thread** z uwzględnieniem grupowania

**Thread(ThreadGroup group, Runnable
target);**

Thread(ThreadGroup group, String name);

**Thread(ThreadGroup group, Runnable
target, String name);**

Tworzenie grupy wątków

- Grupa wątków reprezentowana jest przez obiekt klasy **ThreadGroup**.
- Konstruktory klasy **ThreadGroup**:
 - ↳ **ThreadGroup(String name)** — utworzenie nowej grupy, która jest podgrupą grupy wątku bieżącego,
 - ↳ **ThreadGroup(ThreadGroup parent, String name)** — utworzenie nowej grupy, która jest podgrupą grupy wskazanej.

Operowanie na grupach wątków

- **void stop()** — zakończenie działania wszystkich wątków w grupie,
- **void suspend()** — zawieszenie wszystkich wątków w grupie,
- **void resume()** — wznowianie wykonywania zawieszonych wszystkich wątków w grupie.

Wyliczanie wątków w grupie

- `int enumerate(Thread list[])`
- `int enumerate(Thread list[], boolean recurse)`
- `int activeCount()`
- `int enumerate(ThreadGroup list[])`
- `int enumerate(ThreadGroup list[], boolean recurse)`

Usuwanie grupy wątków

- Do usuwania grupy wątków służy metoda **destroy()**.
- Metoda **destroy()** jest skuteczna, jeśli wszystkie wątki w grupie i podgrupach zostały zakończone.
- Metoda **destroy()** rekurencyjnie usuwa również wszystkie podgrupy grupy usuwanej.

Synchronizacja

Koordinacja wątków, spójność danych

Synchronizacja

- ☛ Synchronizacja procesów/wątków
 - ↳ koordynacja realizacji poszczególnych instrukcji (kroków, faz)
 - ↳ kontrola przepływu sterowania
- ☛ Synchronizacja danych
 - ↳ utrzymanie spójności danych
 - ↳ gwarancja dostępu do najświeższych wartości zmiennych/stanów obiektów (uwzględnienie wyników ostatnich modyfikacji)

Mechanizmy synchronizacji

- Poziom architektury systemu komputerowego
 - ↳ zapis/odczyt współdzielonych zmiennych (tzw. współdzielone rejestry)
 - ↳ złożone operacje realizowane niepodzielnie , np. test&set, exchange
- Poziom systemu operacyjnego
 - ↳ zarządzanie procesami/wątkami (ich stanem), integracja z mechanizmami przydziału procesora (szeregowania), np. semafony, zamki, zmienne warunkowe
- Poziom języka programowania
 - ↳ strukturalne mechanizmy synchronizacji udostępniające konstrukcje do wyrażania zależności i ograniczeń w dostępie do współdzielonych zasobów (monitory, regiony krytyczne)

Synchronizacja wątków

- Mechanizmy „niskopoziomowe”
 - ↳ Wzajemne wykluczanie blok/metoda **synchronized**
 - ↳ Oczekiwanie na spełnienie warunku **wait()**, **notify()**, **notifyAll()**
 - ↳ Blok **synchronized** oraz metody **wait()**, **notify()** i **notifyAll()** mogą być realizowane na dowolnym obiekcie (obiekcie klasy **Object**).
- Mechanizmy „wysokopoziomowe” — pakiet **java.util.concurrent** (od wersji 1.5)
 - ↳ Atomowe operacja na obiektach
 - ↳ Zamki (**Lock**) i zmienne warunkowe (**Condition**)
 - ↳ Semaforey (**Semaphore**), bariery (**CyclicBarrier**) itp.
 - ↳ Współbieżnie dostępne kolekcje (**ConcurrentHashMap**, **ConcurrentLinkedQueue**, **CopyOnWriteArrayList**, **CopyOnWriteArraySet**)

Metoda/blok **synchronized**

- Blok **synchronized** na danym obiekcie zajmuje zamek związany (integralnie) z tym obiektem
synchronized (obj){
 ...
}
- Metoda typu **synchronized** zajmuje zamek związany z obiektem, dla którego jest wywoływana, będzie zatem wykluczać wykonanie innych metod typu **synchronized** lub bloków **synchronized** na tym obiekcie.

Przykład metod **synchronized**

```
public class Konto {  
    private float kwota;  
    public synchronized boolean wypłata(float k) {  
        if (k <= kwota) {  
            kwota -= k;  
            return true;  
        }  
        return false;  
    }  
    public synchronized void wpłata (float k) {  
        kwota += k;  
    }  
}
```


Wzajemne wykluczanie fragmentu kodu metody

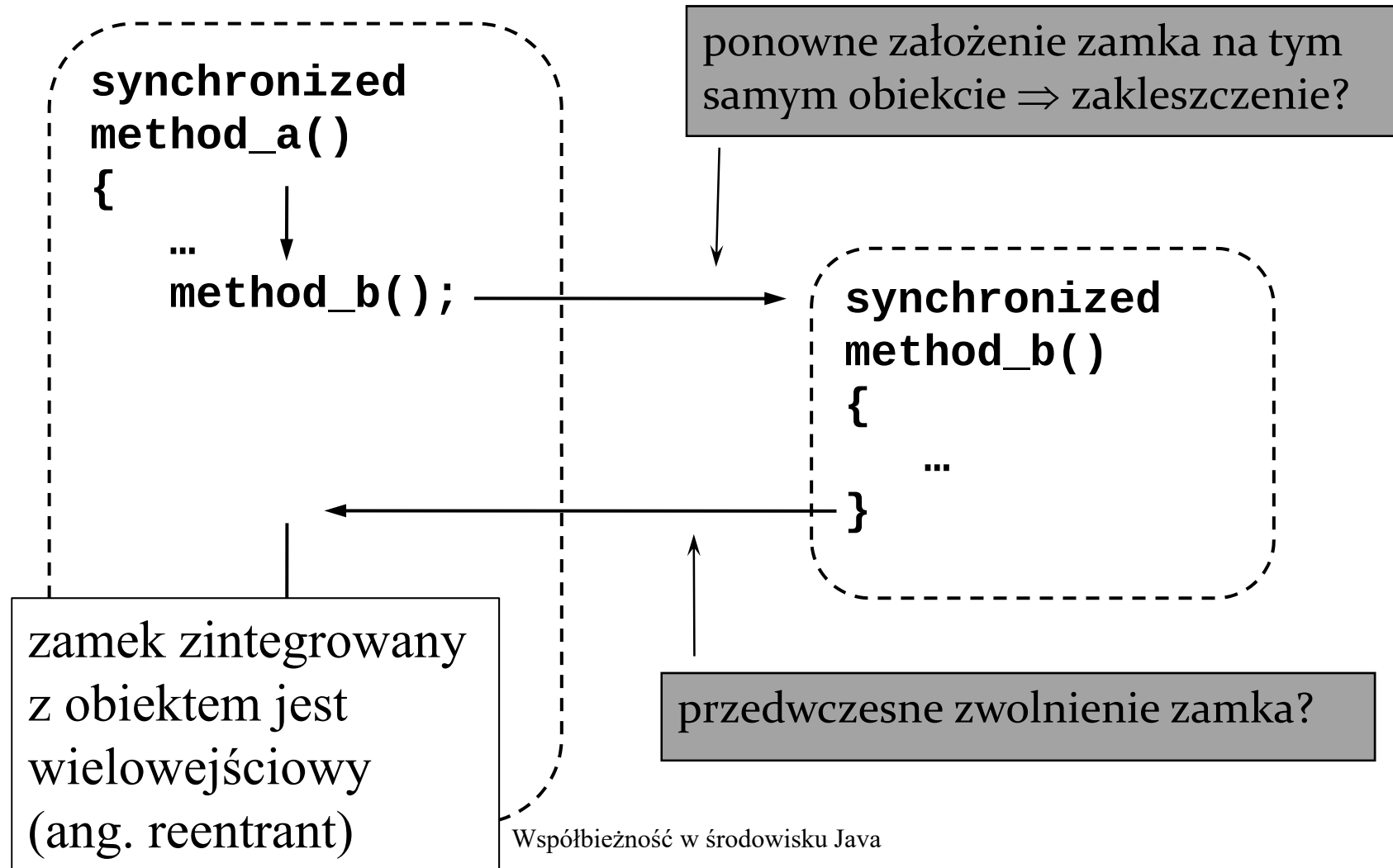
- Jeśli tylko fragment kodu metody ma się wykluczać z innymi metodami typu **synchronized**, można to osiągnąć przez utworzenie bloku **synchronized** na referencji **this** w implementacji tej metody.

```
{  
    ...  
    synchronized (this) {  
        ...  
    }  
}
```

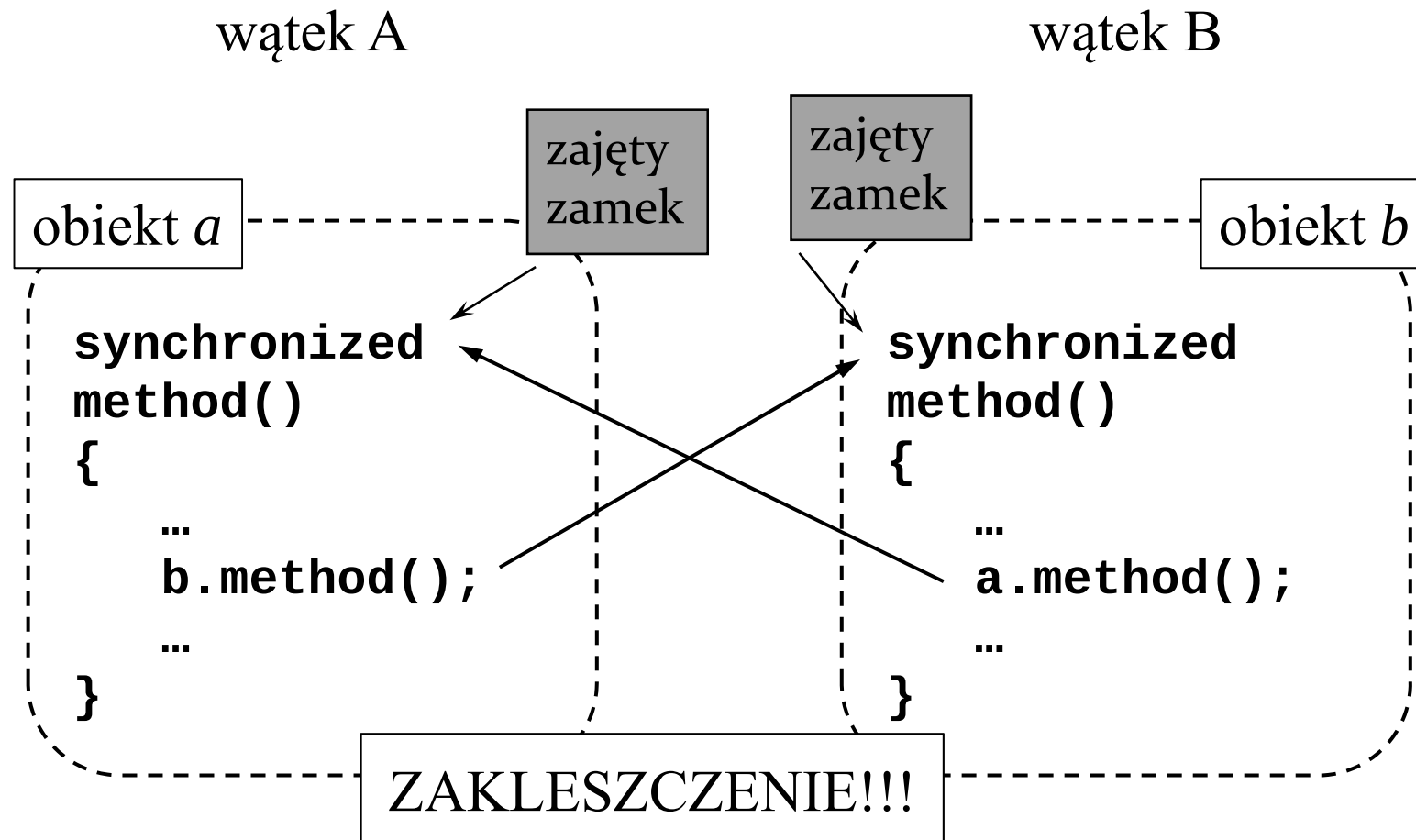
Pytanie

- ☛ Co się stanie, jeśli metoda typu **synchronized** zostanie wywołana z innej metody typu **synchronized** (czyli przez ten sam wątek)?
- ☛ Czy nastąpi zakleszczenie, a jeśli nie, to czy nie nastąpi przedwczesne zwolnienie blokady obiektu?

Przykład zagnieżdżonych blokad



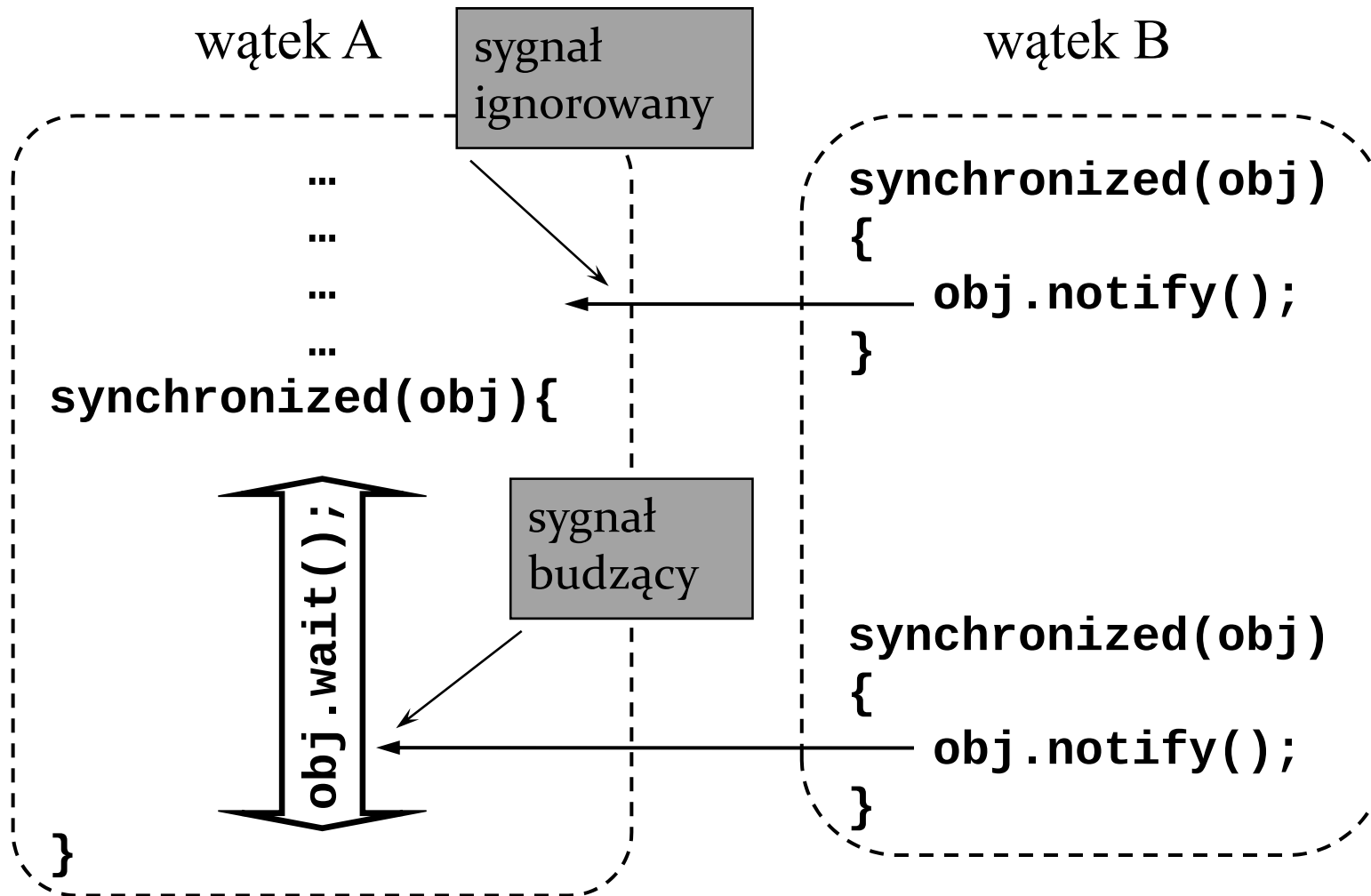
Przykład zagnieżdżonego wzajemnego wywołania metod przez różne wątki



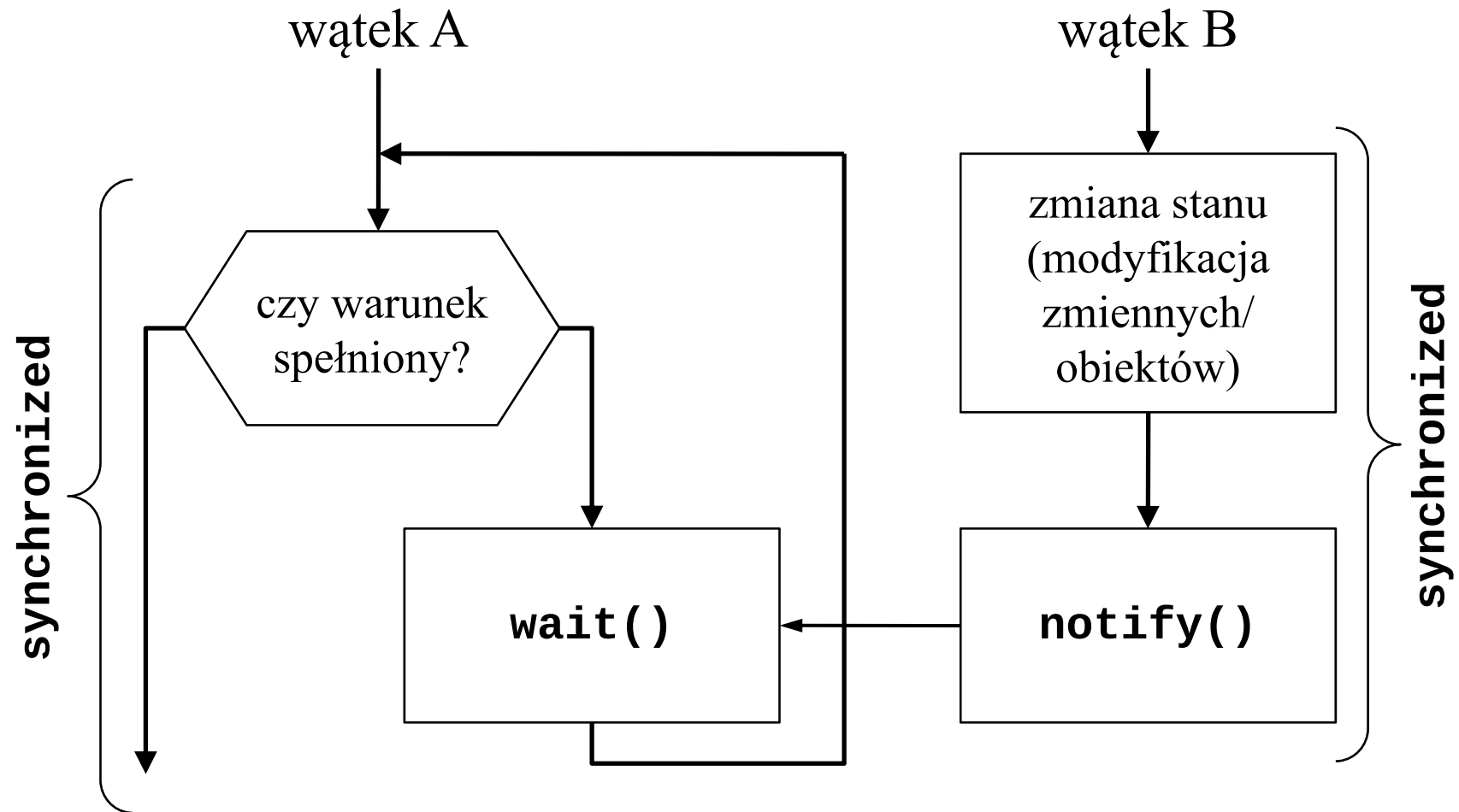
Oczekiwanie na spełnienie warunku

- **`void wait([long milsec [, int nanosec]])`** — czeka na spełnienie warunku (na sygnał wysyłany przez **`notify()`** lub **`notifyAll()`**),
- **`void notify()`** — wysyła sygnał do wątku oczekującego po wywołaniu metody **`wait()`** danego obiektu,
- **`void notifyAll()`** — wysyła sygnał do wszystkich wątków oczekujących po wywołaniu metody **`wait()`**,
- Metody **`wait()`**, **`notify()`** i **`notifyAll()`** muszą być wywoływane w bloku (metodzie) **`synchronized`** na tym samym obiekcie, w przeciwnym przypadku zgłaszany jest wyjątek **`NotOwnerException`**.

Budzenie wątków oczekujących na spełnienie warunku



Schemat synchronizacji na zmiennych warunkowych



Pytanie

- Czy przy pomocy mechanizmów synchronizacji w Java'ie da się zbudować monitor?

Synchronizacja i zarządzanie współbieżnością w środowisku Java

Podejścia do synchronizacji
Pakiet `java.util.concurrent`

Synchronizacja – aspekty

- Przepływ sterowania – kontrola kolejności realizacji operacji w ramach współbieżnych wątków
- Przepływ danych – dostępność aktualnych danych podczas realizacji współbieżnego dostępu

Synchronizacja – poziomy abstrakcji

- Architektura systemu komputerowego (procesor):
 - atomowy zapis/odczyt pamięci
 - atomowa realizacja operacji złożonych
- System operacyjny
 - integracja mechanizmów synchronizacji z kontrolą stanu (cyklu życia) wątku/procesu
- Język programowania
 - konstrukcje programotwórcze określające przepływ sterowania i sposób dostępu do współdzielonych danych

Operacja atomowa

- realizowana w całości (niepodzielność)
albo się zakończy, albo w ogóle się nie rozpocznie
- realizowana w „momencie” czasu (izolacja)
żadne jej efekty nie ujawniają się do momentu
zakończenia

Atomowy zapis/odczyt pamięci

- Atomowość gwarantowana dla zapisu/odczytu danych typu prostego: `int`, `float` (i podobnych) oraz referencji do obiektów.
- Brak atomowości dla zapisu/odczytu danych typu `long`, czy `double`!!!
- Atomowość gwarantowana dla zapisu/odczytu danych typu prostego określonych jako `volatile`.
- Brak atomowości dla pozornie atomowych operacji typu `++`, `--`, czy `+=`, `-=`.

Specyfikacja `volatile`

- Gwarancja, że dowolny wątek czytający tak określoną zmienną otrzyma ostatnią zapisaną w niej wartość
- Nie oznacza wyłączości dostępu do zmiennej (wzajemnego wykluczania)
- Zapis danej ulotnej powoduje aktualizację wcześniej modyfikowanych przez dany wątek nieulotnych zmiennych.

Atomowe zmienne

- Pakiet `java.util.concurrent.atomic`
- Możliwość atomowej realizacji złożonych operacji bez blokowania wątków
- Atomowe „typy proste”: `AtomicBoolean`, `AtomicInteger`, `AtomicLong`, `AtomicReference`.
- Typy złożone z możliwością atomowej realizacji operacji na poszczególnych elementach: `AtomicIntegerArray`, `AtomicLongArray`, `AtomicReferenceArray`

Operacje na atomowych zmiennych

- `get` / `set` – odczyt / ustawienie wartości,
- `getAndSet` – odczyt dotychczasowej wartości i ustawienie nowej wartości,
- `compareAndSet` – ustawienie nowej wartości pod warunkiem zgodności dotychczasowej wartości z wartością parametru
- `lazySet`, `weakCompareAndSet` – optymalizacje związane z realizacją pozostałych operacji w danym wątku (np. opóźnienie realizacji, brak gwarancji `happen-before`)

Operacje na atomowych zmiennych całkowitych

- `getAndAdd` / `addAndGet` – dodanie i zwrócenie wartości sprzed / po modyfikacji
- `decrementAndGet`, `getAndDecrement`, `incrementAndGet`, `getAndIncrement` – modyfikacja o 1 i zwrócenie wartości sprzed lub po modyfikacji

Współbieżne kolejki

- `BlockingQueue<E>`, `BlockingDeque<E>` — kolejka FIFO z możliwości synchronizacji dostępu:
 - blokowanie przy próbie pobrania elementu z pustej kolejki
 - blokowanie przy próbie wstawienia elementu do pełnej kolejki
- `TransferQueue<E>` — kolejka blokująca przy wstawianiu do momentu pobrania elementu (blokowanie producenta do momentu pobrania elementu przez konsumenta)
- `Exchanger<V>` — synchroniczna wymiana danych pomiędzy dwoma wątkami

Współbieżne kolekcje z dostępem bezpośrednim do elementów

- `ConcurrentMap<K, V>` — atomowa realizacji operacji zastępowania elementu, dodawania nowego elementu, usuwania elementu
- `ConcurrentNavigableMap<K, V>` — `ConcurrentMap` obejmujący realizację operacji z interfejsu `NavigableMap`

Obiekty do synchronizacji

- Klasa Semaphore
- Klasy CyclicBarrier, CountdownLatch i Phaser
- Pakiet `java.util.concurrent.locks`
- Interfejsy w pakiecie `locks`:
 - Lock — zamek do wzajemnego wykluczania
 - Condition — zmienna warunkowa do usypiania i budzenia (tworzona przez obiekt typu Lock)
 - ReadWriteLock — blokady współdzielone i wyłączne
- Klasy w pakiecie `locks`:
 - ReentrantLock, ReentrantReadWriteLock (wraz z wewnętrznymi klasami ReadLock i WriteLock)
 - Abstract*Synchronizer, LockSupport — do rozwijania własnych mechanizmów synchronizacji

Semafor (klasa Semaphore)

- Parametry konstruktora
 - `int permits` — liczba jednostek
 - `boolean fair` — gwarancja FIFO
- Nabywanie jednostek (opuszczanie)
 - `acquire`
 - `acquireUninterruptibly`
 - `tryAcquire`
- Zwalnianie jednostek (podnoszenie)
 - `release`
- Likwidowanie jednostek
 - `drainPermits`
 - `reducePermits (protected)`

Bariera (klasa `CyclicBarrier`)

- Parametry konstruktora:
 - `int parties` — liczba wątków do przełamania bariery
 - `Runnable barrierAction` — zadanie uruchamiane po przełamaniu bariery (przed uwolnieniem wątków)
- Operacje synchronizujące:
 - `await`, `await(long timeout, TimeUnit unit)` — oczekiwanie na przełamanie bariery
 - `reset` (uwaga na wyjątek `BrokenBarrierException`)
- Informacje o stanie bariery:
 - `getNumberWaiting`
 - `getParties`
 - `isBroken`

Odliczanie (klasa `CountDownLatch`)

- Parametr konstruktora:
 - `int count` — wartość początkowa, od której rozpoczyna się odliczanie
- Operacje synchronizujące:
 - `await`, `await(long timeout, TimeUnit unit)` — oczekiwanie na osiągnięcie zera (nie zmniejsza wartości licznika!)
 - `countDown` — zmniejszenie wartości licznika o 1
- Informacje o stanie licznika:
 - `getCount`
 - `toString`

Kontrola faz (klasa Phaser)

- Mechanizm bariery, umożliwiający bardziej elastyczną kontrolę przebiegu synchronizacji
- Rozdzielenie przejścia przez barierę od blokowania w oczekiwaniu na przełamanie bariery
 - operacje typu arrive
 - operacje typu awaitAdvance
- Możliwość sterowania liczbą zadań niezbędnych do przełamania bariery (do zakończenia fazy)
 - operacje typu register/deregister

Zamek (interfejs Lock)

- `lock`, `lockInterruptibly` — zajęcie zamka
- `tryLock`, `tryLock(long time, TimeUnit unit)` — nieblokująca próba zajęcia zamka
- `unlock` — zwolnienie zamka
- `newCondition` — utworzenie zmiennej warunkowej związanej z danym zamkiem

Zmienna warunkowa (interfejs Condition)

- `await`, `awaitUninterruptibly` — oczekiwanie na sygnał
- `await(long time, TimeUnit unit)`, `awaitNanos`, `awaitUntil` — oczekiwanie czasowe na sygnał
- `signal`, `signalAll` — wysłanie sygnału budzącego

Blokady współdzielone/wyłączne (interfejs `ReadWriteLock`)

- `Lock readLock()` — zwraca zamek do zakładania blokady współdzielonej
- `Lock writeLock()` — zwraca zamek do zakładania blokady wyłącznej

Zarządzanie wielowątkową realizacją zbioru zadań

- Interfejs `Executor` i pochodne — specyfikacja mechanizmu obsługi puli wątków
- Implementacja interfejsu `Executor` — definicja klas do zarządzania pulą wątków
- Fabryka `Executors` — wygodniejszy sposób tworzenia obiektów do zarządzania pulą wątków
- Interfejs `Callable` — interfejs asynchronicznie realizowanego zadania, które zwraca wynik lub wyjątek
- Interfejs `Future` — interfejs obiektu reprezentującego wynik zadania asynchronicznego

Interfejs Executor

- Metoda do zlecania realizacji zadania:
`void execute(Runnable command)`
- Sposób realizacji zleconych zadań zależy od implementacji interfejsu (np. `ThreadPoolExecutor`)
- Brak możliwości jakiegokolwiek kontrolowania realizacji zleconych zadań!!!

Obiekt klasy `ThreadPoolExecutor`

- Obiekt zarządza pulą wątków:
 - utrzymuje wątki w gotowości do realizacji zadań
 - powiększa pulę o dodatkowe wątki, gdy jest taka potrzeba
 - redukuje liczbę wątków, gdy nie ma zadań do realizacji
- Parametry konstruktora:
 - `int corePoolSize` — liczba wątków utrzymywanych w puli od momentu ich utworzenia,
 - `int maximumPoolSize` — górny limit liczby wątków
 - `long keepAliveTime, TimeUnit unit` — czas utrzymywania dodatkowych wątków (powyżej `corePoolSize`) w stanie bezczynności
 - `BlockingQueue<Runnable> workQueue` — kolejka zadań do realizacji

Zasady realizacji zadań

- Dopóki liczba współbieżnie realizowanych zadań nie przekroczy `corePoolSize`, każde zadanie otrzymuje nowy wątek i jest natychmiast realizowane.
- Po osiągnięciu `corePoolSize` wykonywanych wątków kolejne zadania są kolejgowane w `workQueue`.
- Po przekroczeniu pojemności `workQueue` uruchamiane są kolejne wątki.
- Po osiągnięciu limitu `maximumPoolSize` kolejne zadania (zlecane przez `execute`) są odrzucane (wyjątek `RejectedExecutionException`).

Fabryka obiektów — Executors

- `newSingleThreadExecutor` — pula 1-wątkowa
- `newFixedThreadPool` — pula ustalonej wielkości
- `newCachedThreadPool` — pula zmiennej wielkości
- `newScheduledThreadPool` — pula zadań uszeregowanych czasowo
- `newSingleThreadScheduledExecutor` — 1-wątkowa pula zadań uszeregowanych czasowo

Interfejs ExecutorService

- Pochodny od Executor (implementuje metodę execute)
- Dostarcza mechanizmy umożliwiające kontrolę realizacji zadań i udostępniające wyniki wykonania
- Kontrola realizacji całości zbioru zadań:
 - shutdown — zakończenie zlecenia zadań
 - shutdownNow — wymuszenie zakończenia realizacji zadań
 - isShutdown — sprawdzenie, czy nastąpiło zlecenie operacji shutdown lub shutdownNow
 - isTerminated — sprawdzenie, czy wszystkie zadania zostały wykonane po operacji shutdown lub shutdownNow
 - awaitTermination — oczekiwanie na zakończenie zadań po operacji shutdown lub shutdownNow

Interfejs Future<V>

- `boolean cancel(boolean mayInterruptIfRunning)` — anulowanie wykonywania zadania
- `boolean isCancelled()` — czy zadanie zostało anulowane przed normalnym zakończeniem
- `boolean isDone()` — czy zadanie się zakończyło (w dowolny sposób: normalnie, wyjątkiem, zostało anulowane)
- `V get(), get(long timeout, TimeUnit unit)` — oczekiwanie na wynik (na zakończenie)

Wynik typu Future w operacjach interfejsu ExecutorService

- `Future<?> submit(Runnable task)` — metoda `get` wyniku (typu `Future`) zwróci `null`
- `<T> Future<T> submit(Runnable task, T result)` — metoda `get` wyniku (typu `Future`) zwróci przekazany parametr `result`
- `<T> Future<T> submit(Callable<T> task)` — metoda `get` wyniku (typu `Future`) zwróci wynik zadania `task`, zgodnie z definicją interfejsu `Callable`

Interfejs Callable<V>

- Koncepcja zbliżona do interfejsu Runnable, ale w przeciwieństwie do interfejsu Runnable operacja `call` zwraca wynik i może sygnalizować wyjątek:

```
public interface Callable<V> {  
    V call() throws Exception;  
}
```

Przedkładanie kolekcji zadań do realizacji (interfejs `ExecutorService`)

- `invokeAll`
 - `<T> List<Future<T>> invokeAll (Collection<? extends Callable<T>> tasks)`
 - `<T> List<Future<T>> invokeAll (Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit)`
- `invokeAny`
 - `<T> T invokeAny (Collection<? extends Callable<T>> tasks)`
 - `<T> T invokeAny (Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit)`

Realizacja zadań szeregowanych czasowo (interf. ScheduledExecutorService)

- Zadanie jednorazowe wykonywane z opóźnieniem:
`<V> ScheduledFuture<V> schedule (
 Callable<V> callable,
 long delay, TimeUnit unit);`
`ScheduledFuture<?> schedule (
 Runnable command,
 long delay, TimeUnit unit);`
- Zadania powtarzane okresowo:
`ScheduledFuture<?> scheduleAtFixedRate (
 Runnable command, long initialDelay,
 long period, TimeUnit unit);`
`ScheduledFuture<?> scheduleWithFixedDelay (
 Runnable command, long initialDelay,
 long delay, TimeUnit unit);`

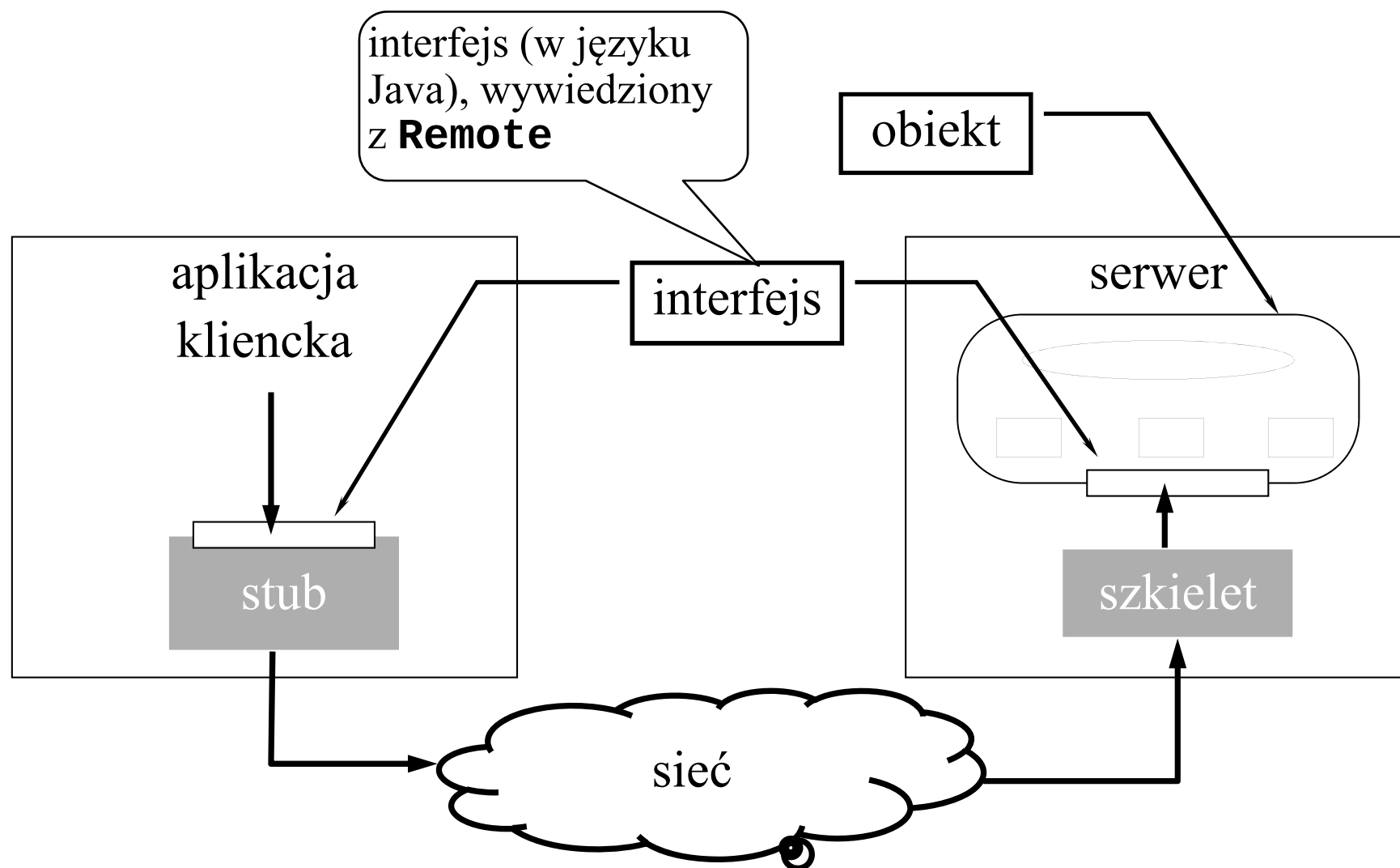
Obiekty niezmiennie

- Stan obiektu po jego skonstruowaniu nie zmienia się.
- Strategia definiowania klasy dla obiektu niezmiennego:
 1. brak metod modyfikujących
 2. wszystkie zmienne instancji określone jako final i private
 3. zabezpieczenie przed redefinicją metod w podklasach (deklaracja klasy jako final, prywatny konstruktor i tworzenie obiektów w fabrykach).
 4. zabezpieczenie przed zmianą obiektów modyfikowalnych, do których istnieją referencje w danym obiekcie
 - brak metod modyfikujących takie obiekty
 - brak powielania referencji do takich obiektów (tworzenie kopii obiektów zamiast przekazywania/zwracania referencji)

Wywoływanie metod zdalnych

Podejście obiektowe do budowy systemów
rozproszonych

Wywoływanie metod zdalnych — model systemu



Obiekt zdalny w środowisku Java

- Mechanizm RMI umożliwia tworzenie obiektów zdalnych (brak bezpośredniego wsparcie dla tworzenia obiektów rozproszonych)
- Jedyna forma zdalnego dostępu polega na wywoływaniu metod wyspecyfikowanych w interfejsie wywiedzionym (dziedziczącym) `java.rmi.Remote`
- Interfejs zdefiniowany jest w języku implementacji
- Obiekt może implementować wiele interfejsów
- Ten sam interfejs może być implementowany w wielu klasach i występować w wielu instancjach (obiektach) każdej klasy
- Interfejs traktowany jest jak typ danych

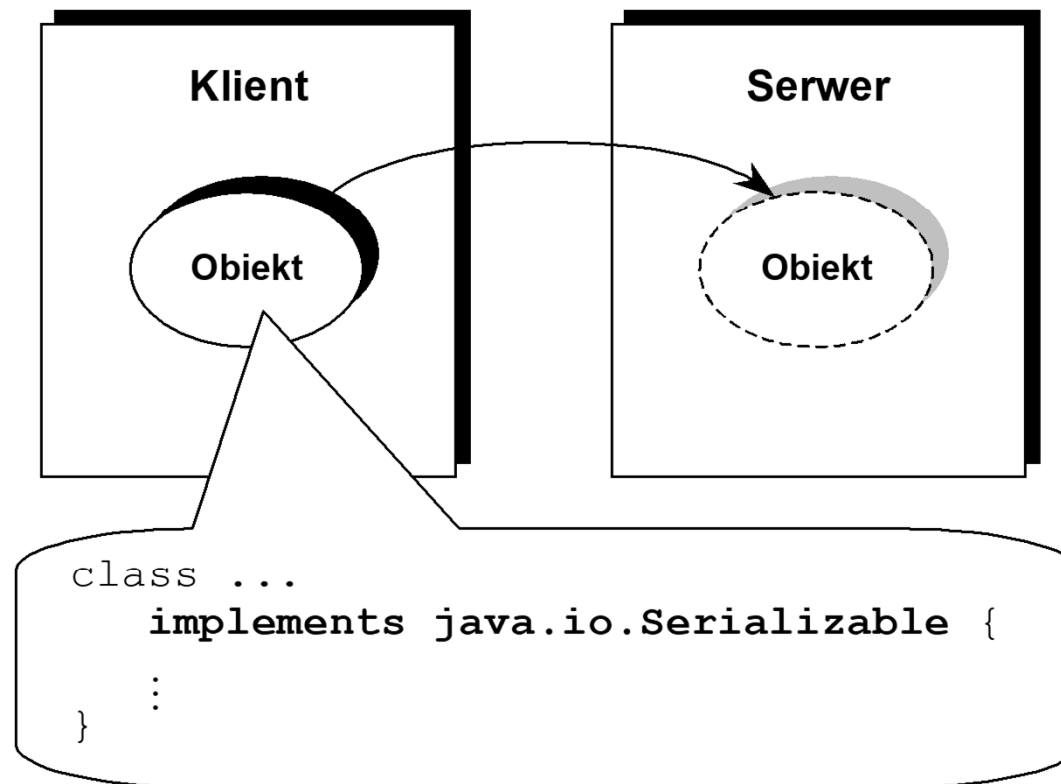
Dostępność obiektu

- Informacja o typie obiektu (czyli o zdalnym interfejsie) dostępna jest w czasie kompilacji.
- Wiązanie obiektu jest jawne i odbywa się w czasie wykonania
- Trwałość obiektu
 - obiekt udostępniany przez `UnicastRemoteObject` ma charakter przejściowy (istnieje tylko w czasie działania serwera)
 - dostępny jest mechanizm obiektów aktywowalnych, ale brak bezpośredniego wsparcia dla utrwalania stanu obiektu

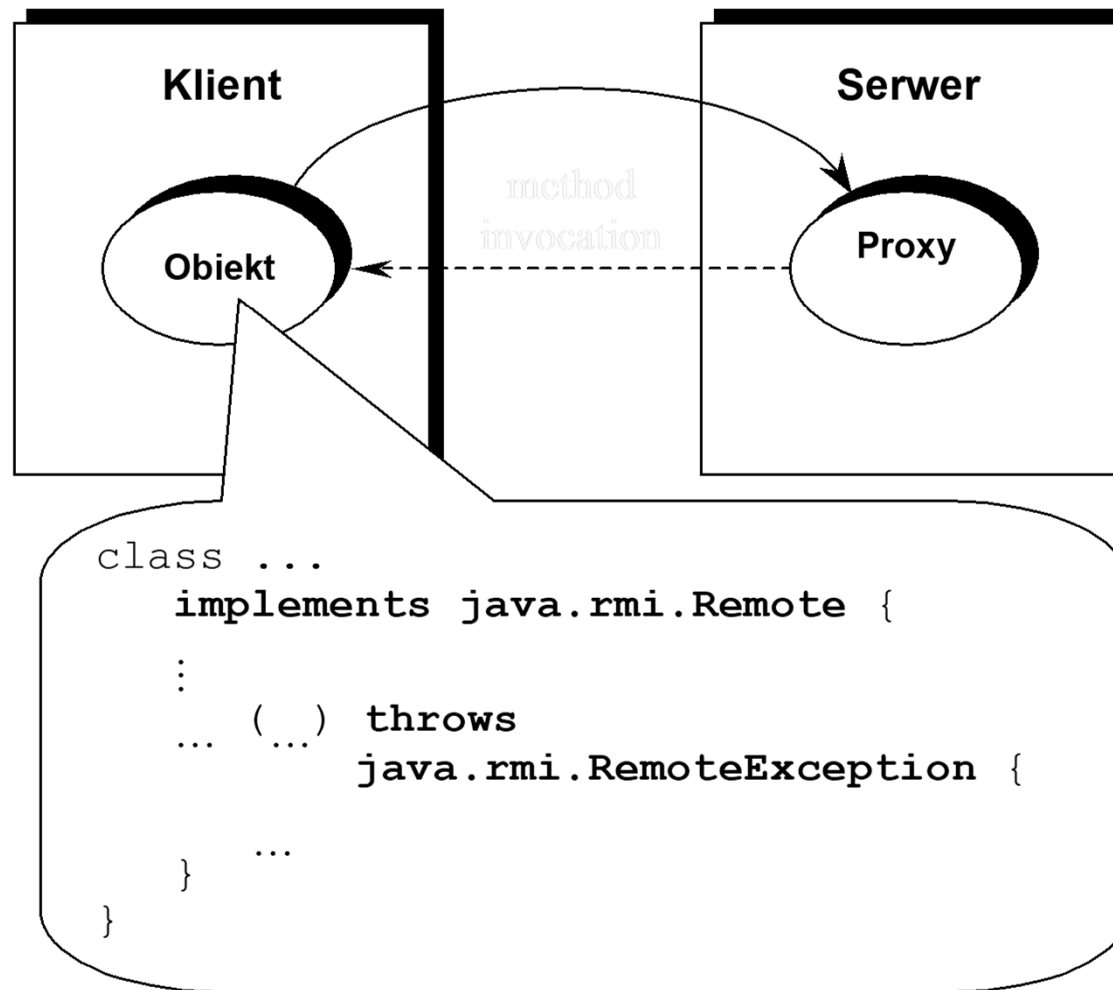
Przekazywanie obiektów jako parametrów

- Obiekty mogą być przekazywane przez wartość (kopie) — konieczna jest deklaracja implementacji interfejsu `java.io.Serializable` w klasie obiektu.
- Obiekty zdalne (implementujące interfejs `Remote`) przekazywane są przez referencję — do zdalnej metody przekazywana jest zdalna referencja, za którą udostępniany jest proxy (stub).
- Brak bezpośredniego wsparcia dla przekazywania parametrów przez kopiowanie i odtwarzanie.

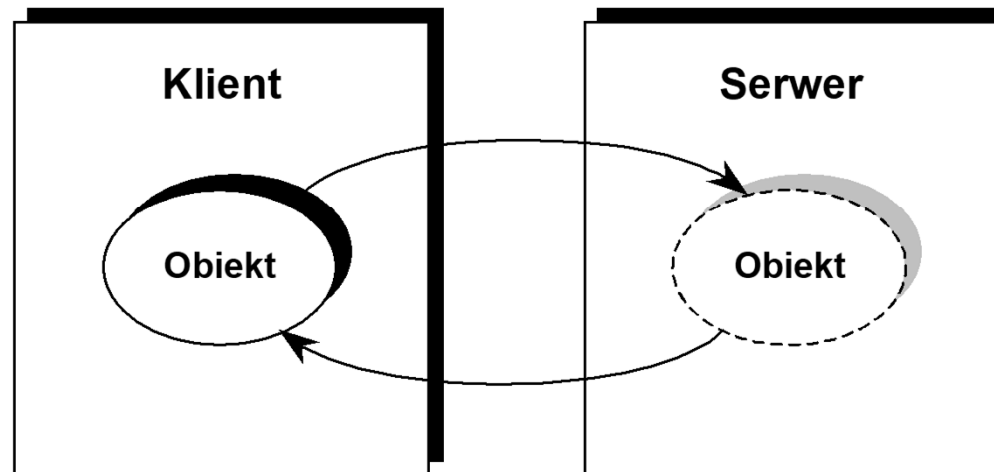
Przekazywanie przez wartość — przykład w Java RMI



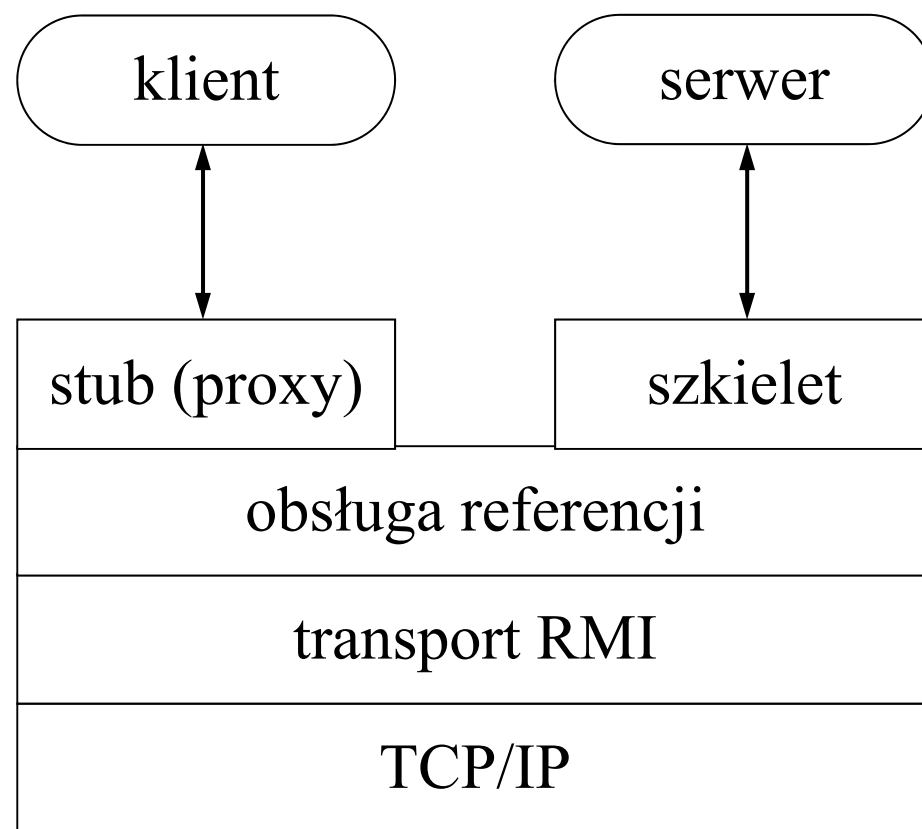
Przekazywanie przez referencję — przykład w Java RMI



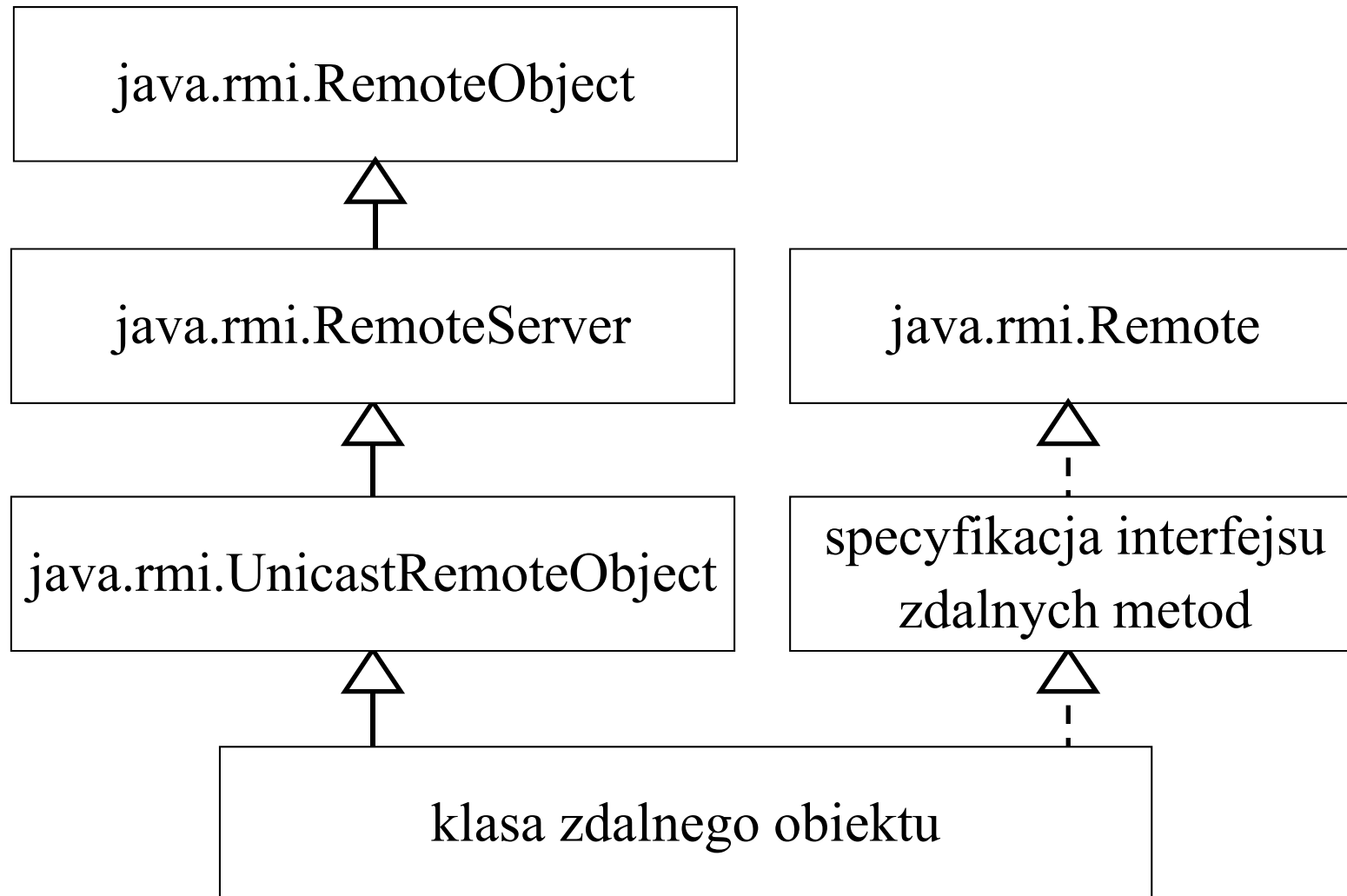
Przekazywanie przez kopiowanie i odtwarzanie



Architektura RMI



Hierarchia klas w definicji zdalnego obiektu



Tworzenia aplikacji rozproszonej w środowisku Java RMI (1)

1. Zdefiniowanie i implementacja odpowiednich klas (w szczególności klas dla obiektów dostępnych zdalnie)
 - zdefiniowanie interfejsu pochodnego od `Remote`
 - zdefiniowanie klasy wywiedzionej z klasy `java.rmi.server.UnicastRemoteObject`, implementującej interfejs pochodny od `Remote` lub użycie statycznej metody `exportObject` klasy `UnicastRemoteObject`
2. Kompilacja źródeł (`javac`, `rmic`)
 - `javac xxx.java ⇨ xxx.class`
 - `rmic xxx ⇨ xxx_Stub.class
xxx_Skel.class`

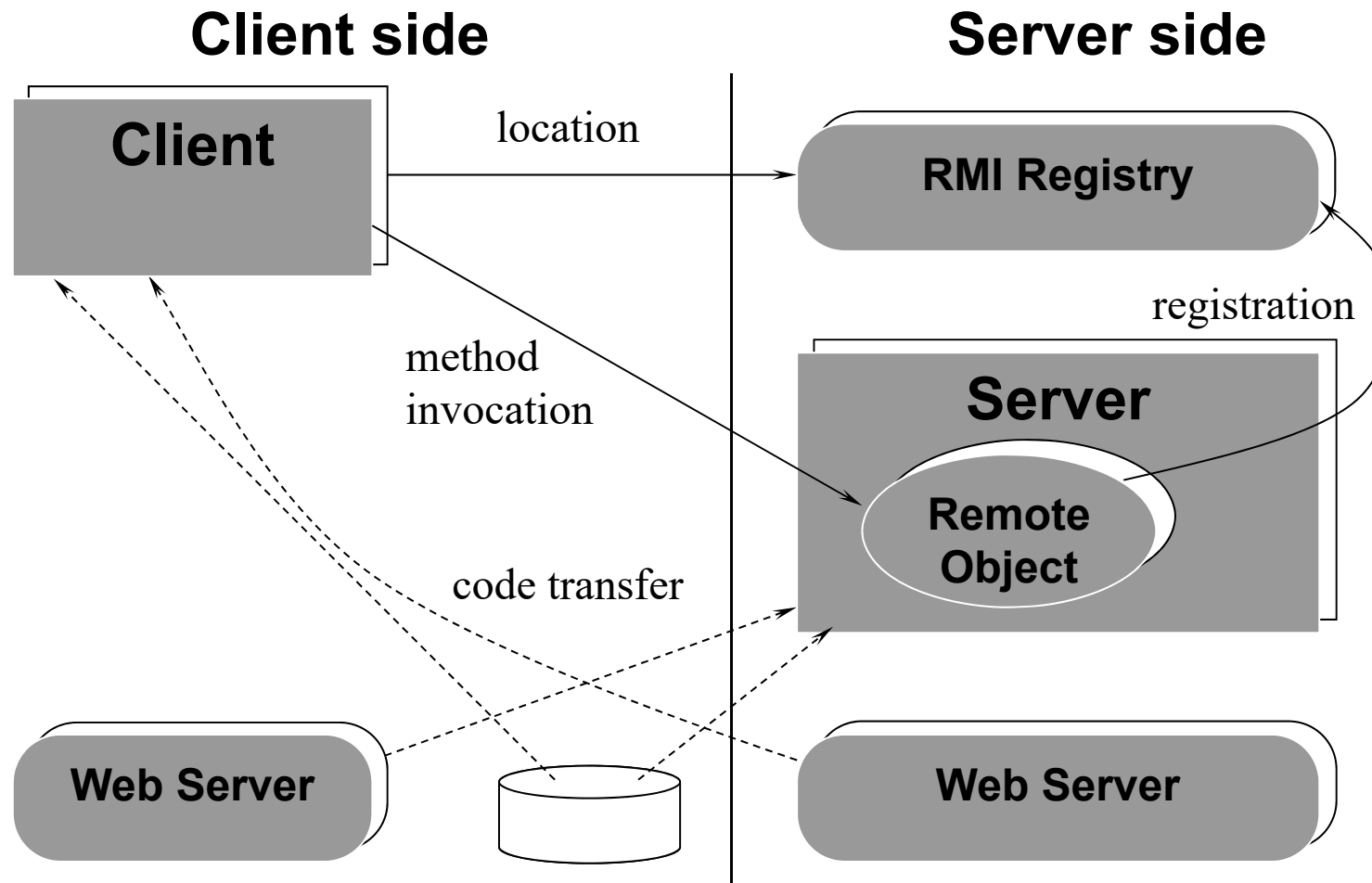
zbędne od Java 5

we wcześniejszych
wersjach Javy

Tworzenia aplikacji rozproszonej w środowisku Java RMI (1)

3. Udostępnienie wygenerowanego kodu klas
 - wspólny system plików
 - kopia kodów klas w różnych systemach plików
 - udostępnianie kodu przez serwer www
4. Uruchomienie aplikacji
 - uruchomienie `rmiregistry` (name server)
 - uruchomienie serwera: utworzenie zdalnych obiektów i ich rejestracja w `rmiregistry`
 - uruchomienie klienta: zlokalizowanie zdalnych obiektów (odwołanie do `rmiregistry`) i wywoływanie zdalnych metod

Komunikacja między klientem a serwerem



Interfejs `java.rmi.Remote`

Komunikacja pomiędzy serwerem (obiektom) a klientem jest określona przez definicję interfejsu pochodnego od interfejsu `Remote`. Klasa zdalnego obiektu musi implementować ten interfejs

```
import java.rmi.Remote;  
import java.rmi.RemoteException;  
  
public interface Compute extends Remote {  
    Object executeTask(Task t) throws RemoteException;  
}
```

Zdalne udostępnianie obiektu (serwer)

Definicja klasy obiektu:

```
import java.rmi.*;
import java.rmi.server.*;
import compute.*;

public class ComputeEngine extends UnicastRemoteObject
                           implements Compute {
    public ComputeEngine() throws RemoteException {
        super();
    }
    public Object executeTask(Task t) {
        return t.execute();
    }
}
```

Utworzenie i rejestracja zdalnego obiektu

```
public static void main(String[] args) {  
    if (System.getSecurityManager() == null) {  
        System.setSecurityManager(new RMISecurityManager());  
    }  
    String name = "//host:1099/Compute";  
    try {  
        Compute engine = new ComputeEngine();  
        Naming.rebind(name, engine);  
        System.out.println("ComputeEngine ok");  
    } catch (Exception e) {  
        System.err.println("ComputeEngine excep."  
                           + e.getMessage());  
        e.printStackTrace();  
    }  
}
```

Zdalne wywoływanie metod

```
import java.rmi.*;
public class launchComp {
    public static void main(String args[]) {
        if (System.getSecurityManager() == null){
            System.setSecurityManager(
                new RMISecurityManager());
        }
        try {
            String name = "//"+args[0]+"/Compute";
            Compute comp = (Compute) Naming.lookup(name);
            CompArea task = new CompArea(...);
            BigDecimal area =
                (BigDecimal) (comp.executeTask(task));
            System.out.println(area);
        } catch (Exception e) {
            ...
        } } }
```


Przekazywanie parametrów

- Jeżeli przekazujemy obiekt przez wartość, to klasa tego obiektu musi implementować interfejs `Serializable`.
- Jeśli przekazujemy obiekt przez referencja, to klasa tego obiektu musi implementować interfejs `Remote`.

```
public interface Task extends  
                                java.io.Serializable {  
    Object execute();  
}
```

```
public class CompArea implements Task {  
    :  
}
```

Lokalizowanie zdalnych obiektów

- Rejestr — wiąże z nazwami i udostępnia zdalne obiekty
- Tworzenie rejestru — `rmiregistry [<port number>]`
- Klasa `LocateRegistry` — umożliwia tworzenie rejestru (obiekty klasy `Registry`)
- Klasa `Registry` — klasa obiektu-rejestru
- Klasa `Naming` — ułatwia korzystanie z rejestru, umożliwiając jego specyfikację w adresie URL

Klasa LocateRegistry

static

Registry createRegistry(int port)
throws RemoteException

static

Registry createRegistry(String host, int port)
throws RemoteException

static

Registry getRegistry(int port)
throws RemoteException

static

Registry getRegistry(String host, int port)
throws RemoteException

Klasa Registry

```
void bind(String name, Remote obj)
    throws RemoteException,
        AlreadyBoundException, AccessException
void rebind(String name, Remote obj)
    throws RemoteException, AccessException
void unbind(String name)
    throws RemoteException,
        NotBoundException, AccessException
String[] list()
    throws RemoteException, AccessException
Remote lookup(String name)
    throws RemoteException,
        NotBoundException, AccessException
```



Klasa Naming (1)

static

void **bind**(String urlName, Remote obj)

throws RemoteException,

AlreadyBoundException, AccessException,

MalformedURLException, UnknownHostException

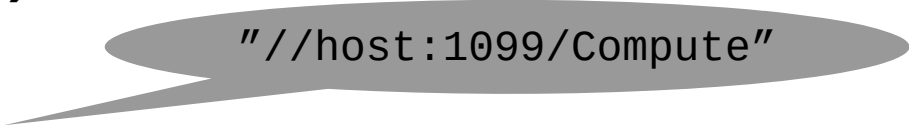
static

void **rebind**(String urlName, Remote obj)

throws RemoteException, AccessException,

MalformedURLException, UnknownHostException

Klasa Naming (2)



"/host:1099/Compute"

static

void **unbind**(String urlName)

throws RemoteException, NotBoundException,
AccessException, MalformedURLException,
UnknownHostException

static

String[] **list**(String urlName)

throws RemoteException, AccessException
MalformedURLException, UnknownHostException

static

Remote **lookup**(String urlName)

throws RemoteException, NotBoundException,
AccessException, MalformedURLException

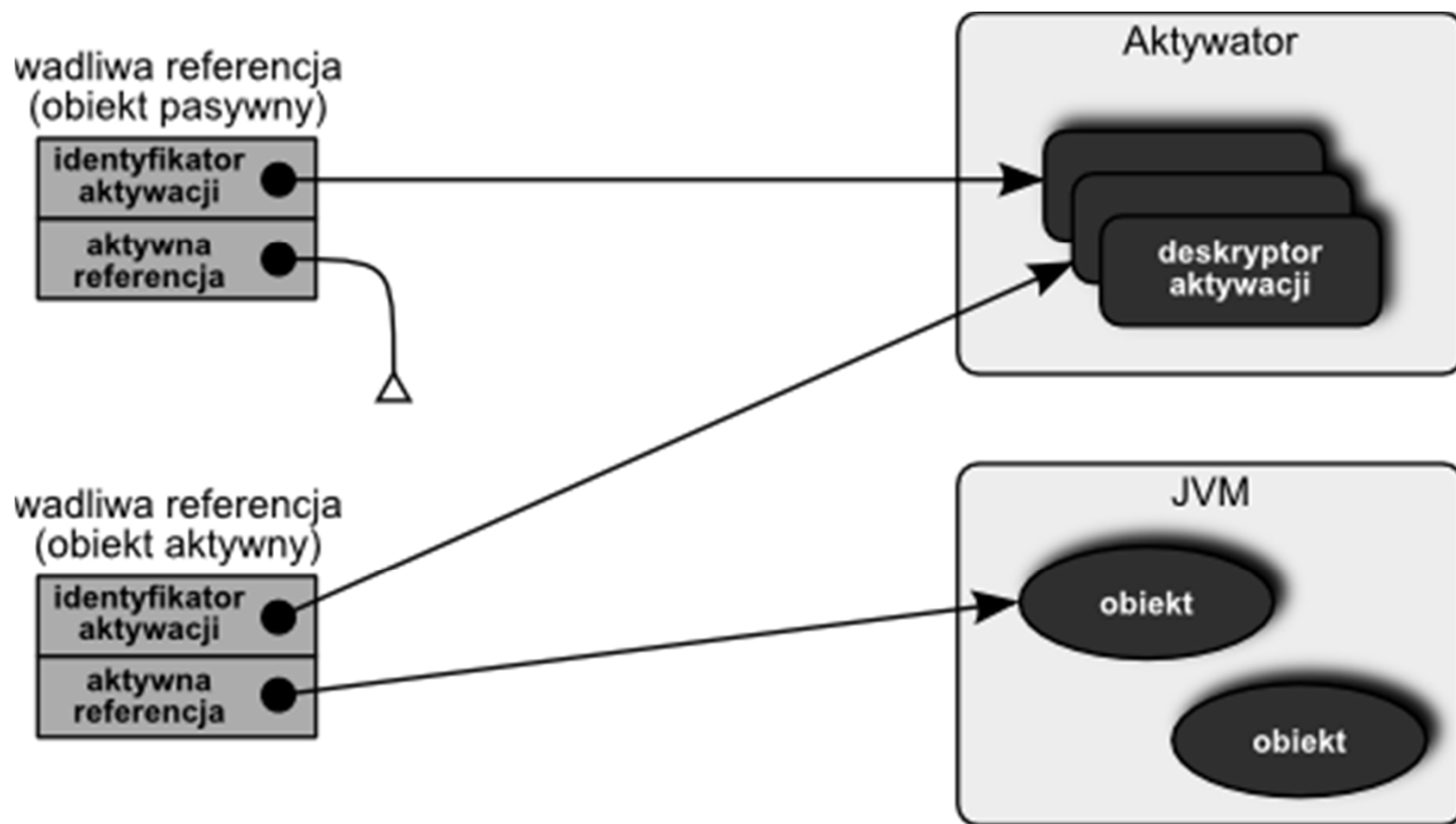
Obiekty aktywowalne

- Obiekt aktywowalny jest to taki obiekt, którego instancja może w danej chwili nie istnieć w żadnej maszynie wirtualnej pomimo istniejącej referencji do niego.
- W momencie odniesienia do obiektu (czyli wywołania jednej z jego metod) instancja może zostać utworzona w działającej lub specjalnie w tym celu uruchomionej maszynie wirtualnej.
- Obiekt aktywowalny jest obiektem klasy pochodnej od `Activatable` (`java.rmi.activation`) lub obiektem jawnie obsługiwanym na potrzeby aktywacji przez odpowiednie metody statyczne klasy `Activatable`.

Uaktywnianie obiektu

- Obiekt aktywowalny może być w stanie:
 - aktywnym — jest on eksportowany (udostępniony zdalnie), a jego instancja istnieje w maszynie wirtualnej,
 - pasywnym — jego instancja nie istnieje (albo nie jest udostępniona zdalnie), ale może zostać utworzona i wyeksportowana w reakcji na odniesienie do niego.
- Tworzenie obiektu aktywowalnego:
 - rejestrowanie obiektu w systemie aktywacji,
 - eksportowanie obiektu — udostępnienie zdalnym klientom.
- Uaktywnienie (eksportowanie) obiektu zdalnego polega na uruchomieniu maszyny wirtualnej, w której następnie tworzona jest instancja tego obiektu.

Wadliwa referencja (ang. faulty reference)



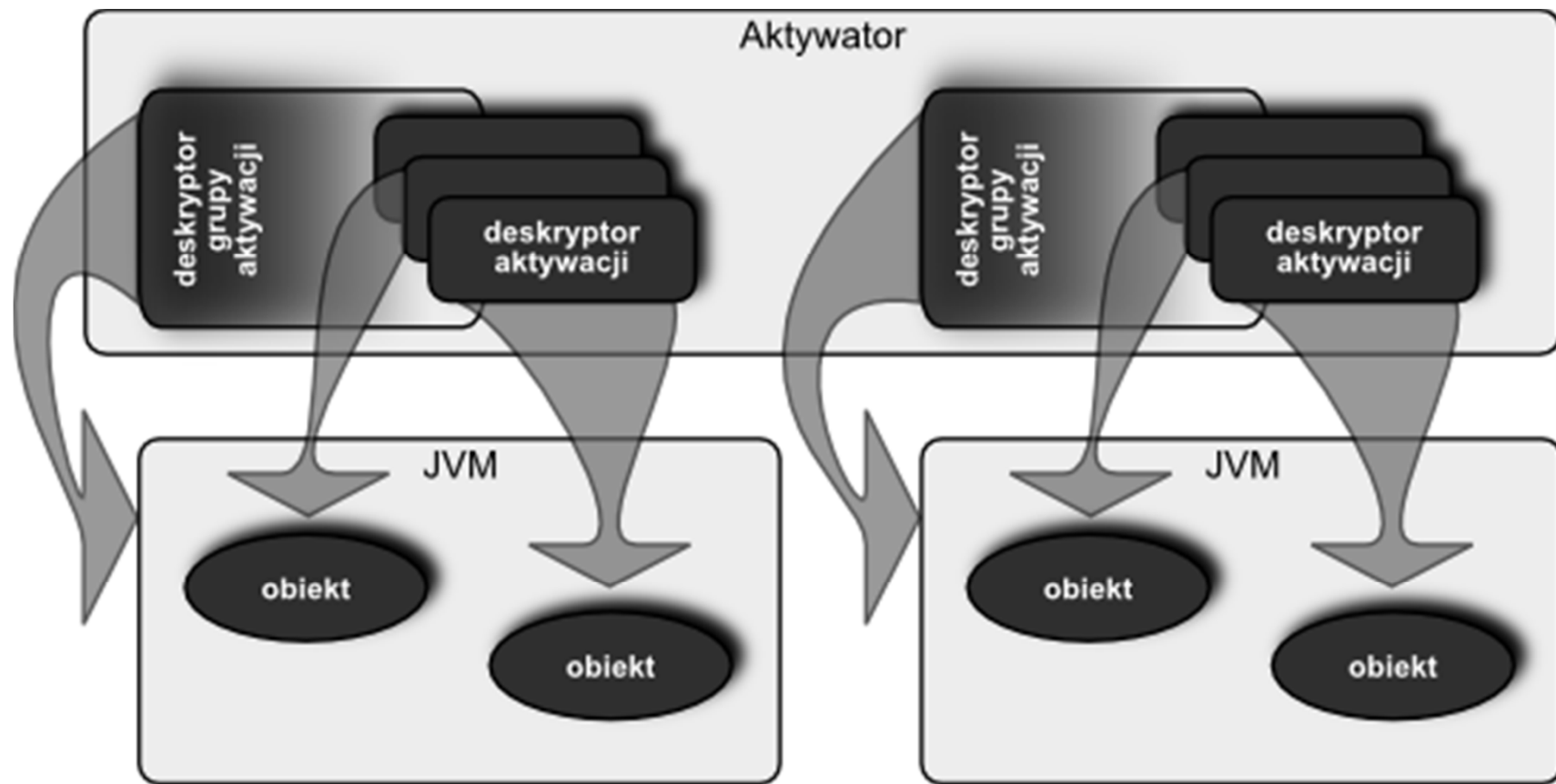
Grupa aktywacji

- Żeby mechanizm aktywacji mógł zaktywować obiekt, zarówno maszyna wirtualna, jak i sam obiekt muszą być odpowiednio opisane.
- Opis maszyny wirtualnej związany jest z *grupą aktywacji*. Instancje obiektów należących do tej samej grupy tworzone są w tej samej maszynie wirtualnej.
- *Deskryptor grupy aktywacji* (ActivationGroupDesc) dostarcza informacji niezbędnych do zidentyfikowania lub uruchomienia właściwej maszyny wirtualnej (JVM).

Aktywator

- Aktywator (np. `rmid`) nadzoruje aktywację obiektów
 - utrzymuje bazę informacji o obiektach aktywowalnych — *deskryptory aktywacji* (klasa `ActivationDesc`) identyfikowane przez *identyfikatory aktywacji* (klasa `ActivationID`),
 - zarządza maszynami wirtualnymi, w których udostępniane są obiekty.
- Aktywator funkcjonuje według następujących zasad:
 - działa zawsze podczas pracy systemu,
 - nie aktywuje (reaktywuje) obiektów, które są już aktywne.

Aktywator i deskryptory aktywacji



Deskryptor aktywacji

(klasa `ActivationDesc`)

- Deskryptor aktywacji dostarcza aktywatorowi informacji niezbędnych do utworzenia instancji aktywowanego obiektu.
- Przechowuje on następujące informacje opisujące obiekt:
 - identyfikator grupy aktywacji obiektu
 - nazwa klasy obiektu
 - ścieżka do implementacji obiektu (codebase URL path)
 - dane inicjalizujące (`MarshaledObject<?>`)

Identyfikator aktywacji

(klasa `ActivationID`)

- Id. aktyw. zawiera informacje o obiekcie aktywowalnym:
 - zdalną referencję do aktywatora obiektu,
 - unikalny identyfikator obiektu.
- Identyfikator aktywacji powstaje w wyniku rejestracji obiektu w systemie aktywacji.
- Rejestracja obiektu w systemie aktywacji odbywa się w jeden z następujących sposobów:
 - przez wywołanie metody `Activatable.register`,
 - przez użycie odpowiedniego konstruktora klasy `Activatable`, który rejestruje i eksportuje obiekt,
 - przez wywołanie `Activatable.exportObject`

Message Oriented Middleware

Systemy kolejkiowania komunikatów

Plan

1. Koncepcja
 - ↳ paradygmat kolejowania (punkt-punkt)
 - ↳ paradygmat publish/subscribe
2. Model systemu
3. Przykłady rozwiązań
4. JMS

Cechy MOM

- Uniezależnienie funkcjonowania składników aplikacji od dostępności informacji o interfejsach innych składników
- Uniezależnienie funkcjonowania warstwy komunikacyjnej (kanału komunikacyjnego) od działania (obecności) komunikujących się procesów $\Rightarrow$ komunikacja nieustanna
- Mechanizm komunikacji pośredniej
 - ↳ oparty na identyfikacji miejsca pośredniczącego (tzw. skrzynki, a nie adresu nadawcy/odbiorcy)
 - ↳ komunikujące się strony nie muszą znać się wzajemnie
- Łatwość wdrożenia komunikacji asynchronicznej

Koncepcja komunikacji oparta na kolejkowaniu

- ☛ Komunikacja *punkt-punkt*
- ☛ Organizacja warstwy komunikacyjnej w postaci systemu kolejek (ang. queue) {skrzynka $\equiv$ kolejka} realizowanych w oparciu o zasoby pamięci, w tym pamięci dyskowej (gwarancja trwałości na wypadek awarii $\Rightarrow$ niezawodność)
- ☛ Udostępnienie mechanizmów komunikacji polegających na:
 - umieszczeniu komunikatów w kolejkach,
 - pobieraniu komunikatów z kolejek

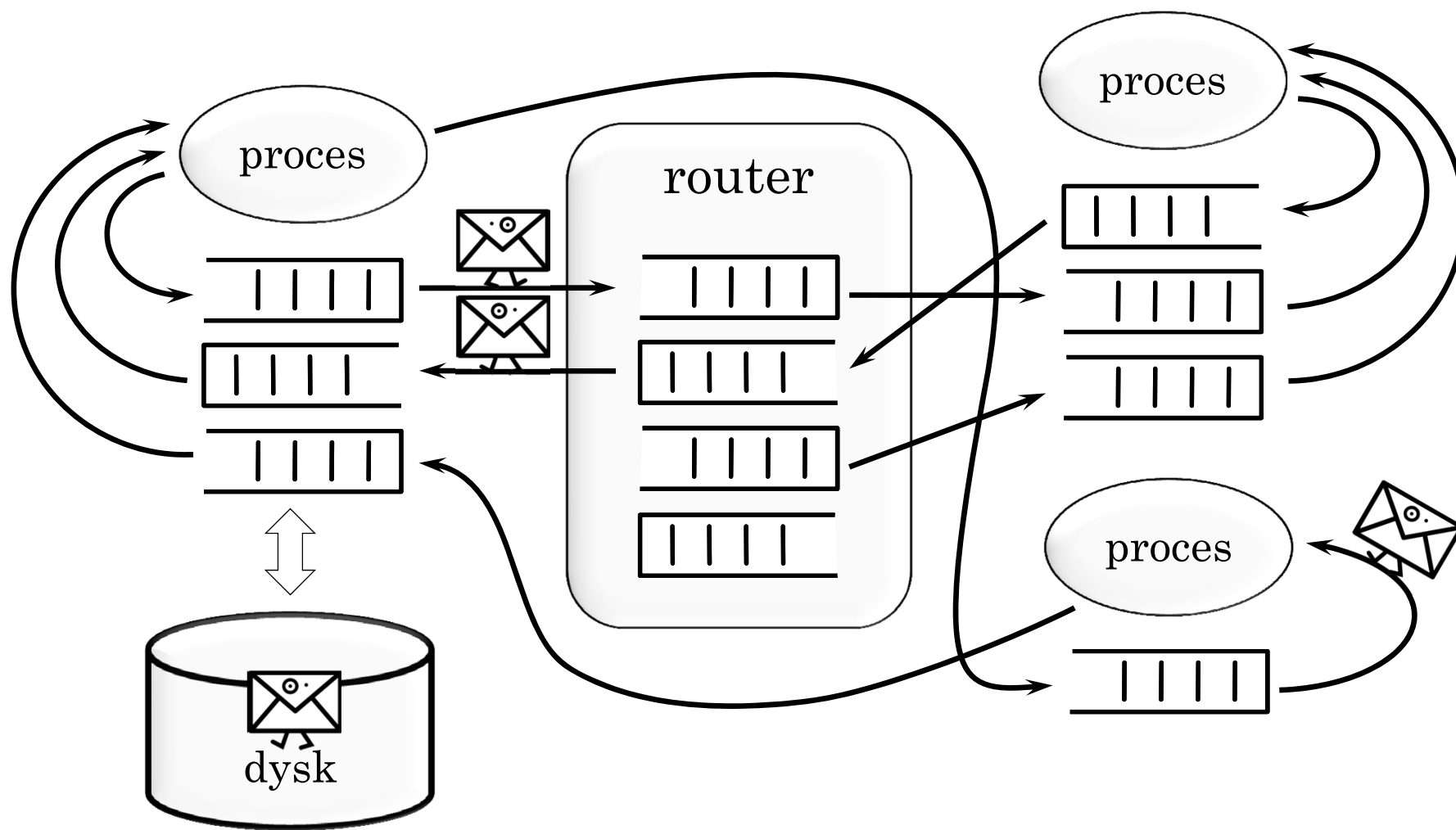
Paradygmat publish/subscribe

- Komunikacja *jeden do wielu* (potencjalnie *wielu do wielu*)
- Strona publikująca (nadawca) udostępnia treść związaną z określonym tematem (ang. topic)
{skrzynka $\equiv$ temat}
- Środowisko komunikacyjne (usługa) przekazuje treść udostępnionych wiadomości odbiorcom (subskrybentom), którzy zarejestrowali (zapisali) się wcześniej na dany temat

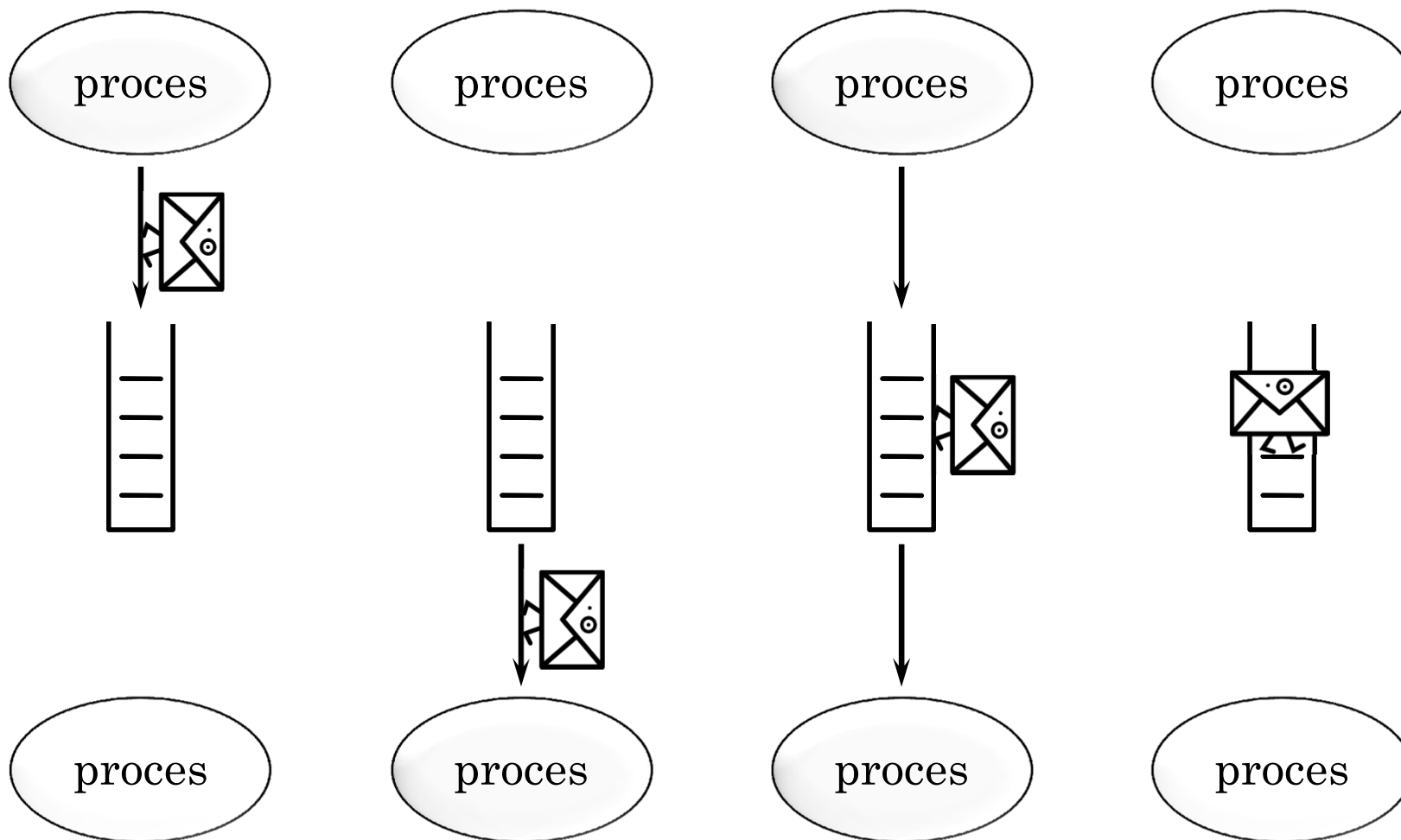
Model systemu typu MOM — podstawowe pojęcia

- Wiadomość (ang. message) — porcja danych (często z dodatkowymi własnościami) składowana w kolejce
- Kolejka (ang. queue) — miejsce przechowywania wiadomości (komunikatów) {temat $\cong$ kolejka dla wielu odbiorców, ang. multiconsumer queue}
- Proces (ang. process) — element aplikacji, zlecający operacje dotyczące wiadomości w kolejce
- Zarządca zbioru kolejek — moduł (proces na określonym węźle, dostawca, *broker*), odpowiedzialny za wykonywanie operacji na kolejkach, np. tworzenie, usuwanie, lokalizowanie kolejek, definiowanie atrybutów kolejek itp.)

Funkcjonowanie systemu kolejkowania komunikatów



Warianty komunikacji



Przykłady rozwiązań typu MOM

- ☛ WebSphere MQ (IBM MQSeries , XMS — Message Service Client)
- ☛ Microsoft Message Queuing (MSMQ)
- ☛ Java Message Service (JMS) — standard, specyfikacja interfejsu. Implementacje oparte na JMS:
 - ☛ Sun Java System Message Queue (OpenMQ — wersja open source)
 - ☛ Apache ActiveMQ
- ☛ Oracle Advanced Queueing
- ☛ RabbitMQ
- ☛ ZeroMQ
- ☛ Apache Kafka

Java Message Service

na podstawie slajdów Cezarego Sobańca

Historia JMS

- Opracowany w 1998
- Pierwotny cel: dostęp do istniejących systemów kolejkowania wiadomości (tzw. MOM — Message Oriented Middleware, np. IBM MQSeries)
- Integralna część Java EE od wersji 1.3

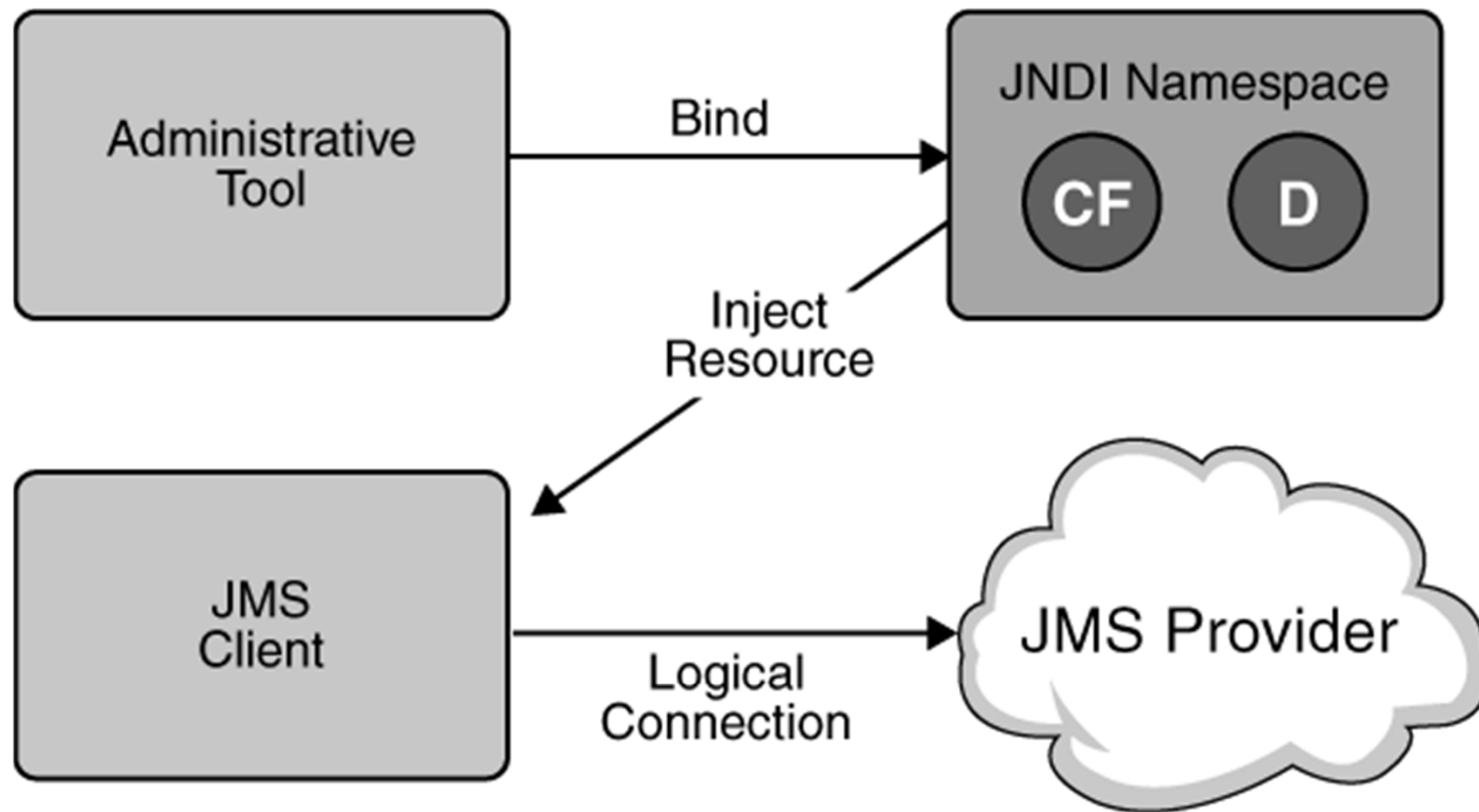
Komponenty JMS

- Dostawca JMS (ang. JMS provider) — implementacja interfejsów JMS, administracja, sterowanie
- Klienci JMS — aplikacje i komponenty wysyłające i odbierające komunikaty
- Wiadomości — obiekty do przenoszenia informacji
- Obiekty zarządzania (ang. administered objects) – prekonfigurowane obiekty na potrzeby zarządzania:
 - ↳ cele (ang. destinations)
 - ↳ fabryki połączeń (ang. connection factories)

Funkcjonalność JMS

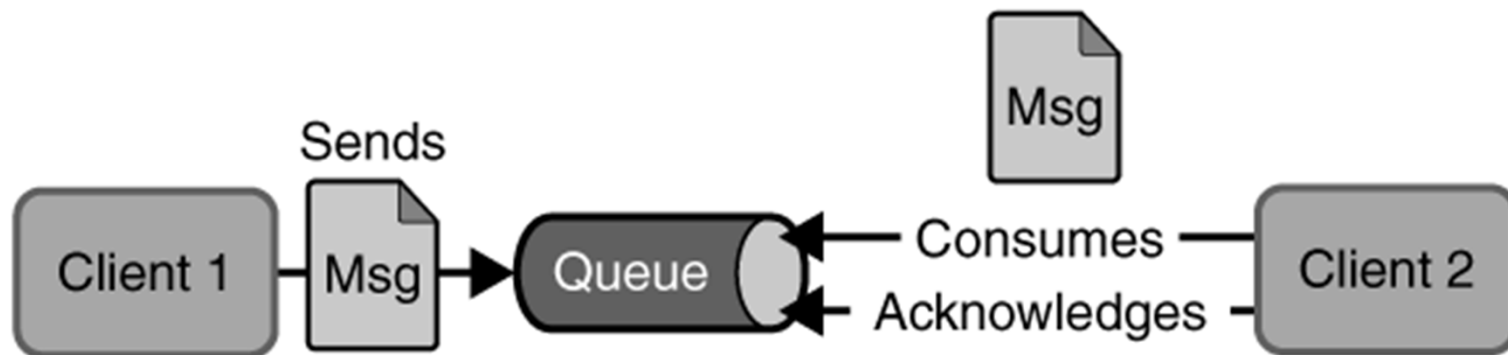
- Nieustanna, niezawodna, asynchroniczna komunikacja międzyprocesowa
- Transakcyjna interakcja z dostawcą JMS
- Modele komunikacji (messaging domains)
 - ↳ punkt-punkt (ang. point-to-point)
 - ↳ subskrypcji (ang. publish/subscribe)

Architektura JMS



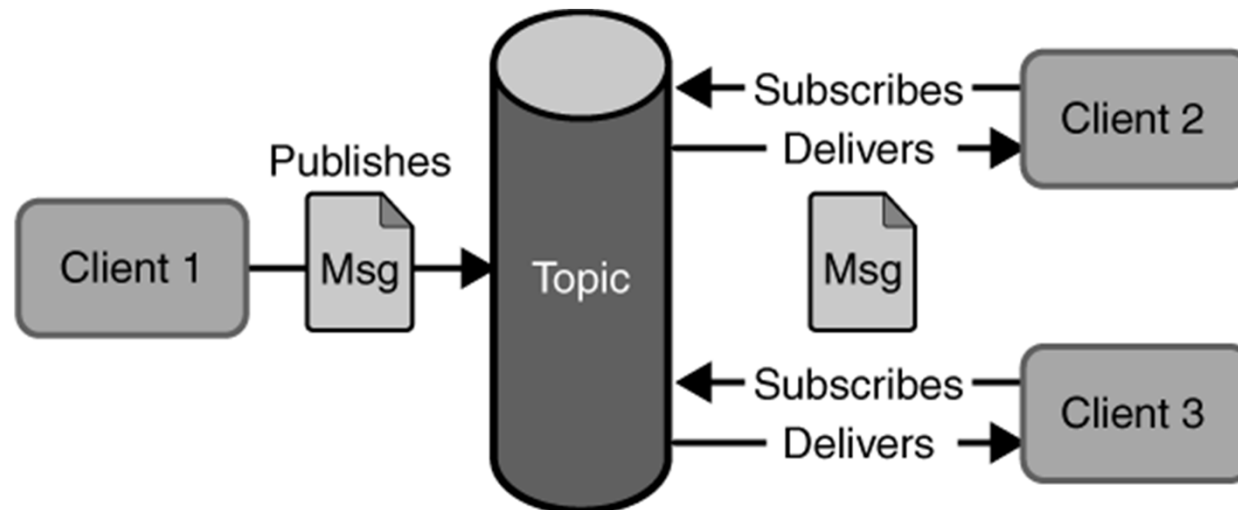
Model punkt-punkt

- Wysyłanie i odbiór poprzez dostęp do kolejki
- Wiadomości pozostają w kolejce do czasu odbioru lub przedawnienia
- Każda wiadomość ma 1 konsumenta
- Brak ograniczeń czasowych



Model subskrypcji

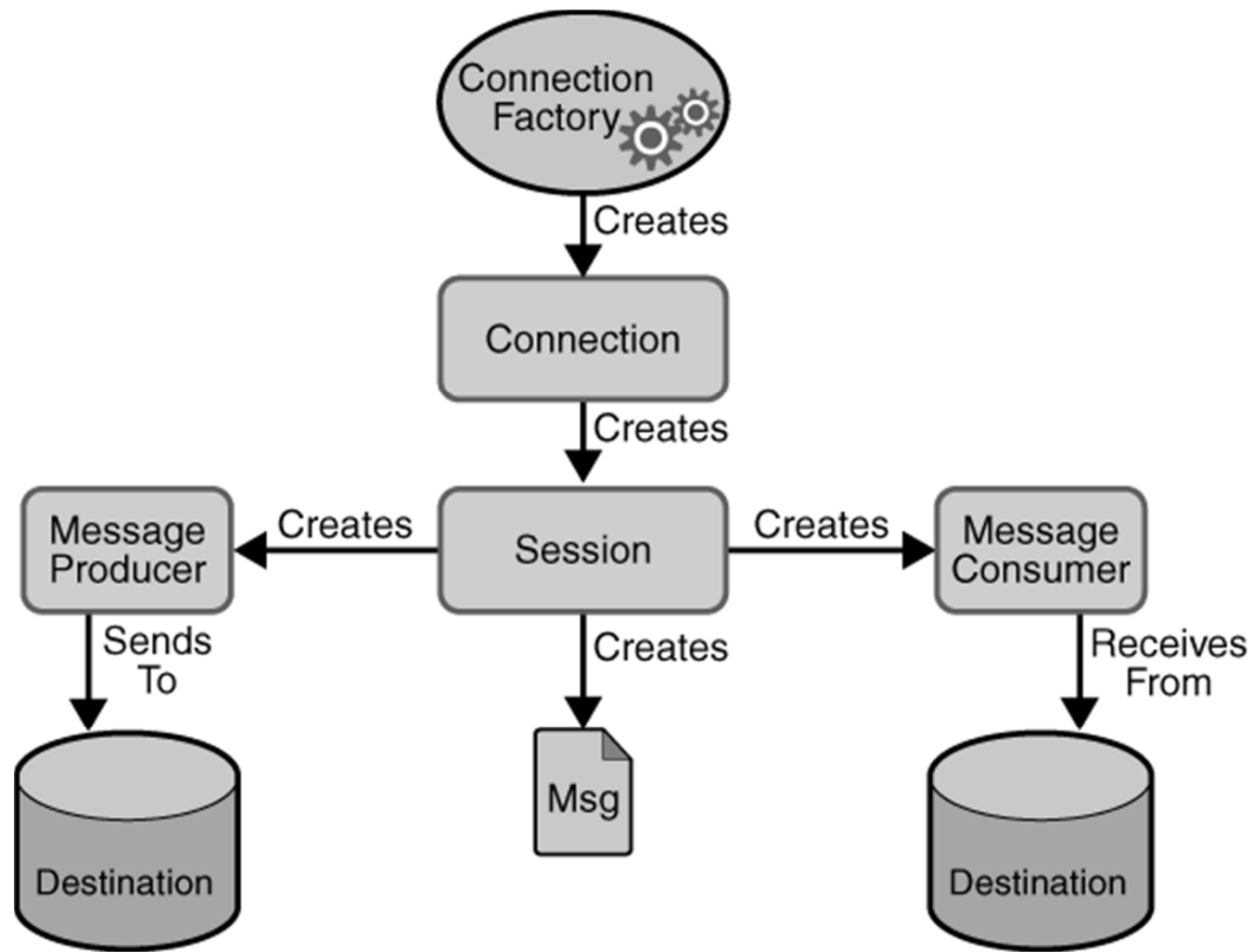
- Każda wiadomość może mieć wielu konsumentów
- Wiadomości są dostarczane do wszystkich aktualnych subskrybentów a następnie są usuwane
- Nie można odczytać wiadomości sprzed subskrypcji
- Trwała subskrypcja (ang. durable subscription) – odbiór wiadomości z okresu nieaktywności klienta



Model odbioru (konsumpcji) wiadomości

- ☛ Konsumpcja synchroniczna – blokująca metoda `receive()` z (ewentualnym) ograniczeniem czasowym
- ☛ Konsumpcja asynchroniczna – odbiornik wiadomości (ang. `message listener`) – asynchroniczne wywołanie metody `onMessage()`

Model programistyczny JMS



Fabryki połączeń

- ☛ Ogólna: `ConnectionFactory` (interfejs bazowy dla poniższych)
- ☛ Dla kolejek: `QueueConnectionFactory`
- ☛ Dla tematów: `TopicConnectionFactory`

Destinations

- ☛ *queue* w przypadku komunikacji punkt-punkt
- ☛ *topic* w przypadku modelu subskrypcji

```
@Resource(mappedName="jms/Queue")  
private static Queue queue;
```

```
@Resource(mappedName="jms/Topic")  
private static Topic topic;
```

Połączenie (ang. connection)

- Reprezentacja wirtualnego połączenia z dostawcą JMS
- Połączenie jest potrzebne do zainicjowania sesji
- Połączenie musi być jawnie zamknięte na końcu aplikacji (zwolnienie zasobów, m.in. otwartych sesji)
- Rozpoczęcie odbioru wiadomości: metoda `start()`
- Wstrzymanie odbioru wiadomości: metoda `stop()`

```
Connection connection =  
    connFactory.createConnection();  
connection.start();  
...  
connection.close();
```

Sesje

- Jednowątkowy kontekst do tworzenia i odbioru wiadomości
- Sesje tworzą:
 - ↳ wiadomości
 - ↳ producentów (nadawców) wiadomości
 - ↳ konsumentów (odbiorców) wiadomości

```
Session session = connection.createSession(false,  
    Session.AUTO_ACKNOWLEDGE);
```

- Pierwszy argument wskazuje czy ma być tworzona transakcja

Producenci wiadomości

```
@Resource(mappedName="jms/Queue")  
private static Queue queue;
```

```
MessageProducer producer =  
    session.createProducer(queue);  
producer.send(message);
```

➤ Wysyłanie do dowolnej kolejki:

```
MessageProducer producer =  
    session.createProducer(null);  
producer.send(queue, message);
```

Odbiorcy wiadomości

```
@Resource(mappedName="jms/Queue")  
private static Queue queue;
```

```
MessageConsumer consumer =  
    session.createConsumer(queue);  
Message m1 = consumer.receive();  
Message m2 = consumer.receive(1000);
```

☛ Message listeners:

```
MessageListener myListener = new AListener();  
consumer.setMessageListener(myListener);
```

- ☛ Listener ma metodę `onMessage(Message)`. Musi obsługiwać wszystkie wyjątki.

Filtry wiadomości

- Filtr jest wyrażeniem zapisywanym jak warunki w SQL92
 - ↳ `typ = 'wyniki' and id = '120'`
- Wyrażenie odwołuje się do właściwości wiadomości
- Filtr może być parametrem tworzenia konsumenta wiadomości

Wiadomości

- ☛ Wiadomości składają się z
 - ↳ nagłówek
 - ↳ listy właściwości
 - ↳ ciało
- ☛ Standardowe właściwości (przechowywane w nagłówku)
 - ↳ identyfikator JMSMessageID
 - ↳ odbiorca JMSDestination
 - ↳ znacznik czasowy JMSTimestamp
 - ↳ priorytet JMSPriority
 - ↳ typ JMSType

Zawartość wiadomości (body)

- `TextMessage` – wiadomość tekstowa (np. dok. XML)
- `MapMessage` – zbiór par nazwa-wartość (`String` i typ prymitywny)
- `BytesMessage` – nieinterpretowany strumień bajtów
- `StreamMessage` – strumień wartości prymitywnych typów
- `ObjectMessage` – serializowalny obiekt Javy
- `Message` – pusta zawartość

```
TextMessage message = session.createTextMessage();  
message.setText("Ala ma kota");  
producer.send(message)
```

```
...
```

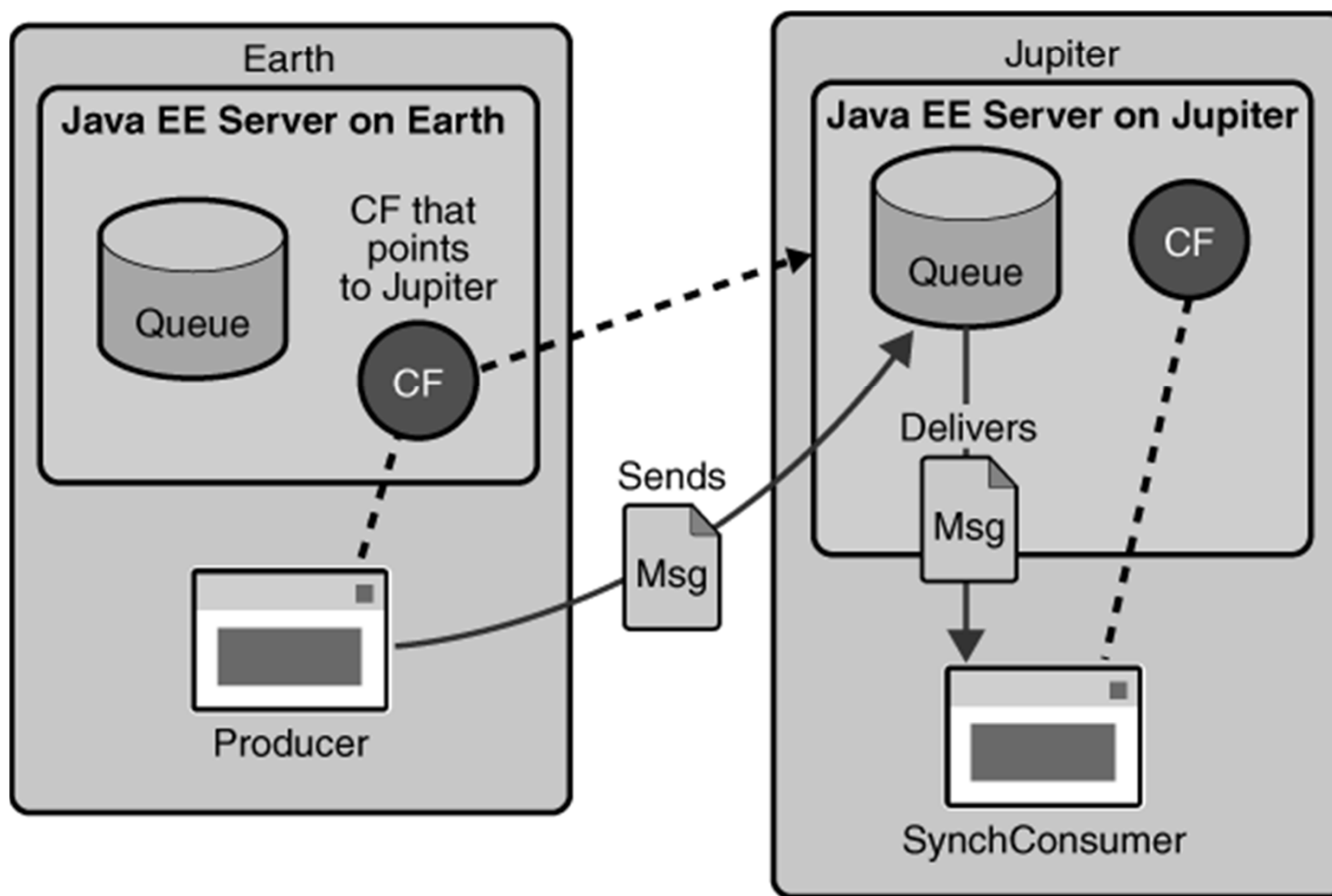
```
Message m = consumer.receive();  
if (m instanceof TextMessage) { ... m.getText(); }
```

Przeglądanie zawartości kolejki

- ☛ Interface QueueBrowser
- ☛ Możliwość zastosowania filtru
- ☛ Nie można przeglądać topic'ów (wiadomości znikają)

```
QueueBrowser browser =  
    session.createBrowser(queue);  
Enumeration msgs = browser.getEnumeration();  
while (msgs.hasMoreElements()) {  
    Message m = (Message)msgs.nextElement();  
    ...  
}
```

Przesyłanie wiadomości między systemami



Niezawodność komunikacji

- ☛ Potwierdzenia wiadomości
- ☛ Trwałość komunikatów – awarie dostawców JMS
- ☛ Priorytety komunikatów
- ☛ Czas życia komunikatów
- ☛ Tymczasowe kolejki

Potwierdzenia

- Po odbiorze wiadomości
 - Po przetworzeniu wiadomości
 - Po odbiorze potwierdzenia
 - Wiadomości są automatycznie potwierdzane po zakończeniu transakcji
 - Wycofanie transakcji → ponowne dostarczenie
-
- `AUTO_ACKNOWLEDGE` – po odbiorze
 - `CLIENT_ACKNOWLEDGE` – jawne potwierdzenie wszystkich odebranych wiadomości w ramach sesji
 - `DUPS_OK_ACKNOWLEDGE` – leniwe potwierdzanie z możliwością powstawania duplikatów

Potwierdzenie

- ☛ Wiadomości niepotwierdzone przed końcem sesji są dostarczane ponownie (przy kolejnym połączeniu)
- ☛ Przechowywanie niepotwierdzonych wiadomości dla trwałych subskrypcji

Potwierdzenie tylko przetworzonych wiadomości:

- ☛ asynchroniczny odbiór i tryb `AUTO_ACKNOWLEDGE`
- ☛ odbiór synchroniczny i tryb `CLIENT_ACKNOWLEDGE`
- ☛ odbiór synchroniczny w trybie `AUTO_ACKNOWLEDGE` powoduje natychmiastowe potwierdzanie (przed przetworzeniem)

Trwałość wiadomości

- Tryb PERSISTENT – każda wiadomość jest rejestrowana w pamięci trwałej (tryb domyślny)
- Tryb NON_PERSISTENT – wiadomość może zostać utracona w przypadku awarii dostawcy JMS (większa wydajność)
- Własność trwałości może być ustawiana dla producenta wiadomości lub dla pojedynczej wiadomości

```
producer.setDeliveryMode(DeliveryMode.NON_PERSISTENT);  
producer.send(msg, DeliveryMode.NON_PERSISTENT, 3, 10000);
```

Priorytety wiadomości

- ☛ Priorytet może być ustawiany dla producenta wiadomości lub dla pojedynczej wiadomości
- ☛ 0 – najniższy priorytet, 9 – najwyższy, domyślnie 4
- ☛ Priorytet określa preferencje i nie decyduje o bezwzględnej kolejności dostarczania

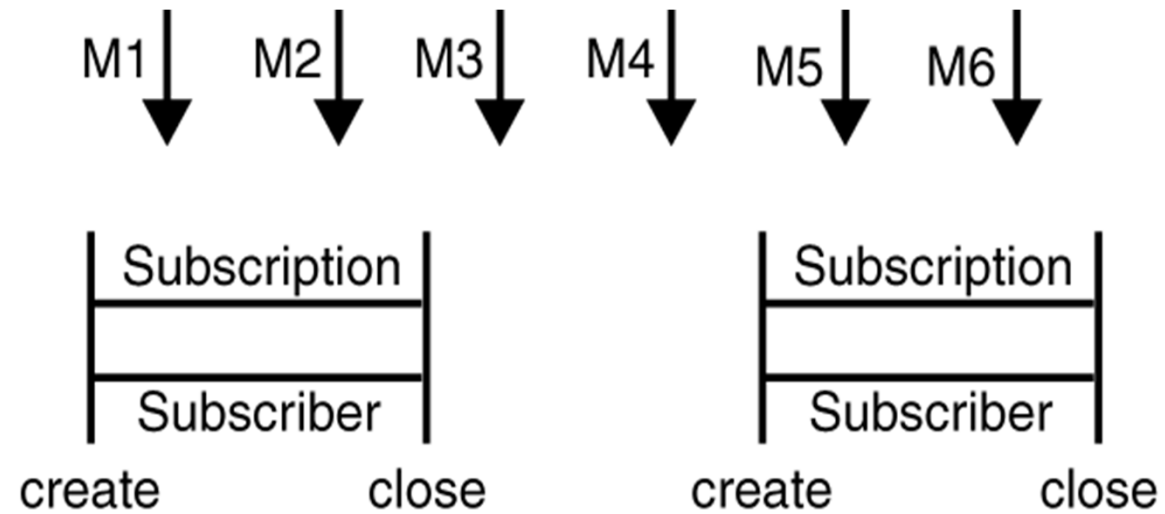
```
producer.setPriority(5);  
producer.send(msg,  
DeliveryMode.NON_PERSISTENT,5,10000);
```


Przedawnianie wiadomości

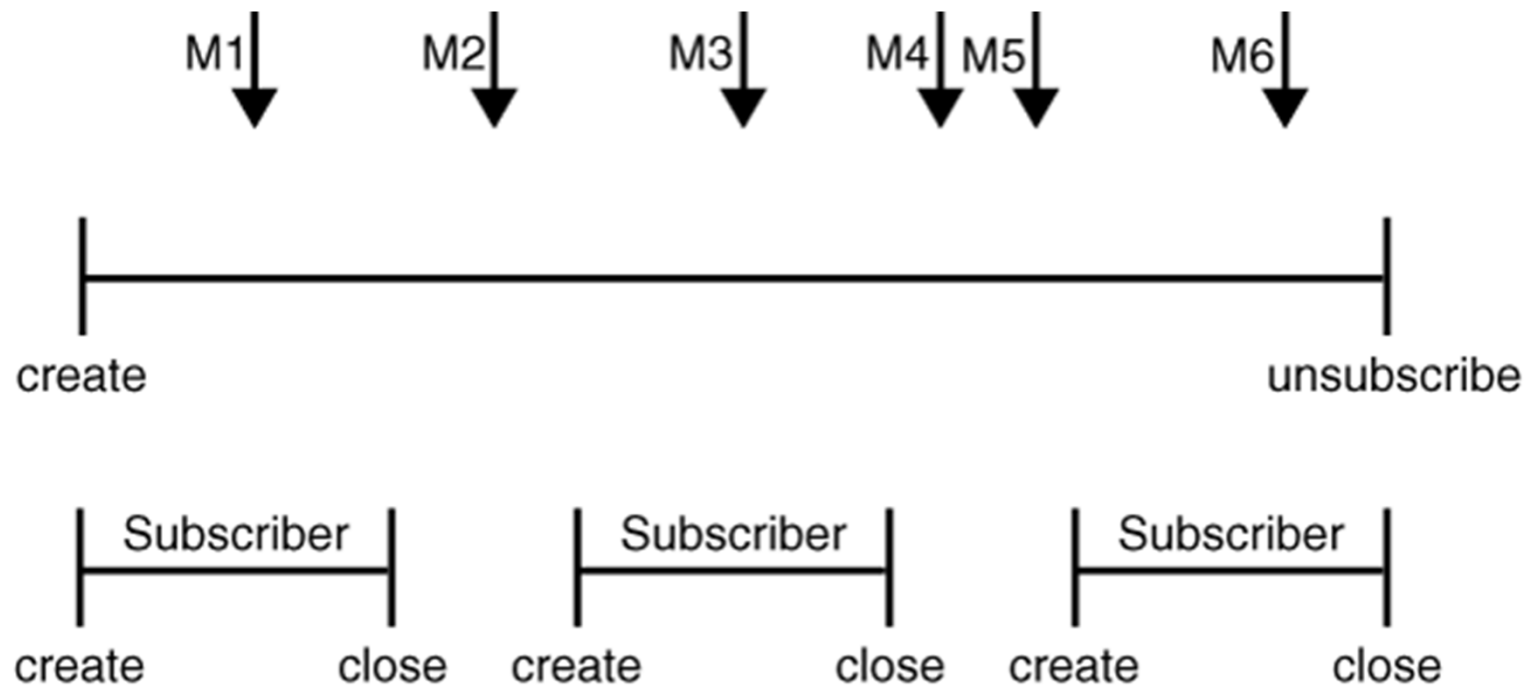
- Czas życia może być ustawiany dla producenta wiadomości lub dla pojedynczej wiadomości
- Domyślnie wiadomości nie ulegają przedawnieniu
- Czas życia 0 oznacza brak przedawniania

```
producer.setTimeToLive(10000);  
producer.send(msg,  
DeliveryMode.NON_PERSISTENT,5,10000);
```

Trwałe subskrypcje



Trwałe subskrypcje (2)



Trwałe subskrypcje (3)

- Identyfikacja subskrypcji
 - ↳ identyfikator klienta
 - ↳ topic
 - ↳ nazwa subskrypcji

```
connection.setClientID("MyId");  
...  
MessageConsumer topicSubscriber =  
    session.createDurableSubscriber(myTopic,  
    "MySub");  
...  
topicSubscriber.close();  
...  
session.unsubscribe("MySub");
```

Lokalne transakcje

- Grupowanie operacji wysyłania/odbioru w transakcji
- Metody `Session.commit()` i `Session.rollback()`
- Zatwierdzenie oznacza wysłanie wyprodukowanych wiadomości i potwierdzenie odebranych
- Wycofanie oznacza usunięcie wyprodukowanych wiadomości i ponowne dostarczenie wiadomości odebranych (z pominięciem przedawnionych)
- Uwaga na zakleszczenia: wysłanie następuje po zatwierdzeniu

Kolejki tymczasowe

- Utworzenie kolejki tymczasowej
`TemporaryQueue tq =
session.createTemporaryQueue();`
- Dołączenie kolejki tymczasowej do komunikatu
`msg.setJMSReplyTo(tq);`
- Uzyskiwanie dostępu do kolejki tymczasowej (po odebraniu komunikatu)
`tq = msg.getJMSReplyTo();`

Przestrzeń krotek

Linda — ogólna koncepcja

- Mechanizm komunikacji międzyprocesowej zaproponowany przez Davida Gelerntera w 1985
- Luźne powiązanie komunikujących się procesów (nie muszą znać się wzajemnie, nie muszą działać jednocześnie — komunikacja nieustanna)
- Asocjacyjna identyfikacja komunikatów (w odróżnieniu od kolejkowania) we współdzielonej przestrzeni
- Współczesne implementacje:
 - ↳ JavaSpaces — Sun Microsystems (w ramach technologii Jini, projekt przejęty przez ASF — Apache River), komercyjna implementacja dostarczana przez Gigaspaces
 - ↳ TSpaces — IBM

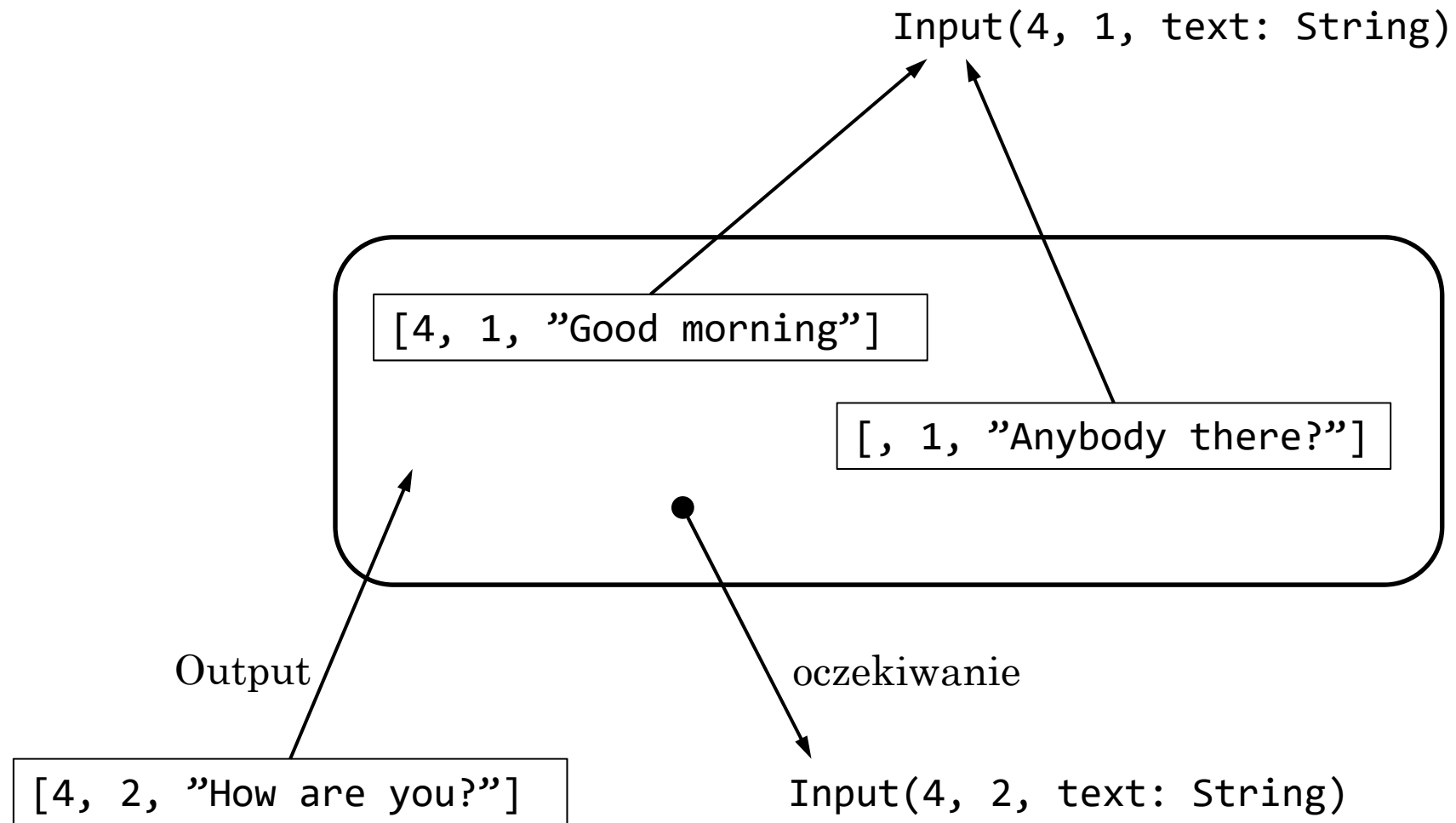
Linda — podstawowe pojęcia

- Krotka — uporządkowana kolekcja danych określonych typów — atrybutów, przy czym atrybuty mogą (ale nie muszą) mieć nadaną konkretną wartość
- Przestrzeń krotek — wspólne miejsce dostępne dla kooperujących procesów, gdzie gromadzone są krotki
- Interfejs dostępu do przestrzeni krotek
 - ↳ Output — umieszczanie krotki w przestrzeni
 - ↳ Input — pobieranie krotki z przestrzeni
 - ↳ Read — odczytywanie krotki bez pobierania (odczytana krotka w dalszym ciągu pozostaje w przestrzeni)
 - ↳ Try_Input, Try_Read — nieblokujące wersje Input i Read

Linda — realizacja operacji dostępu

- Krotka staje się dostępna w przestrzeni po wykonaniu operacji Output, której parametrami są wartości atrybutów, np.:
`Output(4, 1, "Good morning")`
- Operacja Input powoduje pobranie (usunięcie z przestrzeni) krotki, której wartości atrybutów są zgodne z parametrami operacji, np.:
`Input(4, 1, text: String)`
Wartości pozostałych atrybutów zostaną nadane zgodnie z zawartością pobranej krotki.
- Jeśli krotka została umieszczona w przestrzeni bez podania wartości któregoś z atrybutów (np.
`Output(, 1, "Anybody there?")`
może ona zostać pobrana przez Input z dowolną wartością tego atrybutu lub pominięciem tej wartości.

Linda — operacje na przestrzeni krotek



Linda — specyfikacja komunikacji

- Przestrzeń krotek jest zbiorem (nie kolejką) — krotki nie są uporządkowane i mogą być odbierane w innej kolejności niż były umieszczane.
- W realizacji operacji dostępu gwarantowana jest ogólnie rozumiana żywotność:
 - Jeśli procesy czekają (w operacji Input) na krotkę, to przy odpowiednio dużej liczbie umieszczonych krotek z oczekiwanymi wartościami atrybutów każdy w końcu ją otrzyma.
 - Jeśli krotka jest w przestrzeni, to po odpowiednio dużej liczbie operacji Input (ze zgodnymi parametrami) zostanie w końcu odczytana.

JavaSpaces (przestrzeń obiektów)

- Odpowiednikiem krotki jest obiekt klasy implementującej interfejs `Entry`.
- Zmienne instancji obiektu-krotki muszą być publiczne.
- Obiekt-krotka ma ustalony czas życia w przestrzeni (czas wynajmowania przestrzeni — `lease time`).
- Czas życia można zwiększać nawet po umieszczeniu obiektu-krotki w przestrzeni.
- Operacje na przestrzeni krotek realizowane są poprzez metody `write`, `take`, `read`, `takeIfExists`, `readIfExists` interfejsu `JavaSpace`.

Przykład definicji klasy dla obiektu-krotki

```
public class Message implements Entry {  
    public Integer id;  
    public Integer num;  
    public String text;  
  
    public Message () {}  
    public Message (Integer i, Integer n, String t){  
        id = i;  
        num = n;  
        text = t;  
    }  
}
```

Przykład umieszczania krotki w przestrzeni

```
JavaSpace space = getSpace();  
msg = new Message ( new Integer(4),  
                    new Integer(counter++),  
                    "Good morning");  
Lease l = space.write(msg, null, 6*60*60*1000);
```

Przykład pobierania krotki z przestrzeni

```
JavaSpace space = getSpace();  
template = new Message ( new Integer(4),  
                           null,  
                           null);  
Message msg = (Message)space.read( template,  
                                     null,  
                                     60*60*1000);
```


Ada 95 — współbieżność

Współpraca (synchronizacja i komunikacja) zadań za pomocą spotkań asymetrycznych i obiektów chronionych

Współbieżność w Adzie — zadania

- ➡ Zadania są jednostkami strukturalizacji programów współbieżnych.
- ➡ Zadanie jest obiektem typu zadaniowego. W programie można utworzyć wiele zadań na podstawie tego samego typu zadaniowego. Wszystkie obiekty danego typu mają jednakową specyfikację (interfejs) i tę samą implementację.
- ➡ Obiekty zadaniowe są tworzone tak jak inne obiekty języka: mogą być deklarowane statycznie lub kreowane dynamicznie.

Komunikacja pomiędzy zadaniami

- ☞ komunikacja synchroniczna między parą zadań — mechanizm spotkań asymetrycznych (ang. rendez-vous),
- ☞ komunikacja asynchroniczna między wieloma zadaniami — obiekty chronione

Specyfikacja zadania

```
task Zad is
```

```
...
```

```
entry E1 (p1: in t1; p2: out t2;);
```

```
...
```

```
private
```

```
entry E2 (...);
```

```
...
```

```
end Zad;
```

Implementacja zadania

```
task body Zad is
    ...
accept E1 (p1: in t1; p2: out t2;)
do
    ...
end E1;
...
accept E2 (...)
do
    ...
end E2;
...
end Zad;
```

Specyfikacja typu zadaniowego

```
task type T is
    . . .
    entry E1 ( . . . ) ;
    . . .
end T;
```

Przykład zastosowania typu zadaniowego

```
tablica: array(1..5) of T;
```

```
type R is
```

```
record
```

```
    Zadanie: T;
```

```
    ...
```

```
end record;
```

```
...
```

```
R1, R2 : R;
```

Spotkania asymetryczne

- ➡ W komunikacji między zadaniami z wykorzystaniem mechanizmu spotkań istotne jest wyróżnienie roli, jaką może w danej chwili pełnić zadanie — asymetria.
- ➡ W różnych momentach czasu zadanie może być bierne (serwer) — jeśli udostępnia lub jest gotowe udostępnić usługi identyfikowane przez nazwy wejść – lub czynne (klient) — jeśli wywołuje właśnie wejście jakiegoś serwera.

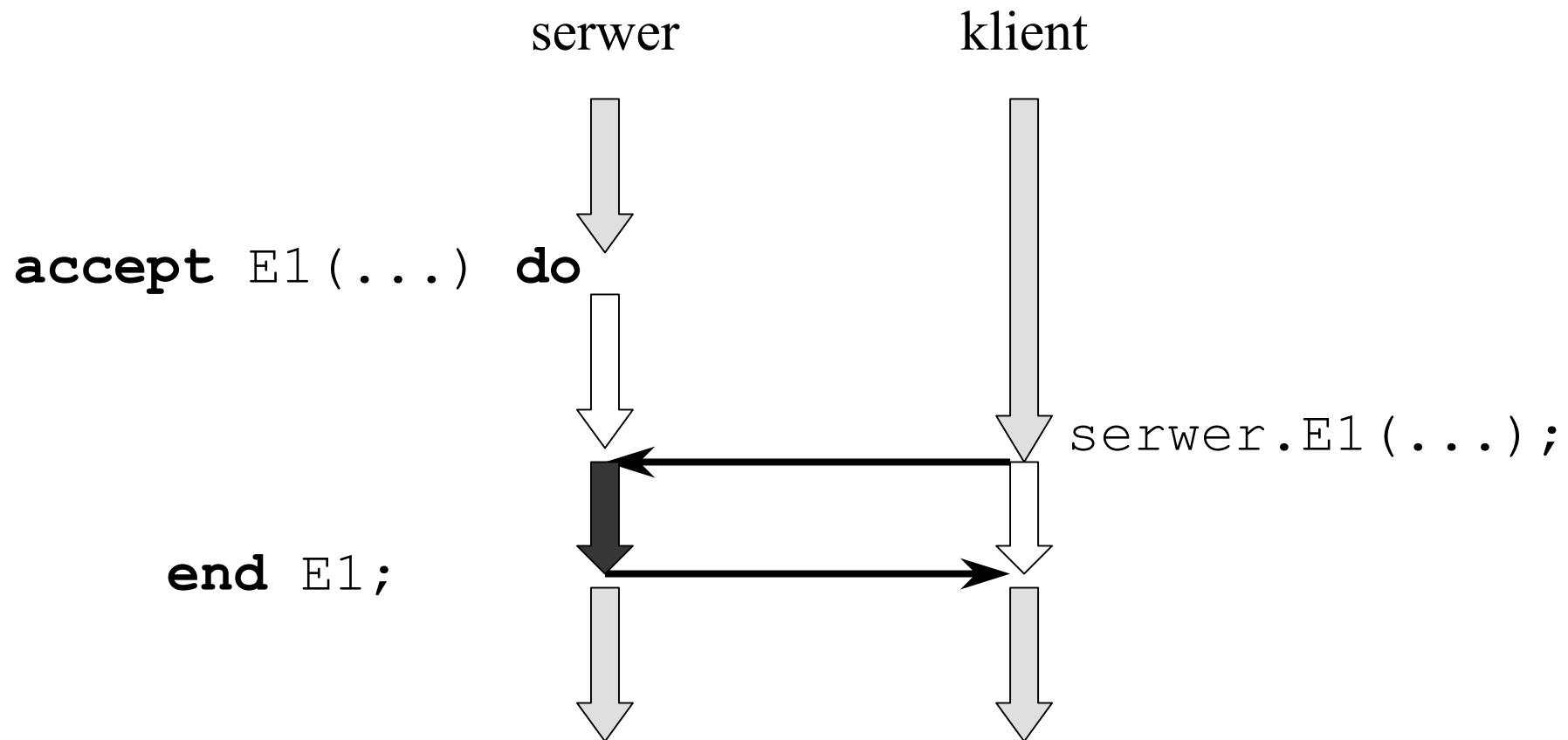
Wywołanie wejścia

```
Nazwa_zadania.wejście(param, ...);
```

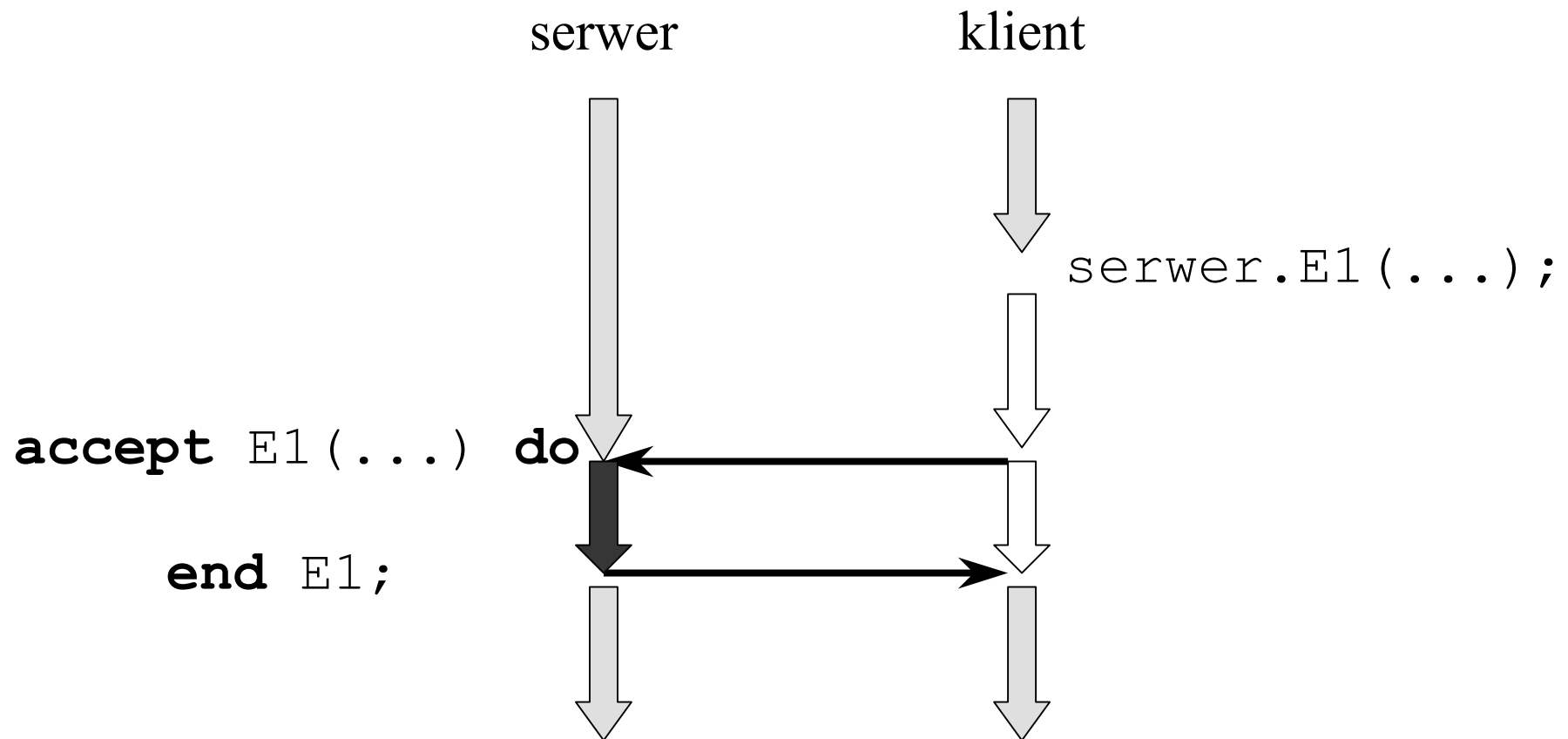
Obsługa wejścia

```
accept wejście (param: typ) do  
    ...  
end wejście;
```

Spotkanie — oczekiwanie serwera



Spotkanie — oczekiwanie klienta



Instrukcja select

- ➡ Oczekiwanie selektywne
- ➡ Terminowe wywołanie wejścia
- ➡ Warunkowe wywołanie wejścia
- ➡ Asynchroniczna zmiana wątku sterowania

Oczekiwanie selektywne

Umożliwia po **stronie serwera**:

- 👉 oczekiwanie na więcej niż jedno spotkanie — alternatywa,
- 👉 oczekiwanie na rozpoczęcie spotkania w ustalonym odcinku czasu,
- 👉 wycofanie oferty spotkania, jeżeli nie może ono nastąpić natychmiast,
- 👉 zakończenie istnienia zadania, jeżeli nie istnieją klienci, którzy wywołują jego wejścia

Alternatywa

```
task body T is
begin
  loop
    select
      accept E1 (...) do
        ...
      end;
    or
      accept E2 (...) do
        ...
      end;
    end select;
  end loop;
end T;
```

Alternatywa z gałęziami dozorowanymi

```
task body T is
begin
  loop
    select
      when i > 0 => accept E1 (...) do
        ...
      end;
    or
      when i = 0 => accept E2 (...) do
        ...
      end;
    end select;
  end loop;
end T;
```


Obsługa kolejki żądań

- ➡ Przy braku pragmatów przyjmowana jest strategia FIFO obsługi kolejki żądań.
- ➡ Jeśli w momencie osiągnięcia instrukcji **select** zadania-klienci czekają w kolejkach kilku wejść, to wybór kolejki jest **niedeterministyczny**.

Dozory (1)

- ➡ Każda gałąź **accept** instrukcji **select** może być poprzedzona wyrażeniem logicznym, nazywanym dozorem (guard).
- ➡ Gałęzie bez dozorów równoważne są gałęziom z dozorami `True`.
- ➡ Gałęzie, dla których dozór jest spełniony, są nazywane gałęziami otwartymi (pozostałe – zamkniętymi).

Dozory (2)

- ➡ Wykonanie instrukcji **select** rozpoczyna się od obliczenia dozorów wszystkich gałęzi.
- ➡ Tylko gałęzie otwarte brane są pod uwagę w trakcie dalszego wykonania instrukcji **select**.
- ➡ Wartości wszystkich dozorów obliczane są każdorazowo tylko raz na początku wykonania instrukcji **select**.
- ➡ Jeśli wszystkie gałęzie chronione są dozorami, a ich wartości są równe `False`, to generowany jest wyjątek `Program_Error`.

Przeterminowanie spotkań

```
task body T is
begin
  loop
    select
      accept E1 (...) do
        ...
      end;
    or
      delay 5.0; -- możliwe również delay until
      exit; -- wyjście z pętli
    end select;
  end loop;
end T;
```

Gałąź **else**

```
task body T is
begin
  loop
    select
      accept E1 (...) do
        ...
      end;
    else
      exit; -- wyjście
    end select;
  end loop;
end T;
```

Gałąź **else** pozwala serwerowi wycofać ofertę spotkania, gdy brak jest zadań-klientów już oczekujących na spotkanie. Gałąź **else** może wystąpić instrukcji **select** tylko raz i nie może być chroniona dozorem.

Gałąź **terminate**

```
task body T is
begin
  loop
    select
      accept E1 (...) do
        ...
      end;
    or
      terminate;
    end select;
  end loop;
end T;
```

Gałąź **terminate** jest wybierana gdy jednostka macierzysta zadania zakończyła się i wszystkie zadania potomne albo zakończyły się, albo gotowe są wybrać gałąź **terminate**. Gałąź **terminate** może być poprzedzona dozorem. Nie może występować jednocześnie z gałęzią **delay** lub **else**.

Terminowe wywołanie wejścia

umożliwia po **stronie klienta** oczekiwanie na rozpoczęcie spotkania w ustalonym odcinku czasu.

select

```
server.E1 ( . . . ) ;
```

```
-- tu mogą być jeszcze jakieś instrukcje
```

or

```
delay 2.0; -- ewent. delay until
```

```
-- tu mogą być jeszcze jakieś instrukcje
```

end select;

Warunkowe wywołanie wejścia

umożliwia po **stronie klienta** wycofanie oferty spotkania, jeżeli nie może ono nastąpić natychmiast.

select

`server.E1 ( . . . ) ;`

-- tu mogą być jeszcze jakieś instrukcje

else

-- tu mogą być jeszcze jakieś instrukcje

end select;

Asynchroniczna zmiana wątku sterowania

umożliwia przerwanie wykonywania programu po zakończeniu instrukcji wyzwalającej (po ustalonym czasie lub zakończeniu wywołania wejścia itp.).

```
select  
    delay 5.0;  
    Put_line ("Czas minął");  
then abort  
    obliczaj (...);  
end select;
```

Obiekty chronione

- ➡ Obiekt chroniony jest jednostką programową, która organizuje dostęp zadań do grupowanych przez siebie danych współdzielonych.
- ➡ Budowa obiektu chronionego jest podobna do budowy pakietu i zadania – składa się ze specyfikacji i treści implementującej obiekt.
- ➡ Możliwe jest definiowanie typów chronionych.

Struktura obiektu chronionego

- ➡ Specyfikacja obiektu (oraz typu) chronionego zawsze zawiera część publiczną i część prywatną.
- ➡ Część publiczną tworzą deklaracje funkcji, procedur oraz wejść. W części prywatnej występują deklaracje zmiennych współdzielonych i — opcjonalnie — deklaracje wewnętrznych funkcji, procedur i wejść.

Dostępu do obiektu chronionego

- ➡ Dostęp do obiektu chronionego jest możliwy tylko poprzez wywołania funkcji, procedur i wejść publicznych i odbywa się on zgodnie z zasadą wzajemnego wykluczania.
- ➡ Wywołania funkcji pozwalają tylko na odczyt danych współdzielonych (podanych w części `private`), a wywołania procedur i wejść – na ich modyfikowanie.

Blokady

- ➡ W momencie, gdy zadanie wywołuje wejście lub procedurę obiektu chronionego, ten może być zajęty obsługą innego wywołania (zablokowany).
- ➡ Z każdym obiektem chronionym związane są dwie blokady:
 - ↳ do czytania (shared read lock) — aktywna gdy obiekt chroniony obsługuje wywołanie swojej funkcji
 - ↳ do pisania (exclusive read/write lock) — gdy obiekt obsługuje wywołanie procedury lub wejścia.

Synchronizacja dostępu do obiektu chronionego (1)

1. Jeżeli obiekt chroniony ma założoną blokadę read i wywoływana jest jego funkcja, to funkcja ta zostaje wykonana.
2. Jeżeli obiekt chroniony ma założoną blokadę read i wywoływane jest jego wejście lub procedura, to wywołanie jest opóźniane, dopóki są zadania aktywne wewnątrz obiektu chronionego.

Synchronizacja dostępu do obiektu chronionego (2)

3. Jeżeli obiekt chroniony ma założoną blokadę read/write, to wywołanie jest opóźniane, dopóki są zadania aktywne wewnątrz obiektu chronionego.
4. Jeżeli nadchodzi kolej wykonania wywoływanego wejścia obiektu chronionego lecz bariera ma wartość False, to wywołanie jest ustawiane w kolejce związanej z tą barierą czekając na spełnienie warunku bariery; obiekt nie zostaje jeszcze zablokowany.

Przykład specyfikacji obiektu chronionego

```
protected zmienna_chroniona is  
    function czytaj return zapis;  
    procedure pisz (x: in zapis);  
private  
    element: zapis;      -- zmienna  
    współdzielona  
end zmienna_chroniona;
```


Przykład implementacji obiektu chronionego

```
protected body zmienna_chroniona is  
  function czytaj return zapis is  
  begin  
    return element;  
  end czytaj;  
  procedure pisz (x: in zapis) is  
  begin  
    element:=x;  
  end pisz;  
end zmienna_chroniona;
```

Wejścia obiektu chronionego

- ➡ Wejścia podobnie jak procedury umożliwiają modyfikację obiektu.
- ➡ W części implementacyjnej, z każdym z zadeklarowanych wejść jest związany warunek wykonania wejścia, nazywany barierą (*barrier*).
- ➡ Treść wołanego wejścia jest wykonywana tylko jeśli warunek bariery jest spełniony.

Przykład specyfikacji typu chronionego z wejściami

```
subtype Rozmiar is Integer range 1..Rozmiar_MAX;

protected type Bufor_cykliczny(N: Rozmiar:=100) is
    entry Put(x: in Wartość);
    entry Get(x: out Wartość);
private
    bufor: array(0..N-1) of Wartość; -- bufor N-elem.
    put_ptr: Integer :=0; -- indeks miejsca wstaw.
    get_ptr: Integer :=0; -- indeks miejsca pob.
    licznik: Integer range 0..N :=0 -- zajętość buf.
end Bufor_cykliczny;
```

Rekolejkowanie

Żądanie wejścia może zostać przekazane do kolejki związanej z innym wejściem (zadeklarowanym np. w części prywatnej). Instrukcja **requeue** może pojawić się w obsłudze wejścia zadania lub obiektu chronionego.

requeue E1 ;

Zadania

1. Implementacja semafora binarnego
2. Implementacja problemu ograniczonego buforowania (producenta i konsumenta) z buforem jednoelementowym
3. Implementacja problemu ograniczonego buforowania (producenta i konsumenta) z buforem wieloelementowym
4. Implementacja semafora ogólnego